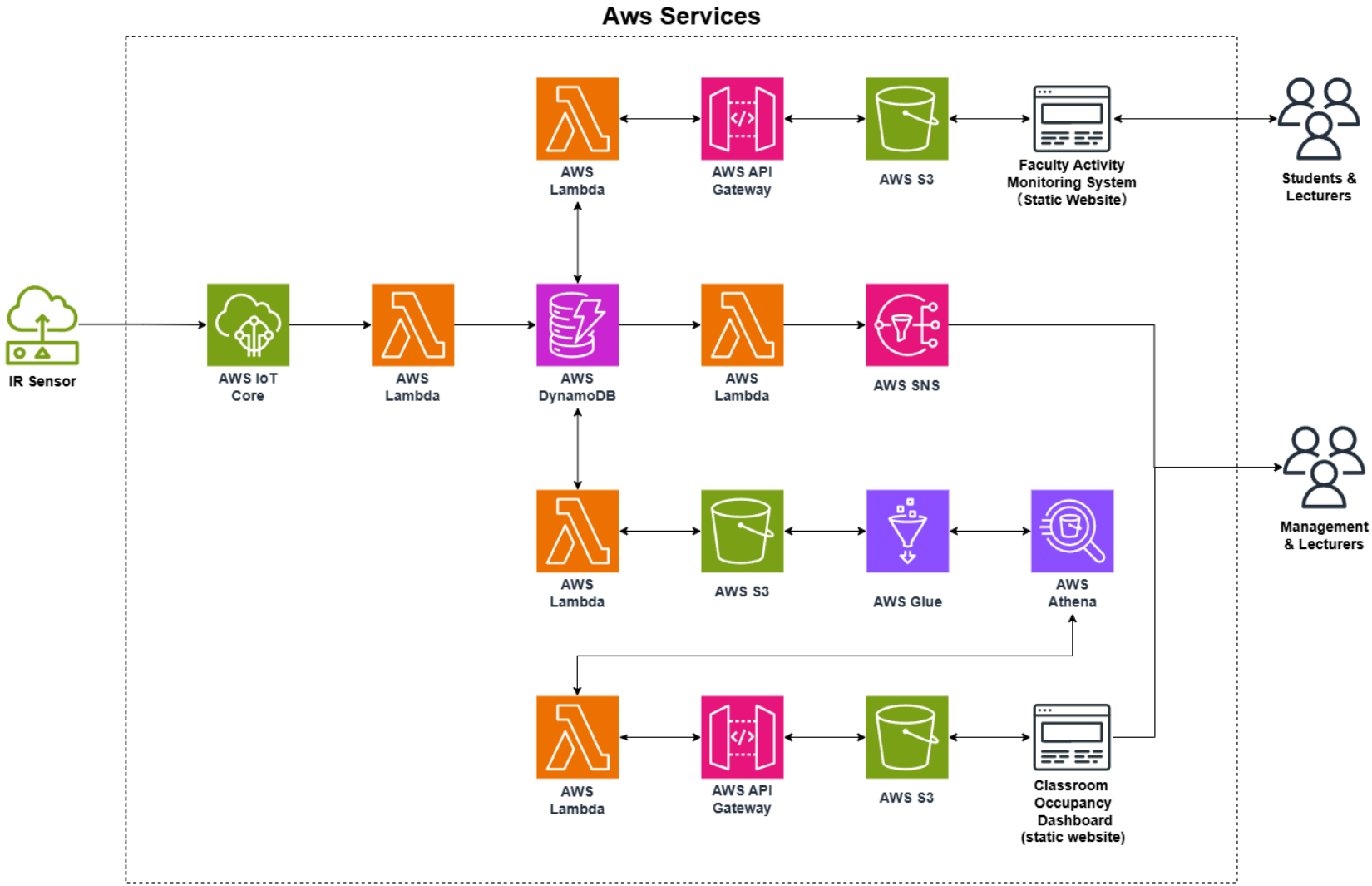
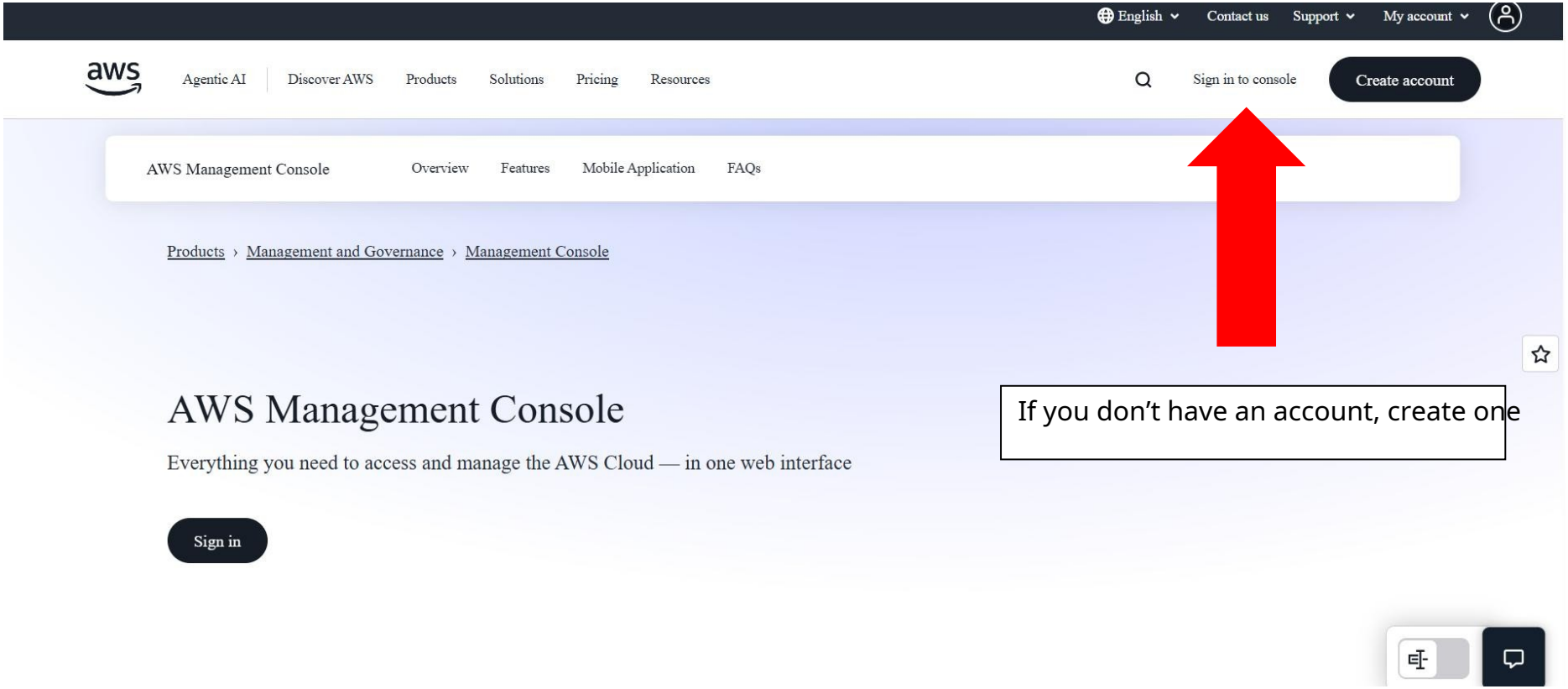
Faculty Activity Monitoring & Space Utilisation Analytics



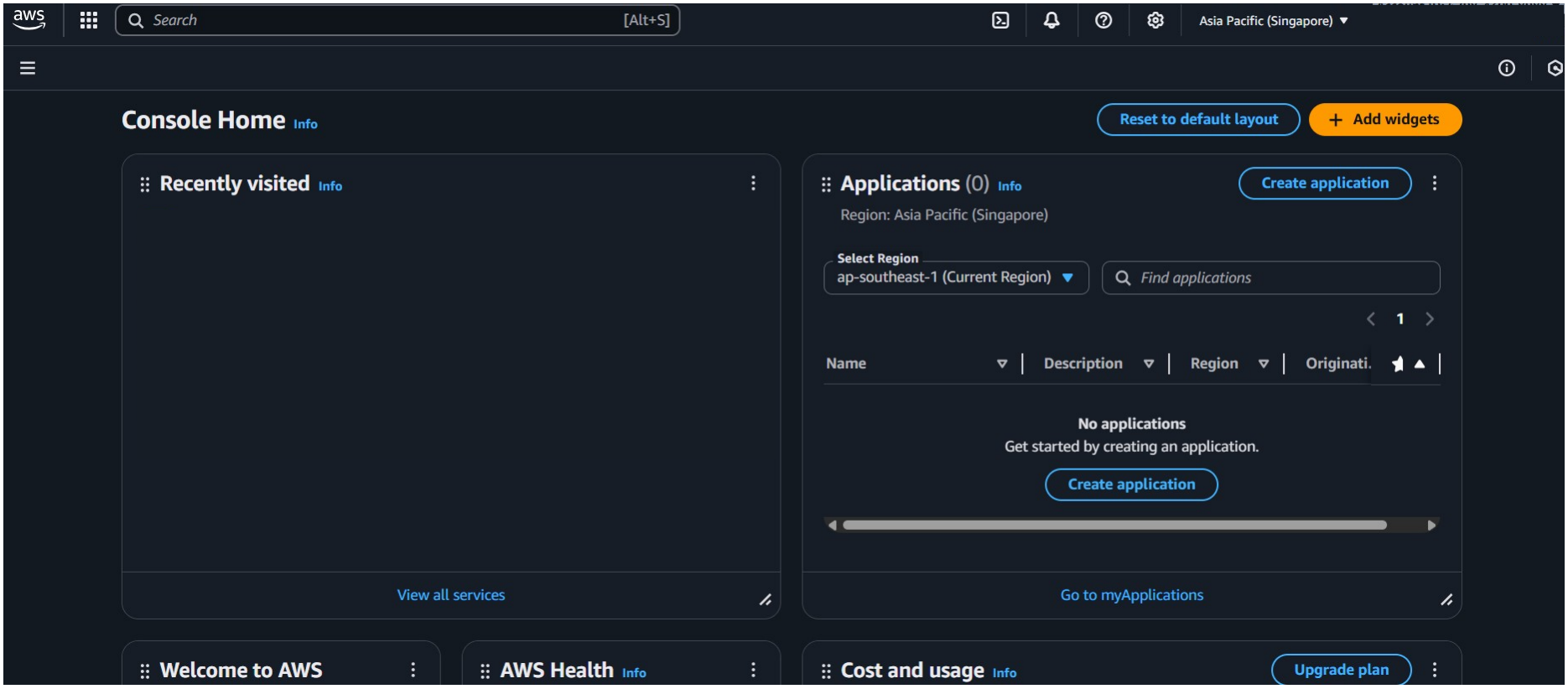
Amazon Platform Configuration

AWS Console

Sign in to the AWS Console



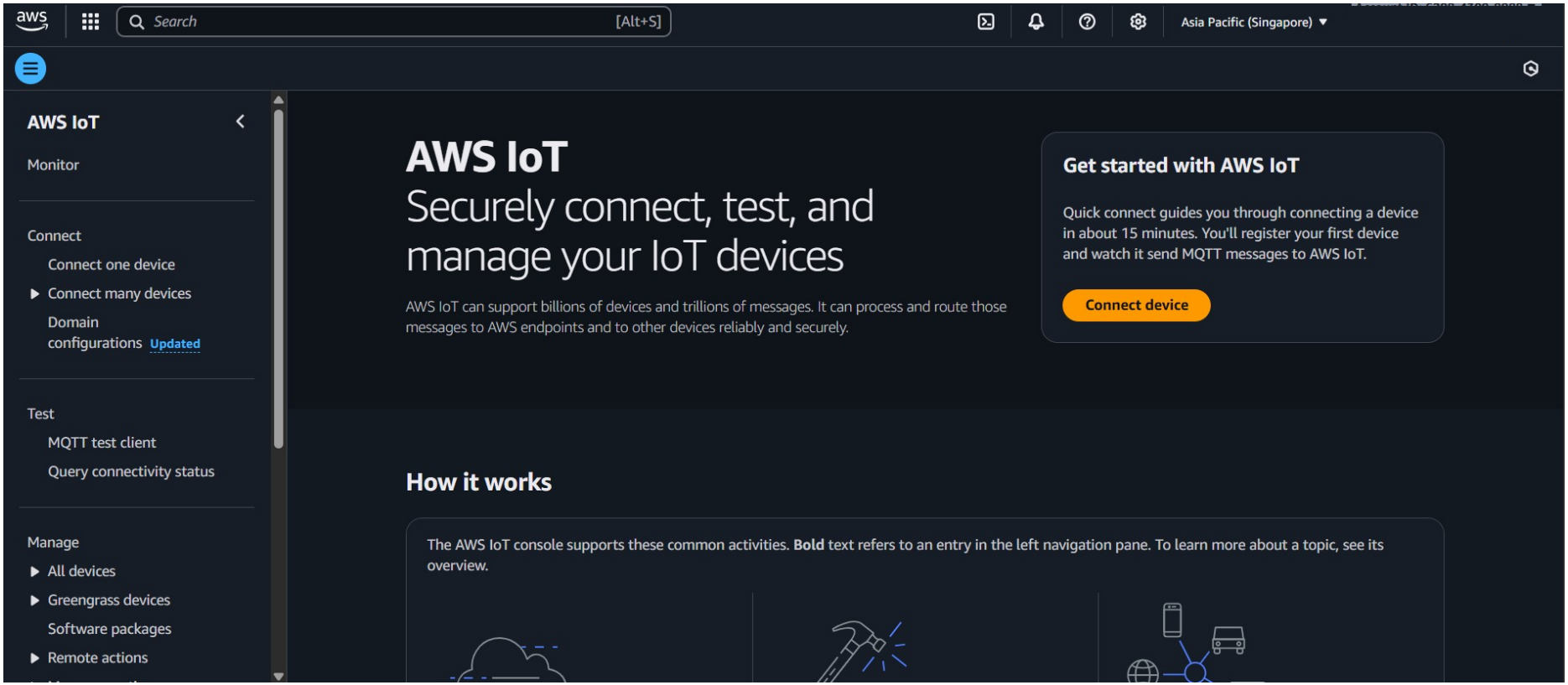
AWS Console Home



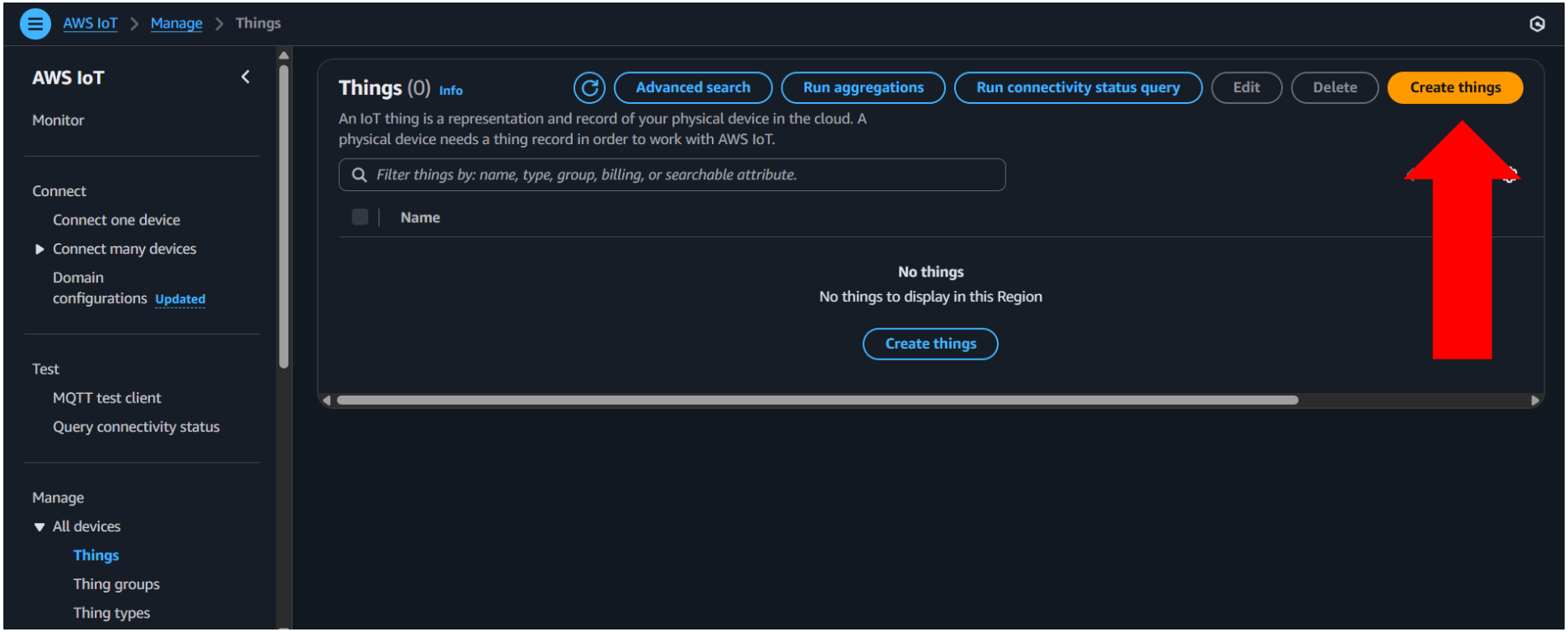
Search services at the *Find Services* search bar

IoT Core

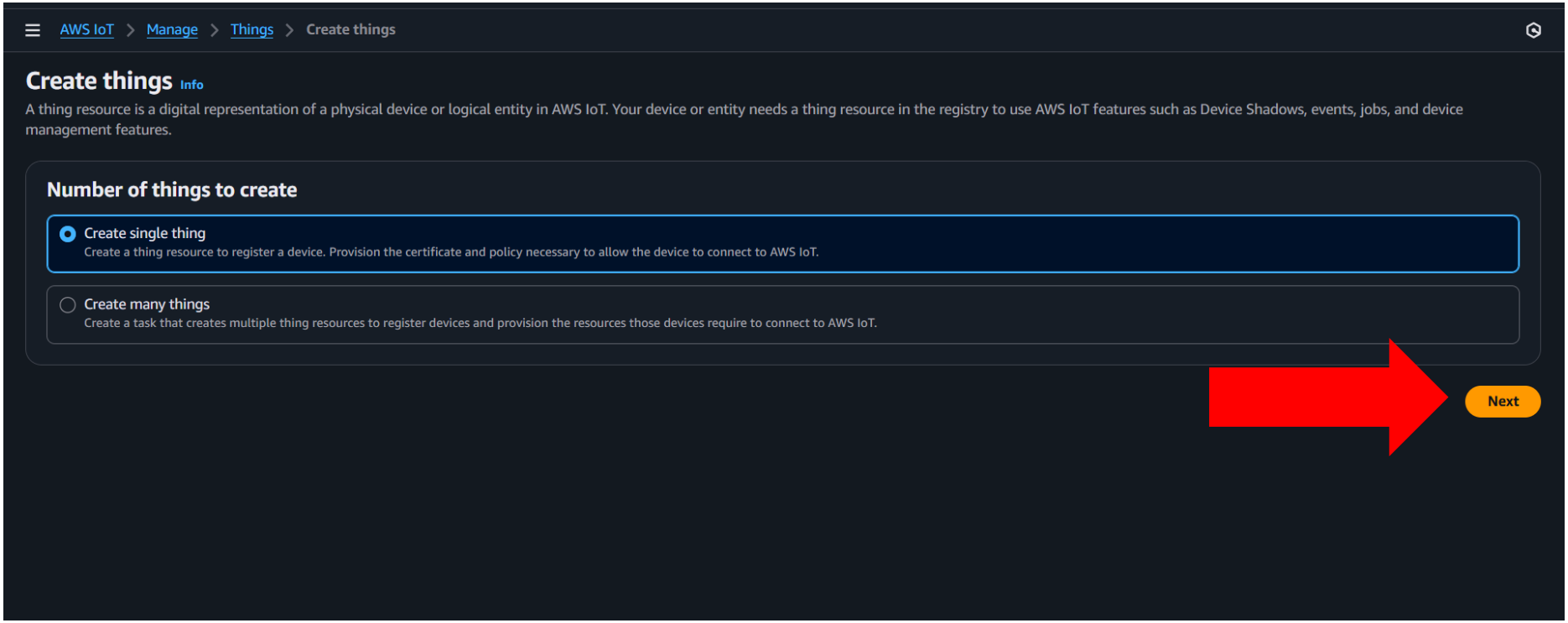
IoT Core Home Page



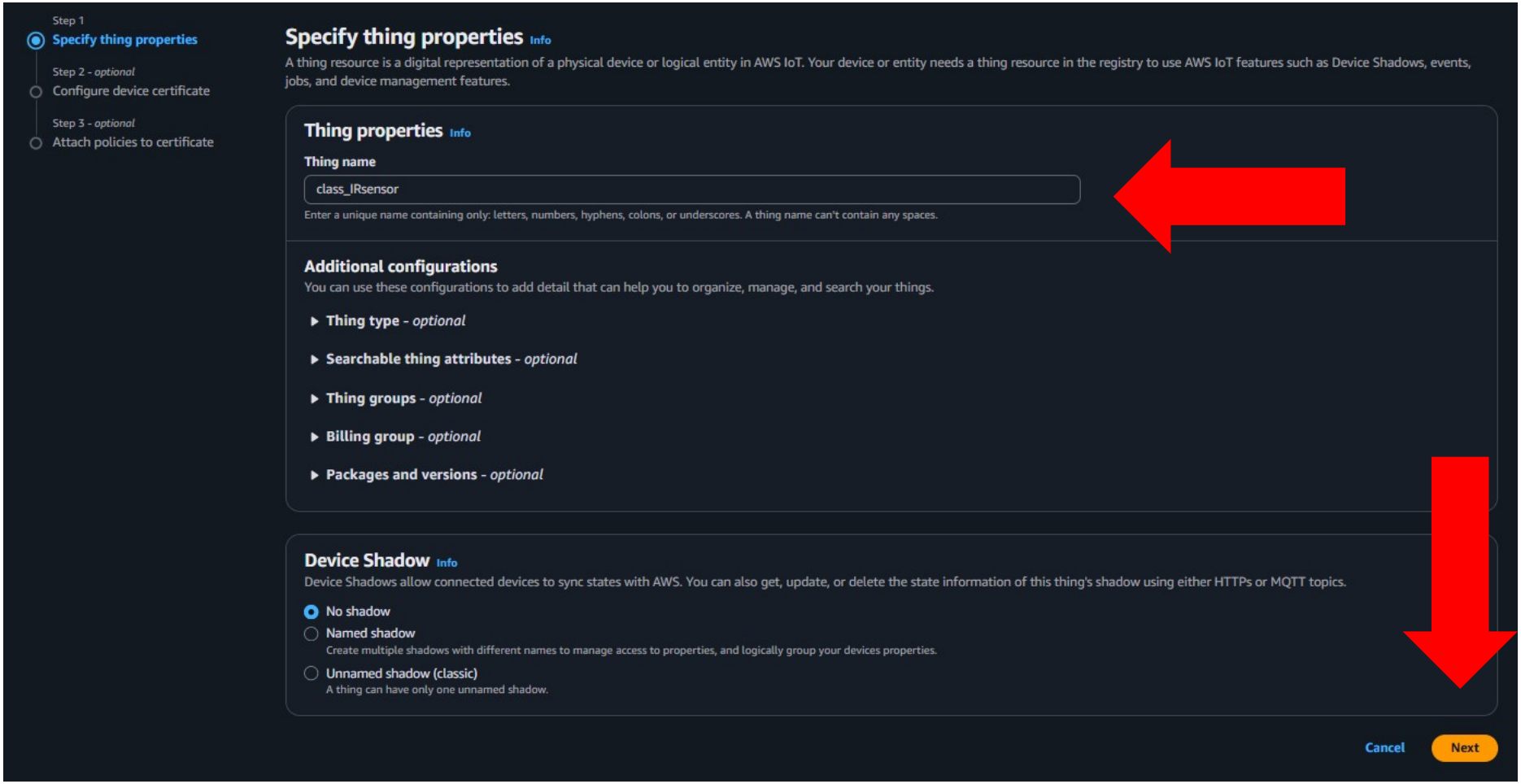
Navigate to *Things* in *All devices* under *Manage* and Create things



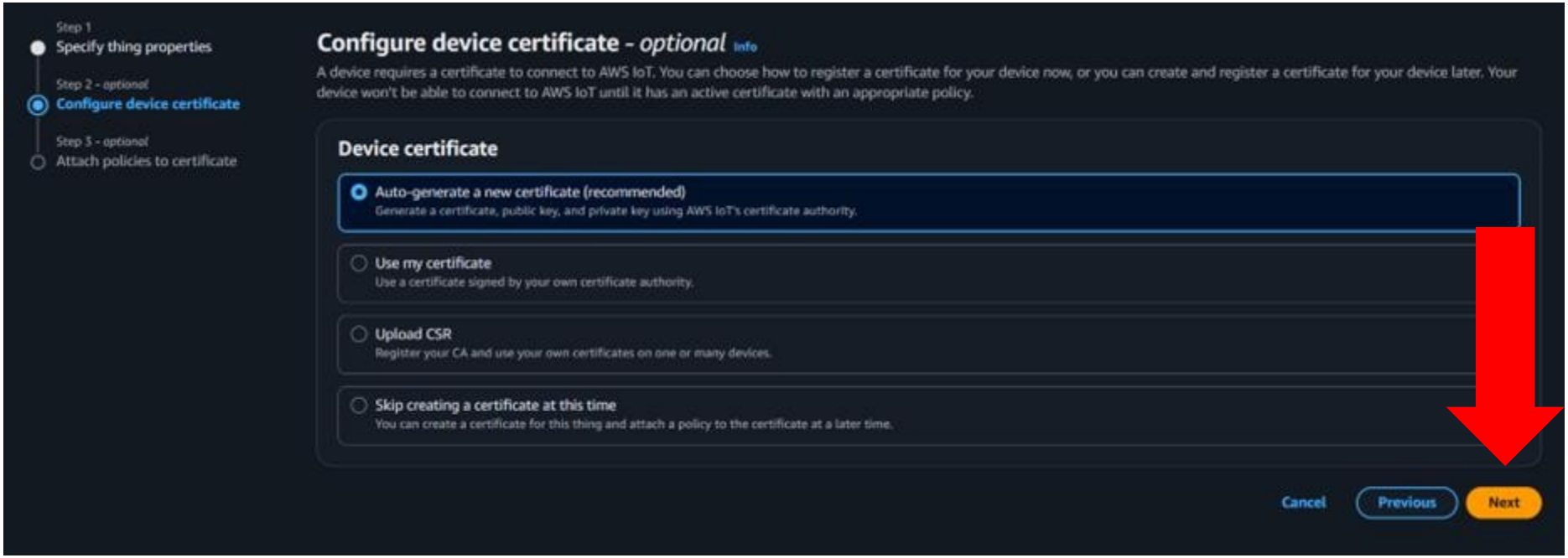
Choose the number of items to create, and click Next



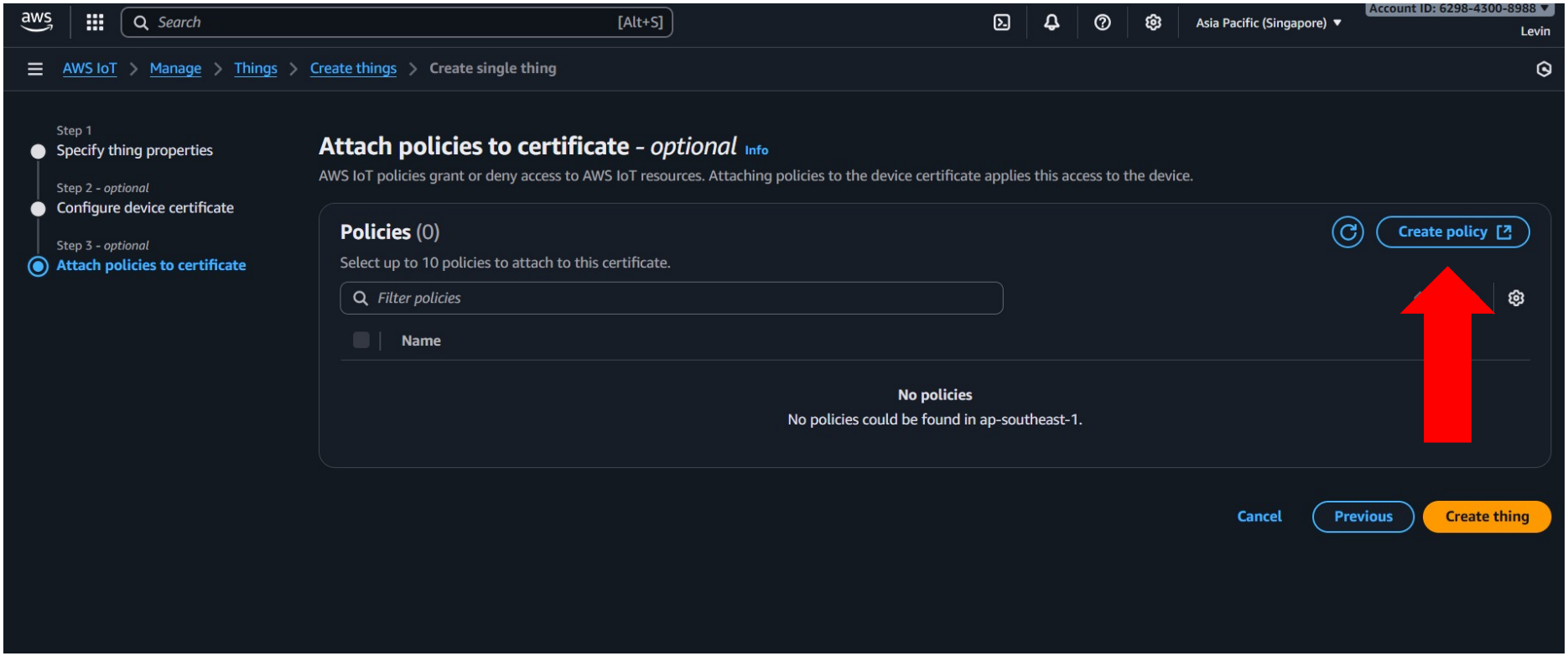
Input thing’s name and click Next



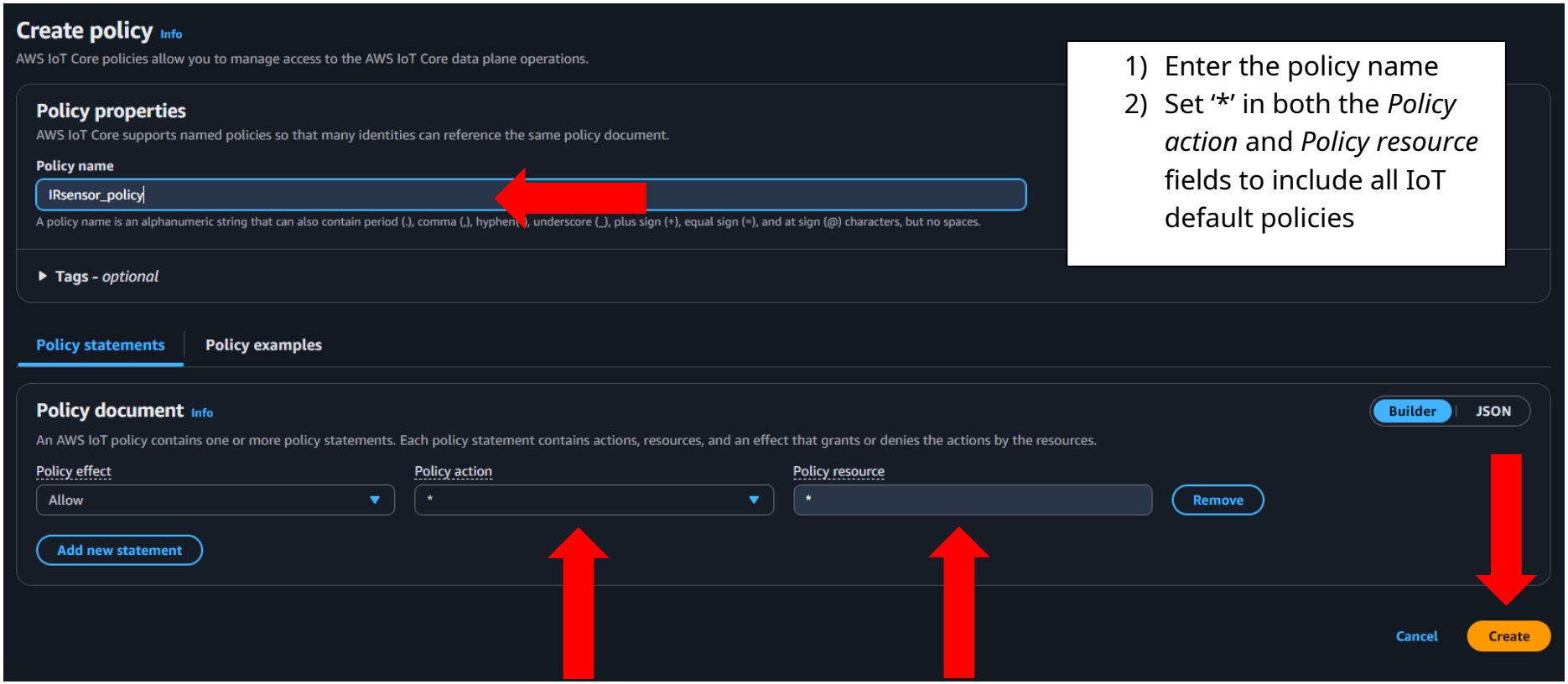
Click Next to continue



Create policy

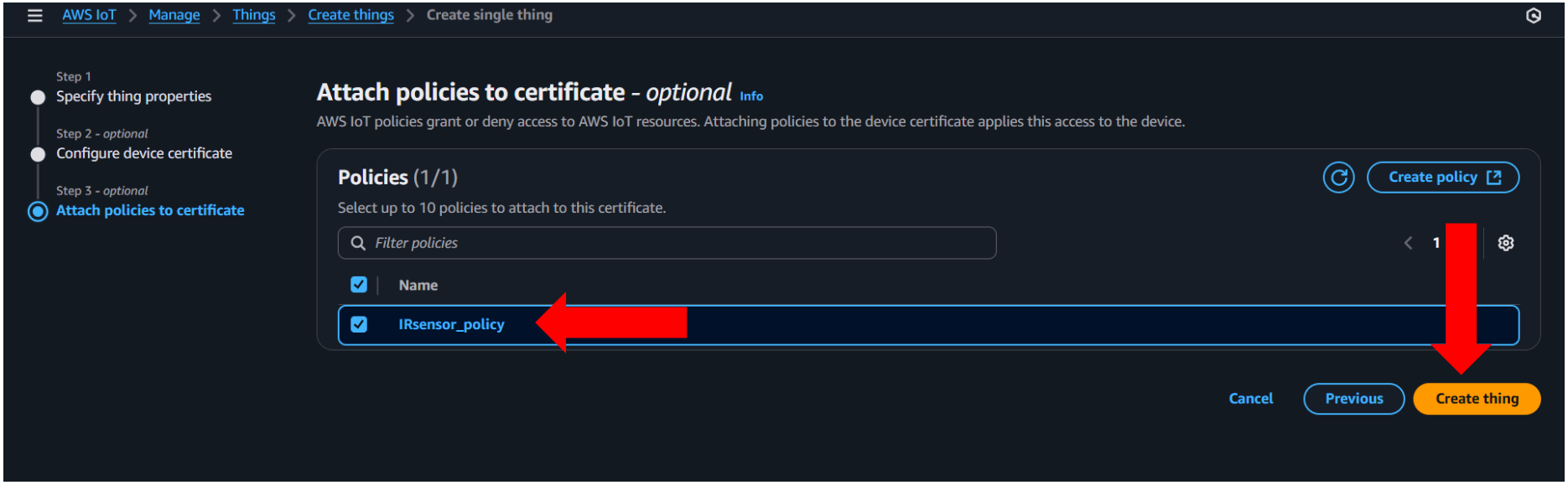


Input policy’s details

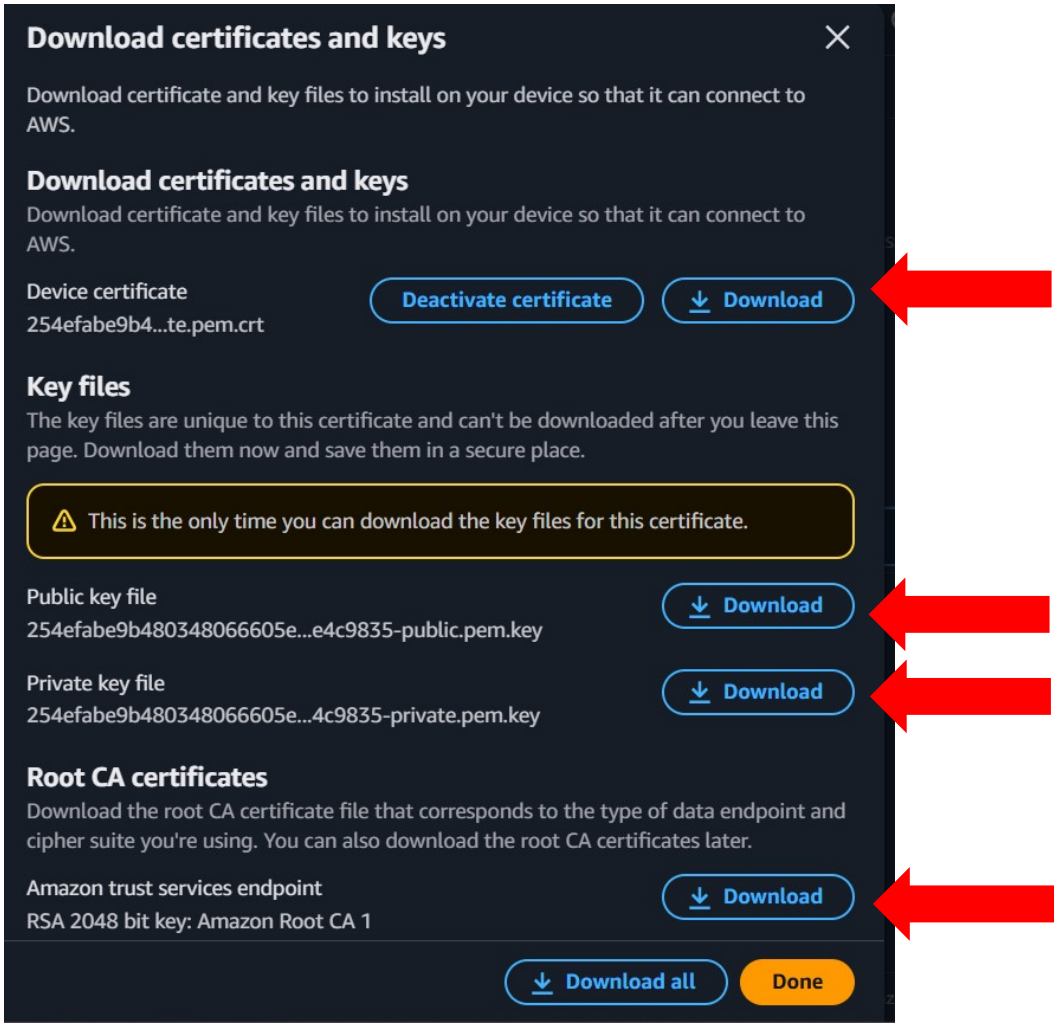


1) Enter the policy name
2) Set ‘*’ in both the *Policy action* and *Policy resource* fields to include all IoT default policies

Go back to the Create thing tab and attach the policy



Download 4 certificates

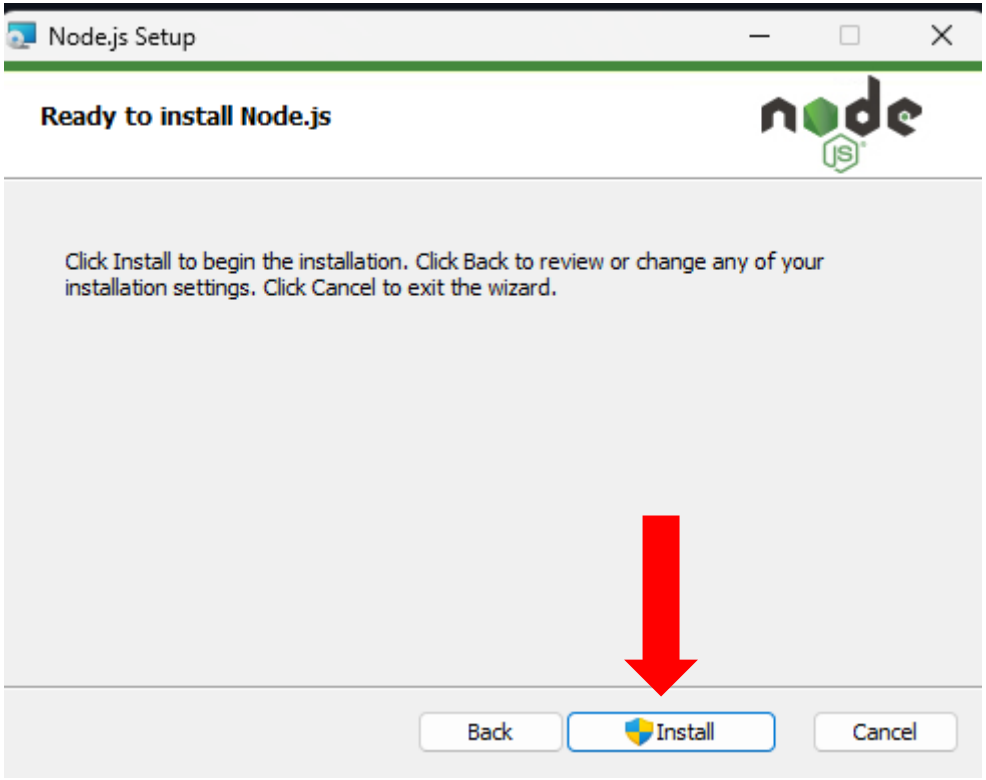


Simulation Installation

Node.js Installation

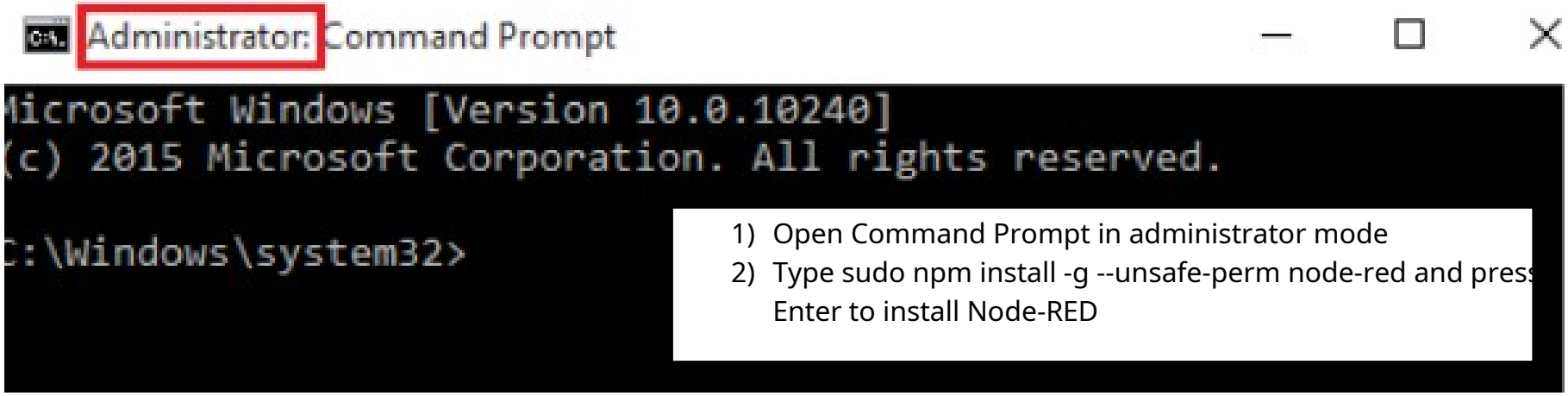
Install Node.js

Execute Node.js Setup

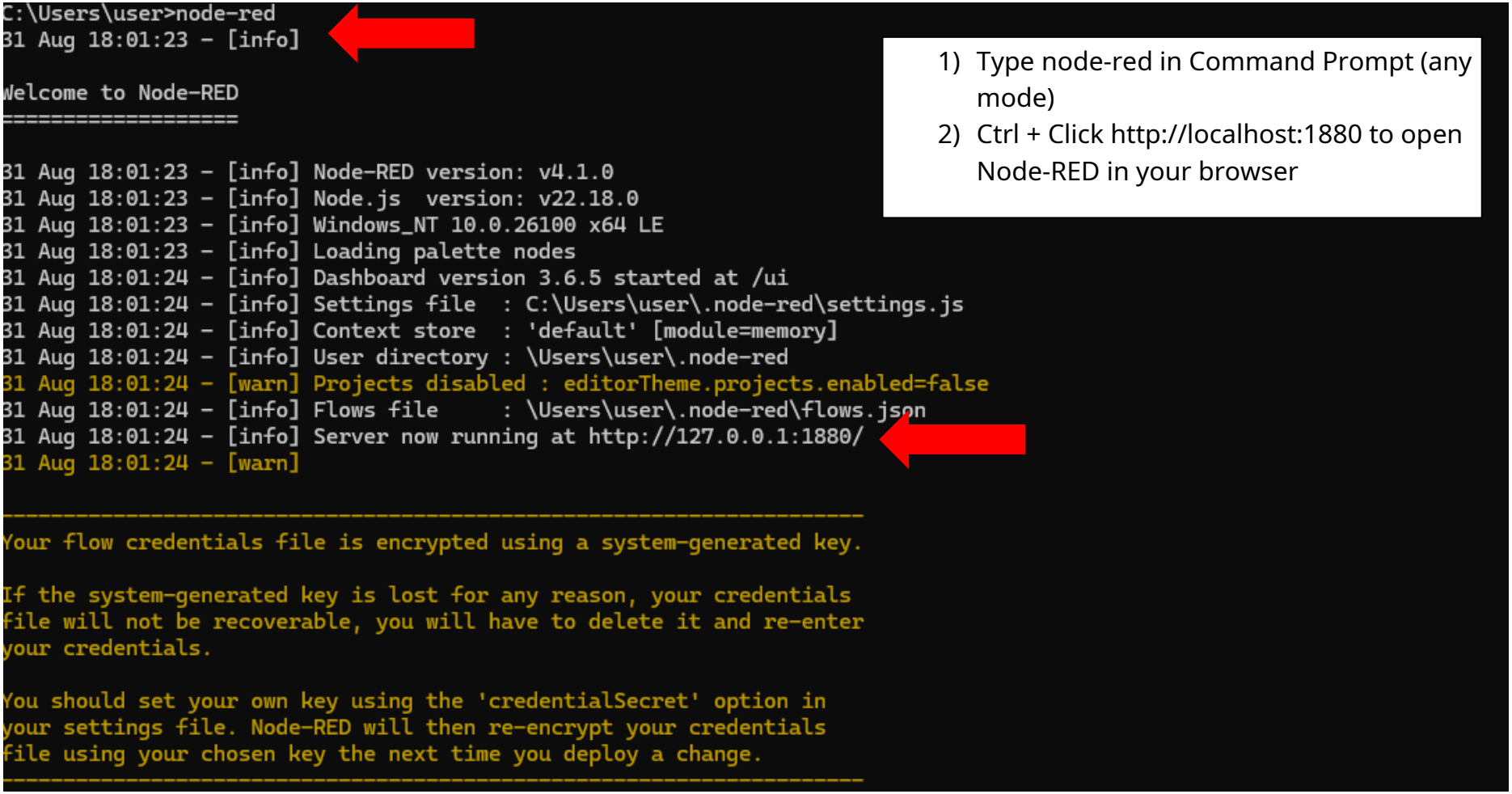


Node-RED Installation

Run command prompt

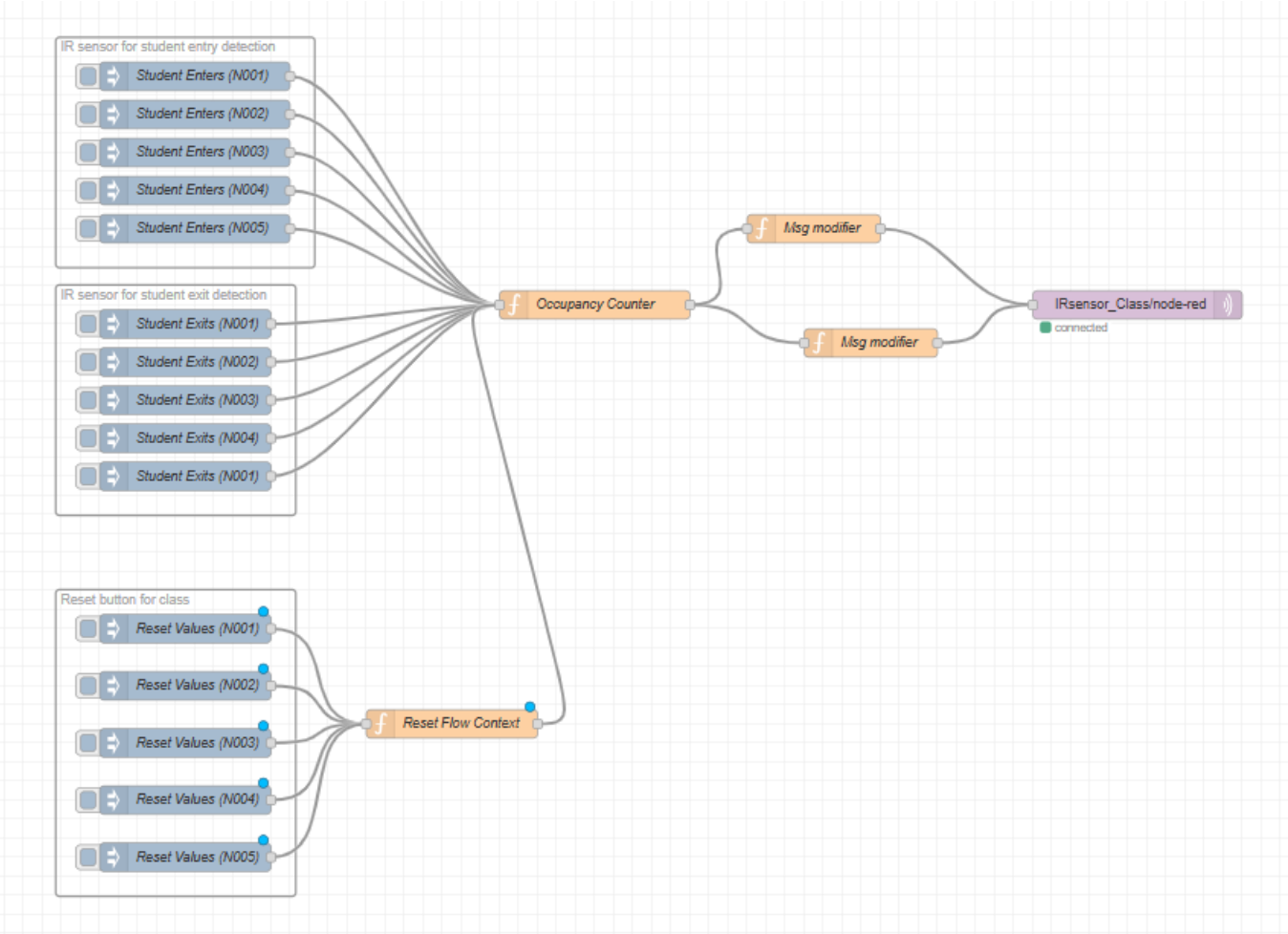


Run the Node-RED

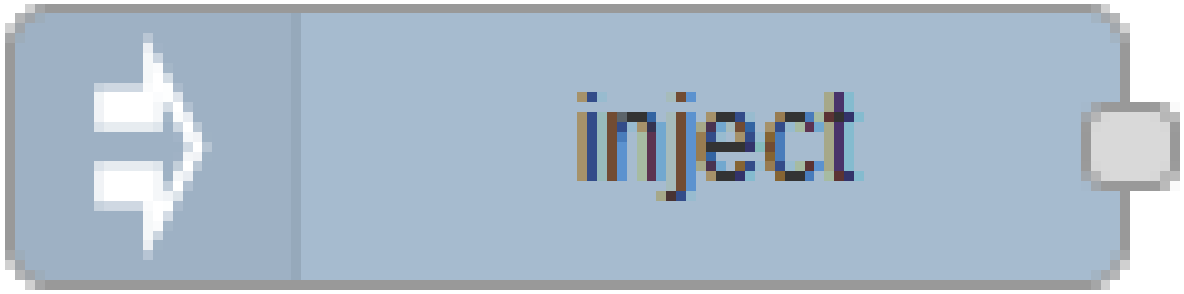


1) Type node-red in Command Prompt (any mode)
2) Ctrl + Click http://localhost:1880 to open Node-RED in your browser

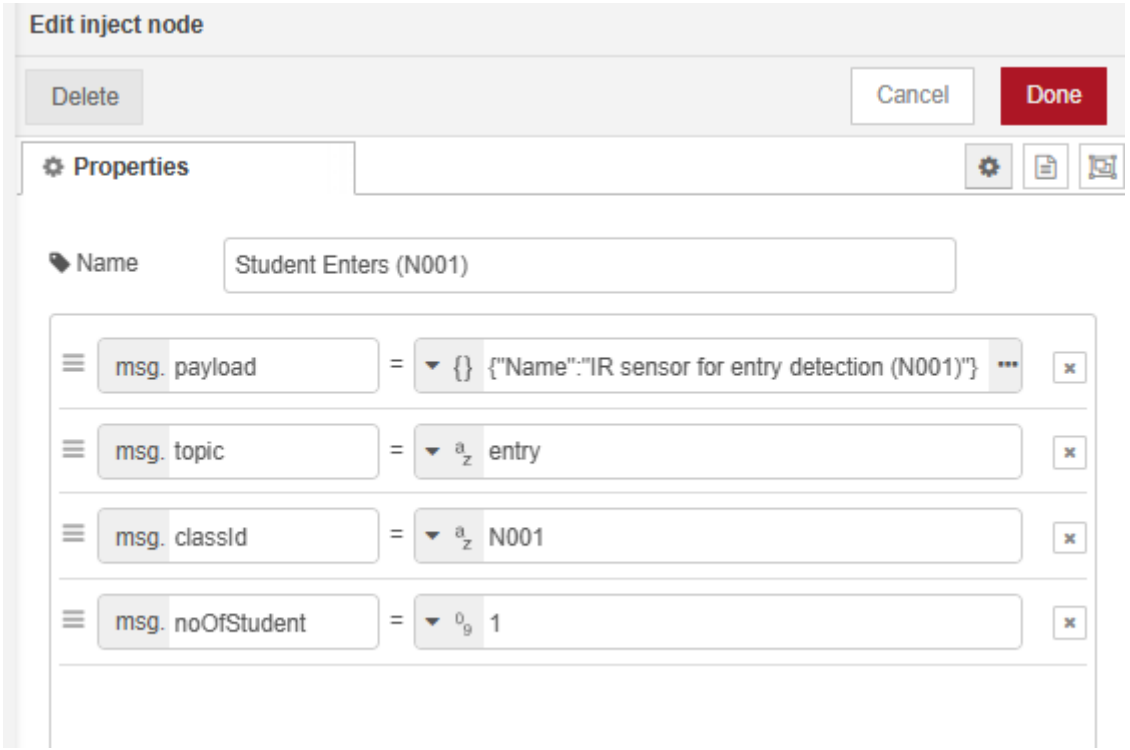
Create the diagram flow



Drag inject node from *common* to start a flow. Double-click to configure.

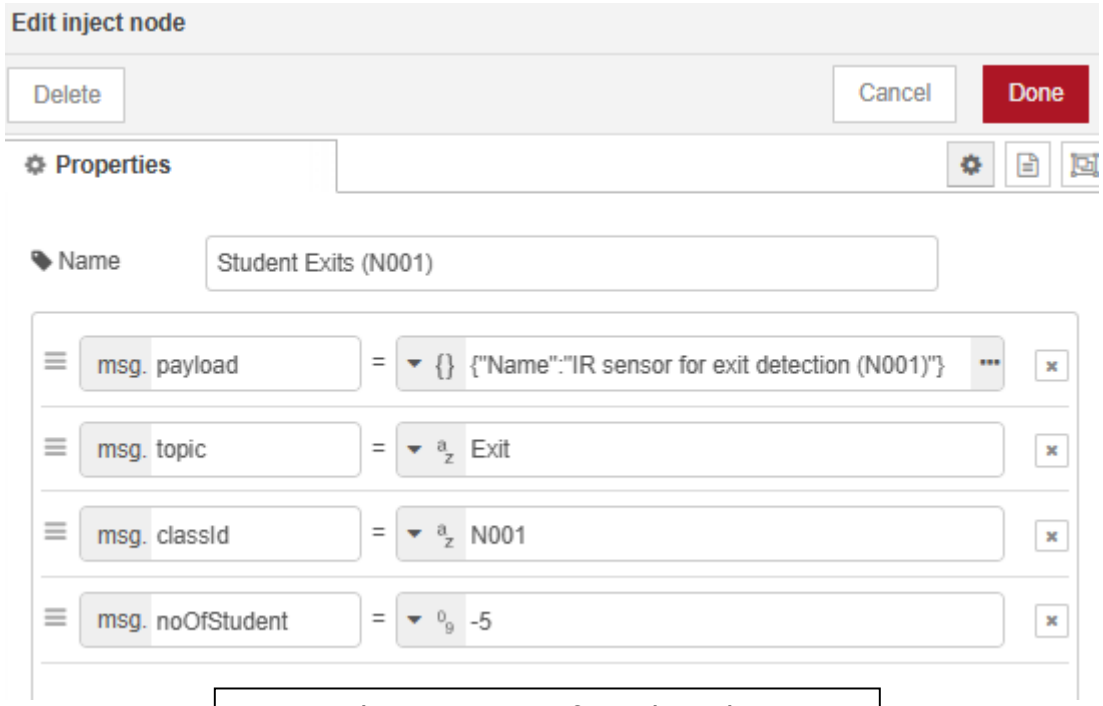


Configure the IR sensor for student entry detection in Class N001



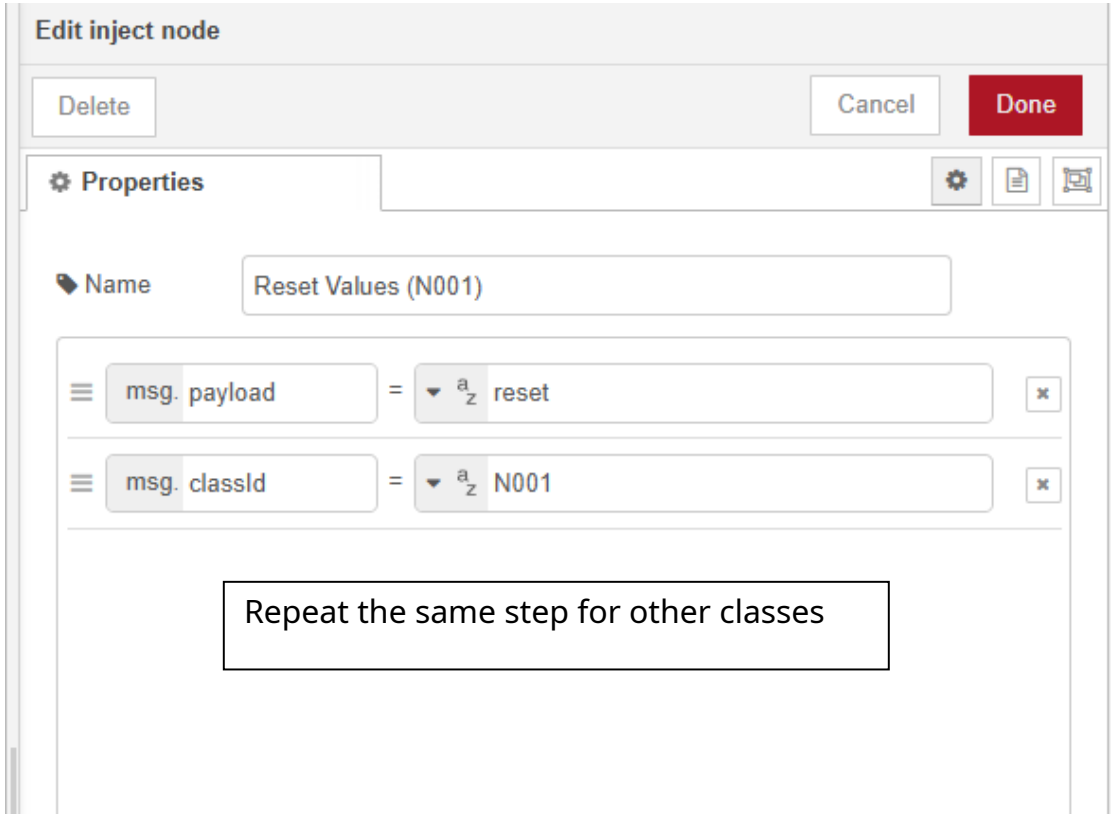
Repeat the same step for other classes

Configure the IR sensor for student exit detection in Class N001

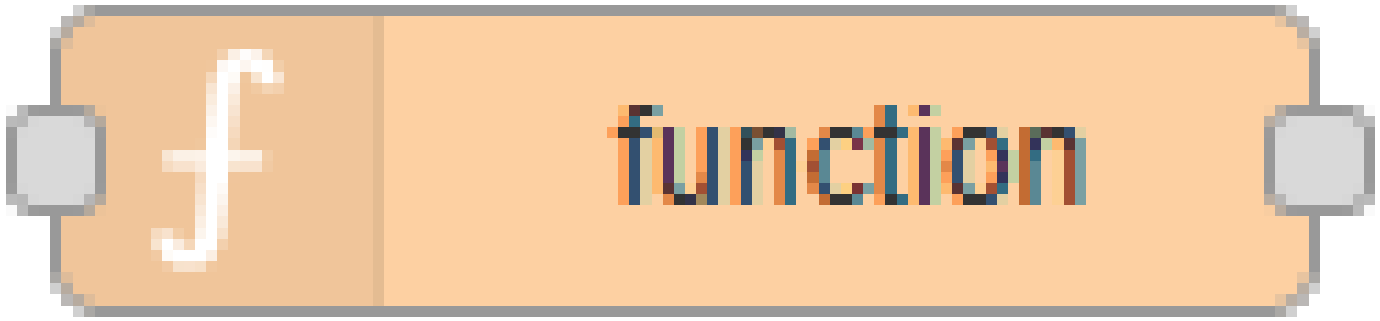


Repeat the same step for other classes

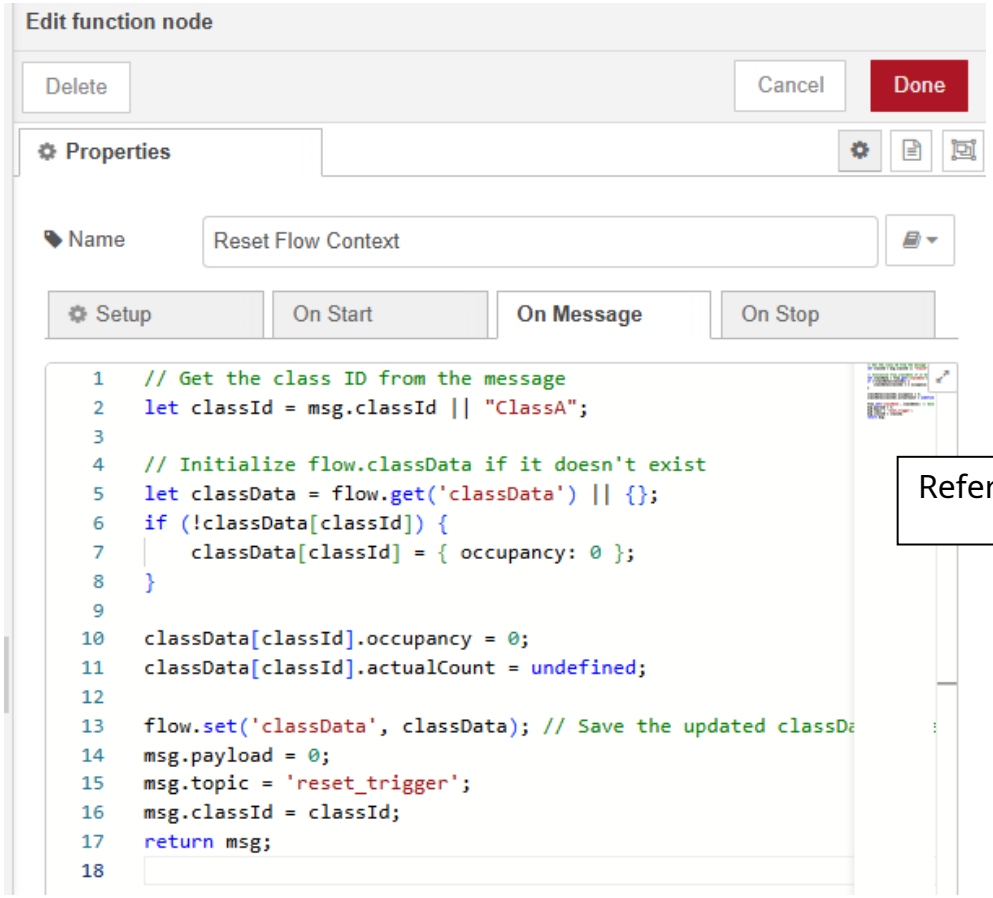
Configure the Reset button for Class N001



Drag function node from *function* to run custom JavaScript.
Double-click to configure.

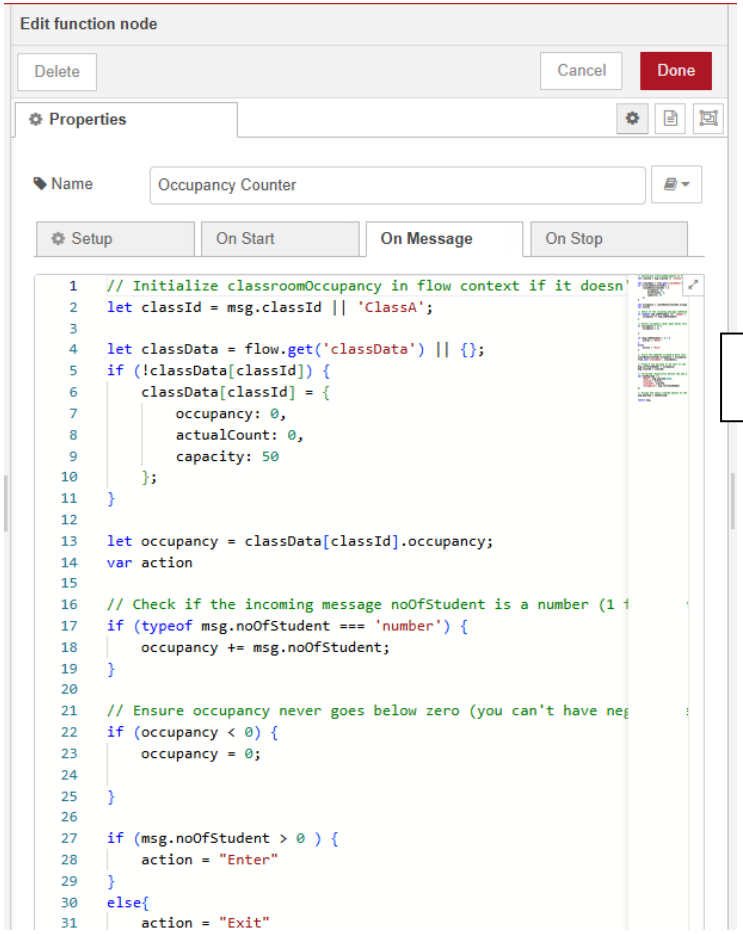


Configure the Reset Flow Context to reset the student count in the class



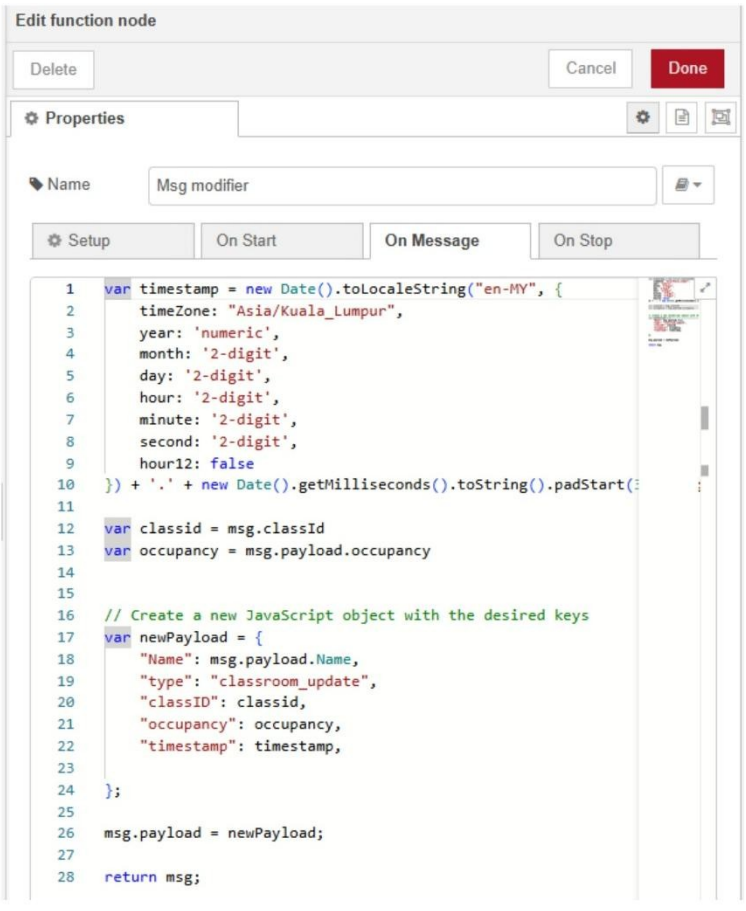
Refer to the Appendix for the code

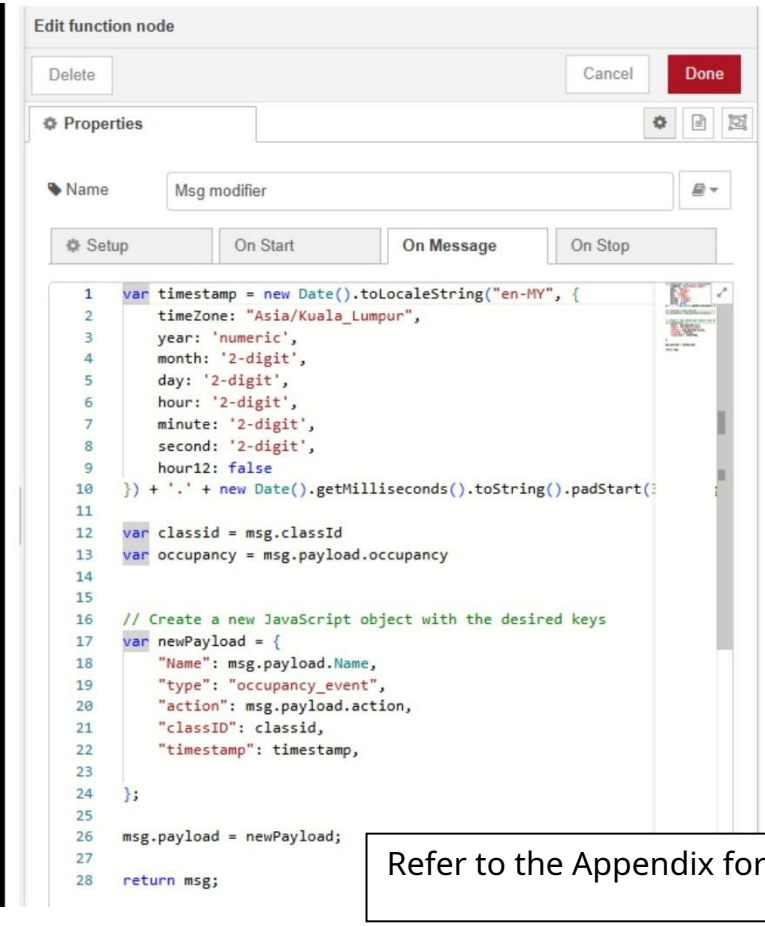
Configure the Occupancy Counter to track students entering and leaving



Refer to the Appendix for the code

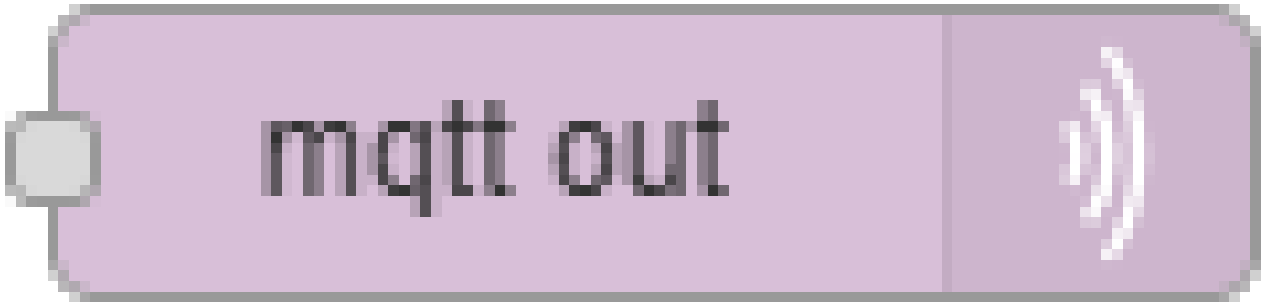
Configure both Msg Modifier nodes to customize messages



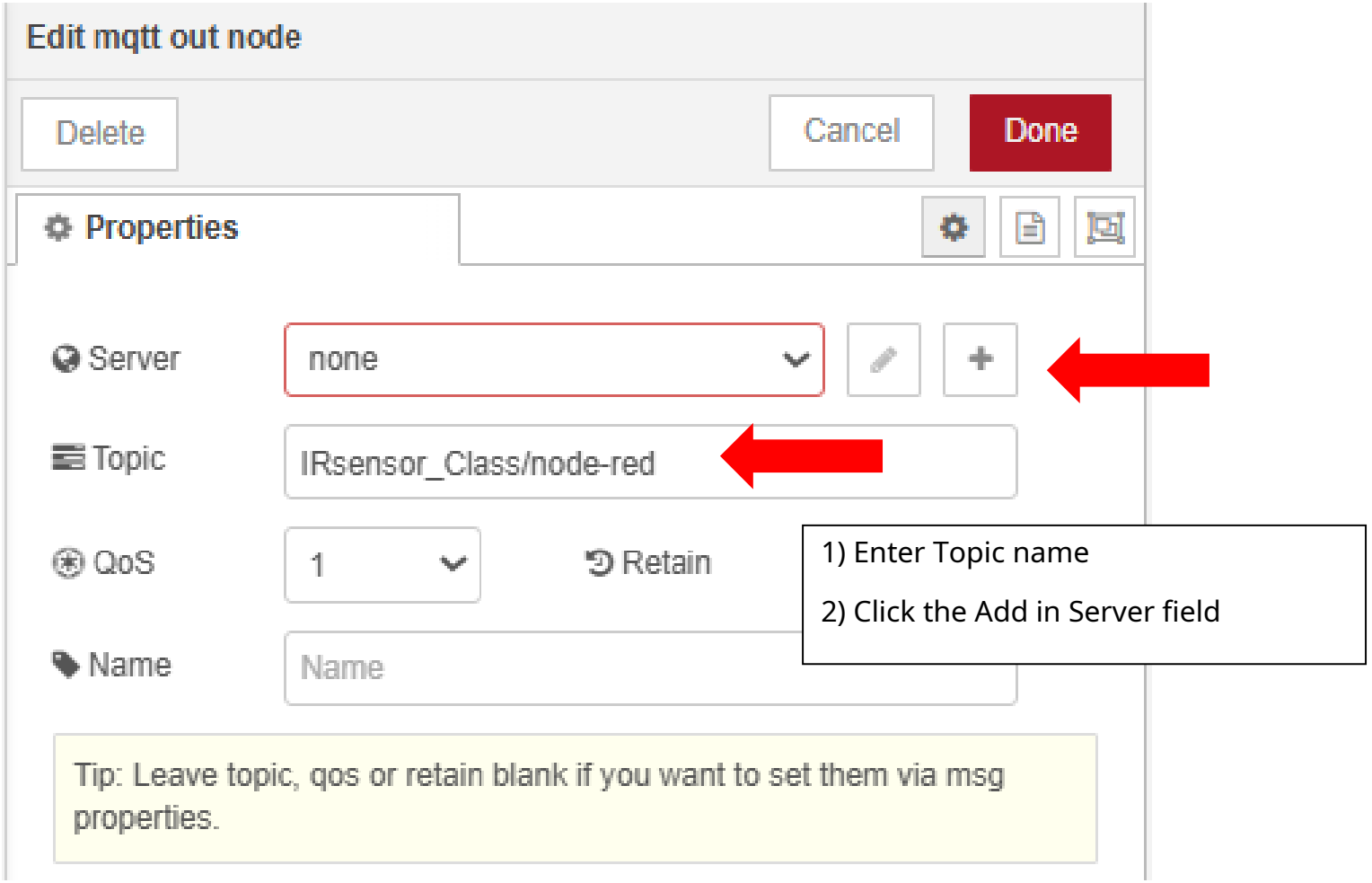


Refer to the Appendix for the code

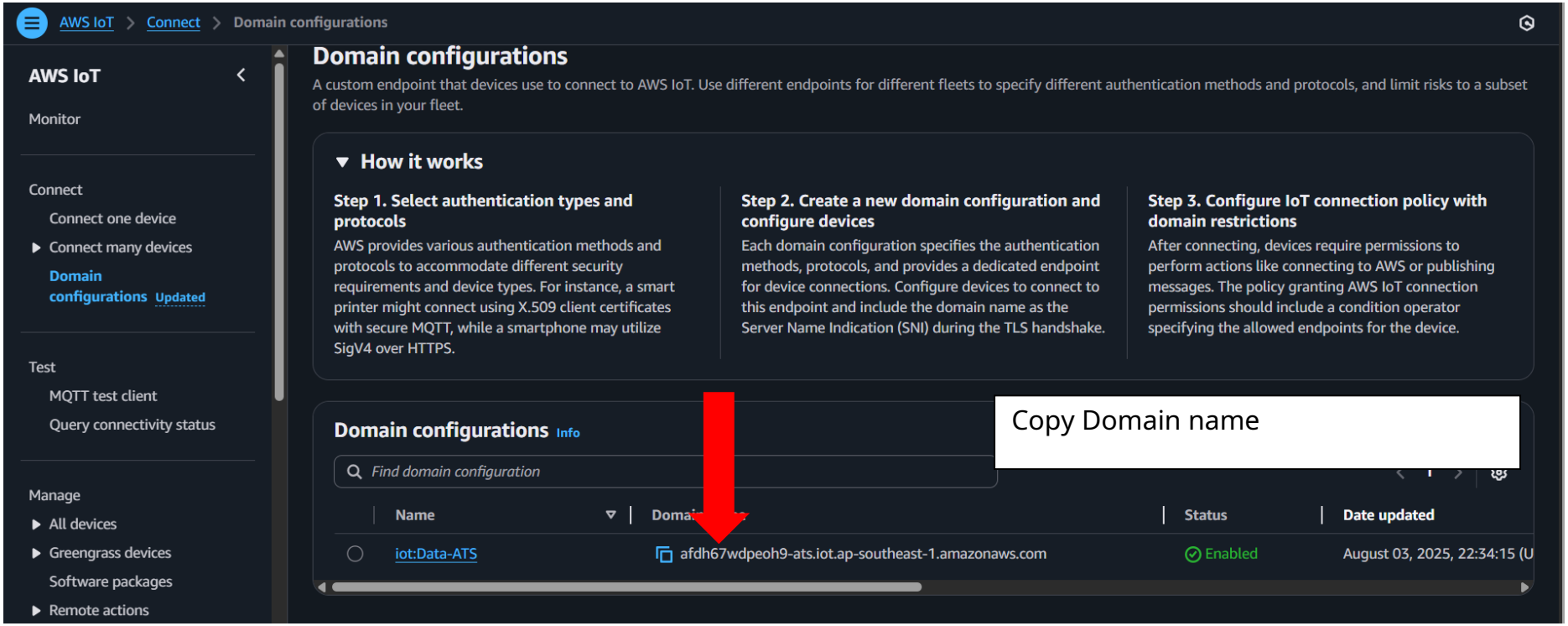
Drag mqtt out node to connect with AWS IoT Core. Double-click to configure



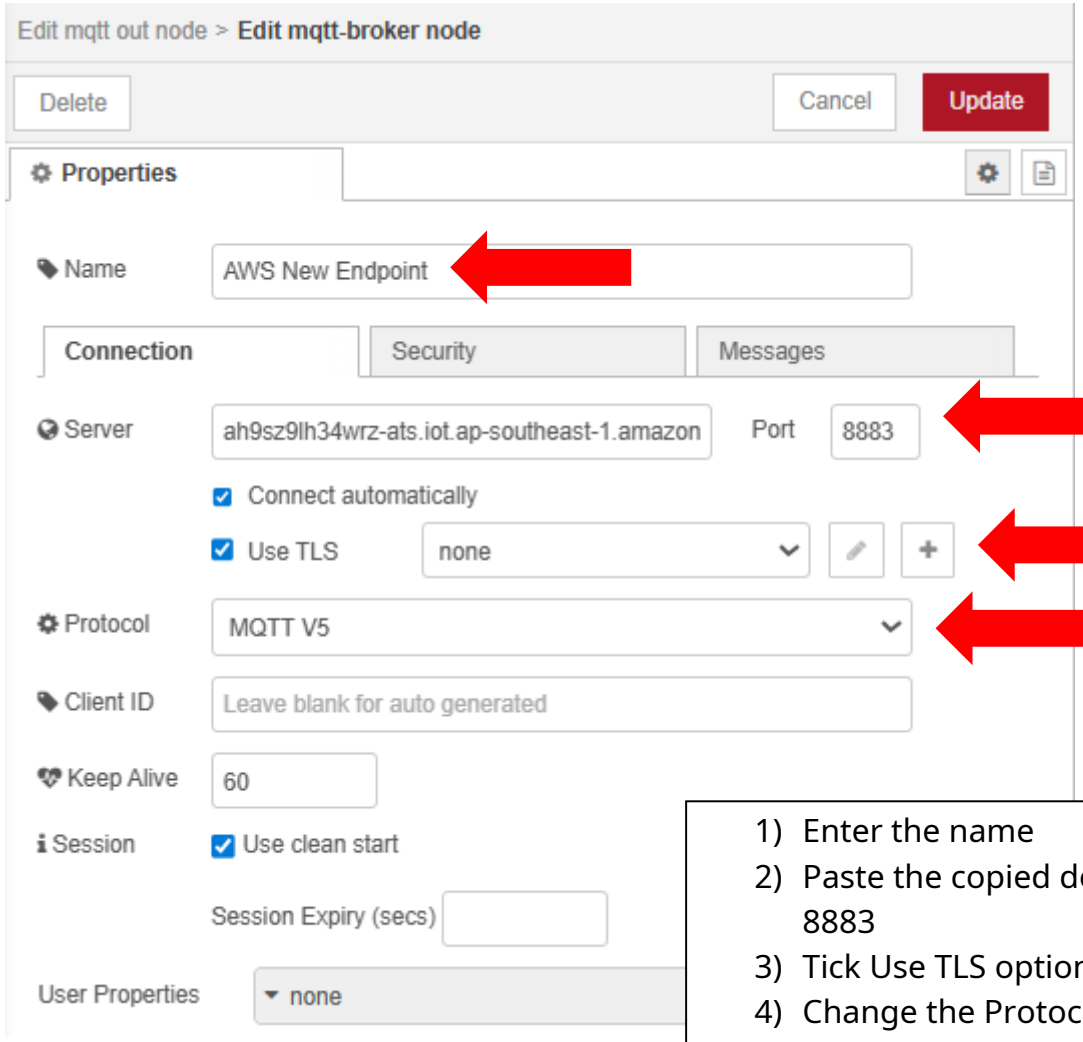
Input the Topic field



Navigate to Domain configuration under *Connect* in AWS IoT Core

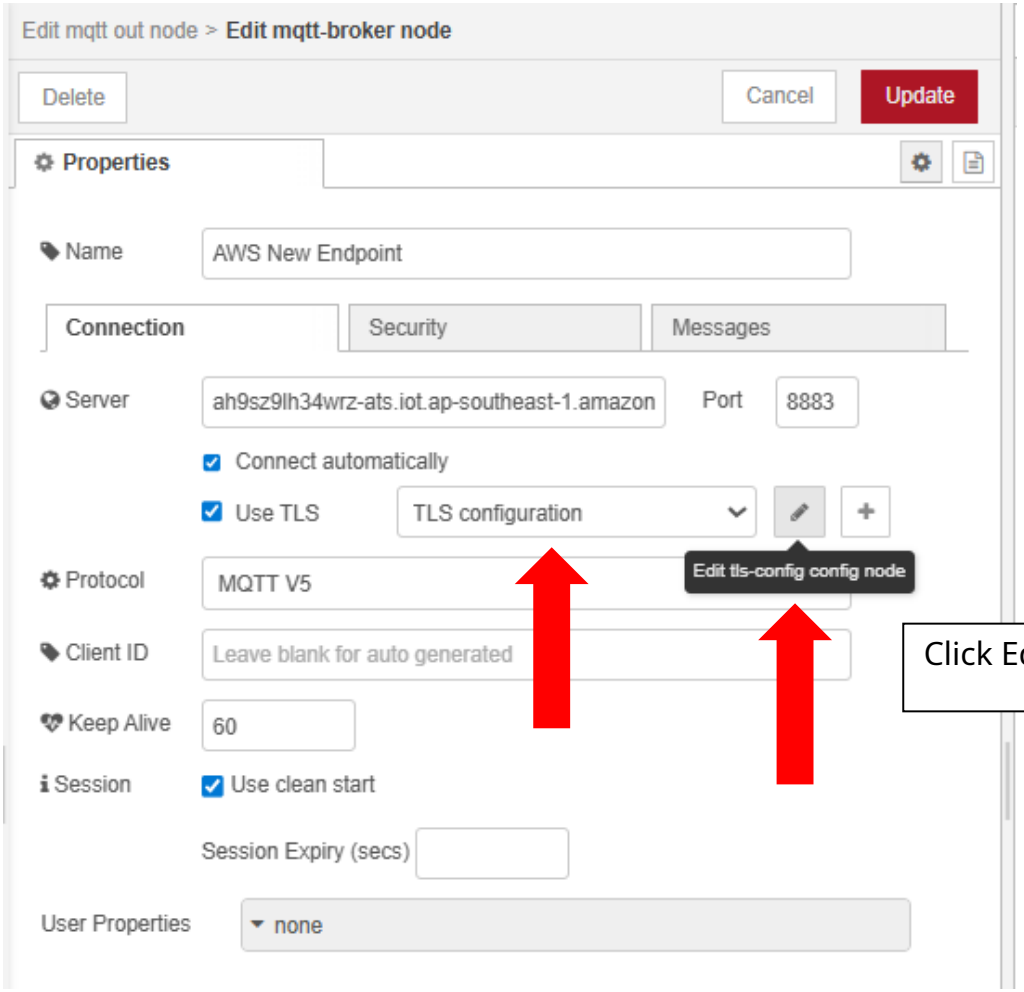


Configure the Server field

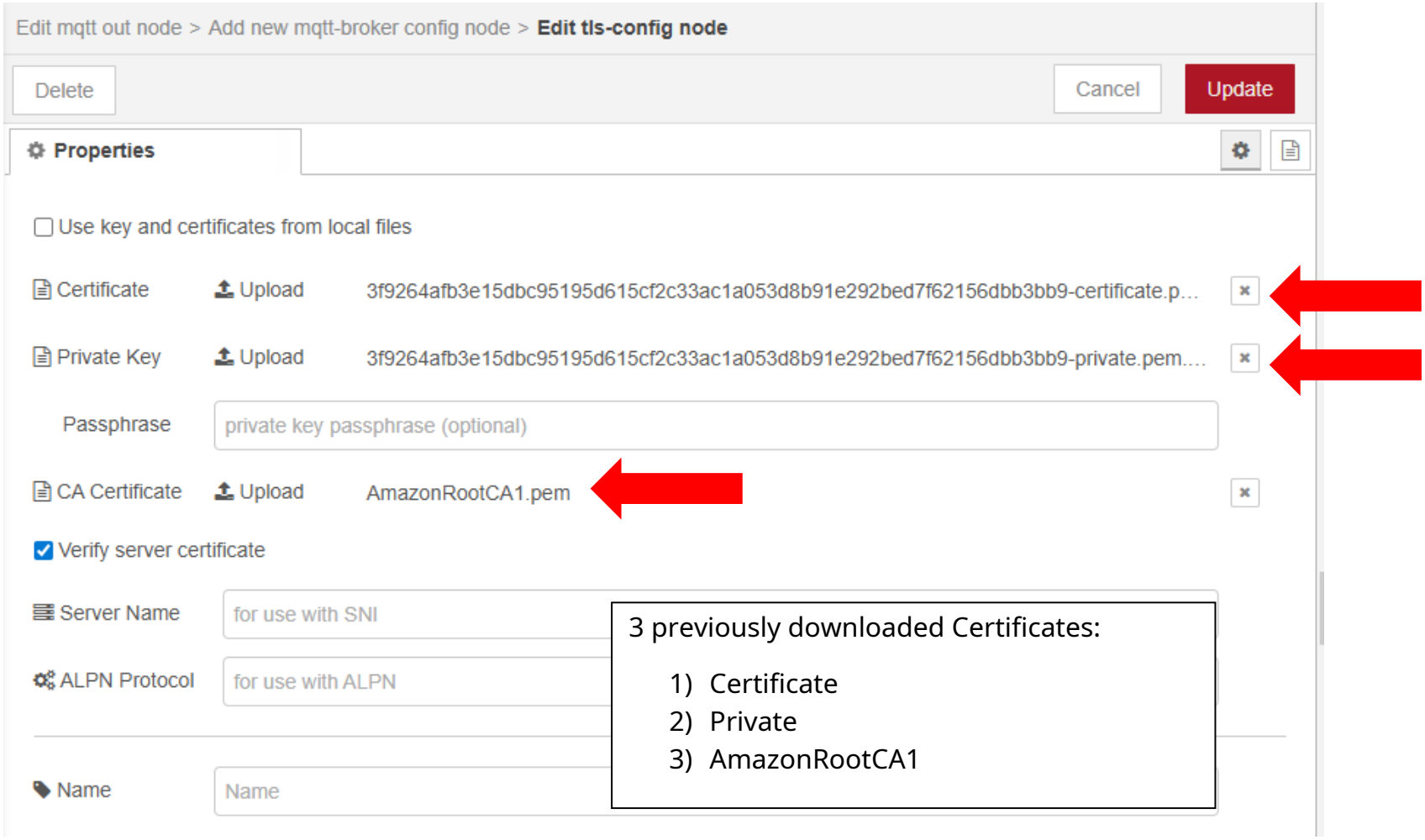


1) Enter the name
2) Paste the copied domain name and set the port to 8883
3) Tick Use TLS option
4) Change the Protocol to MQTT V5

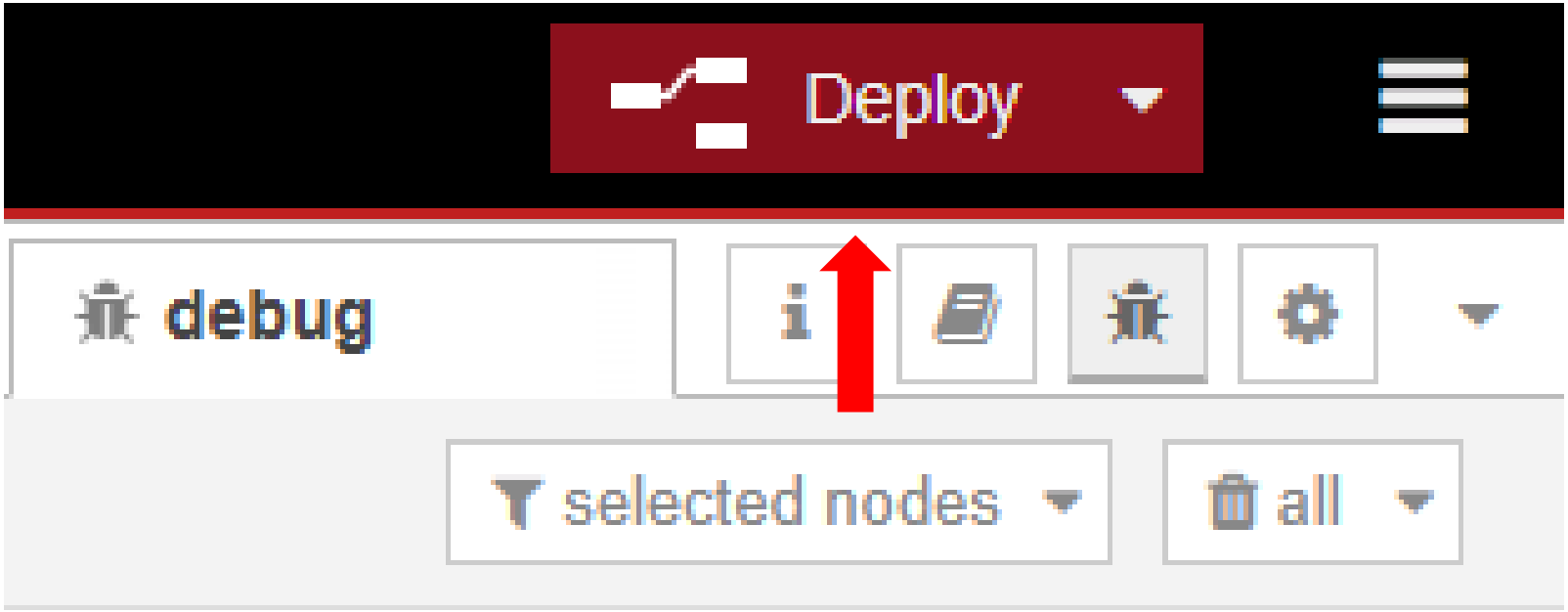
Change Use TLS option



Input the 3 certificates

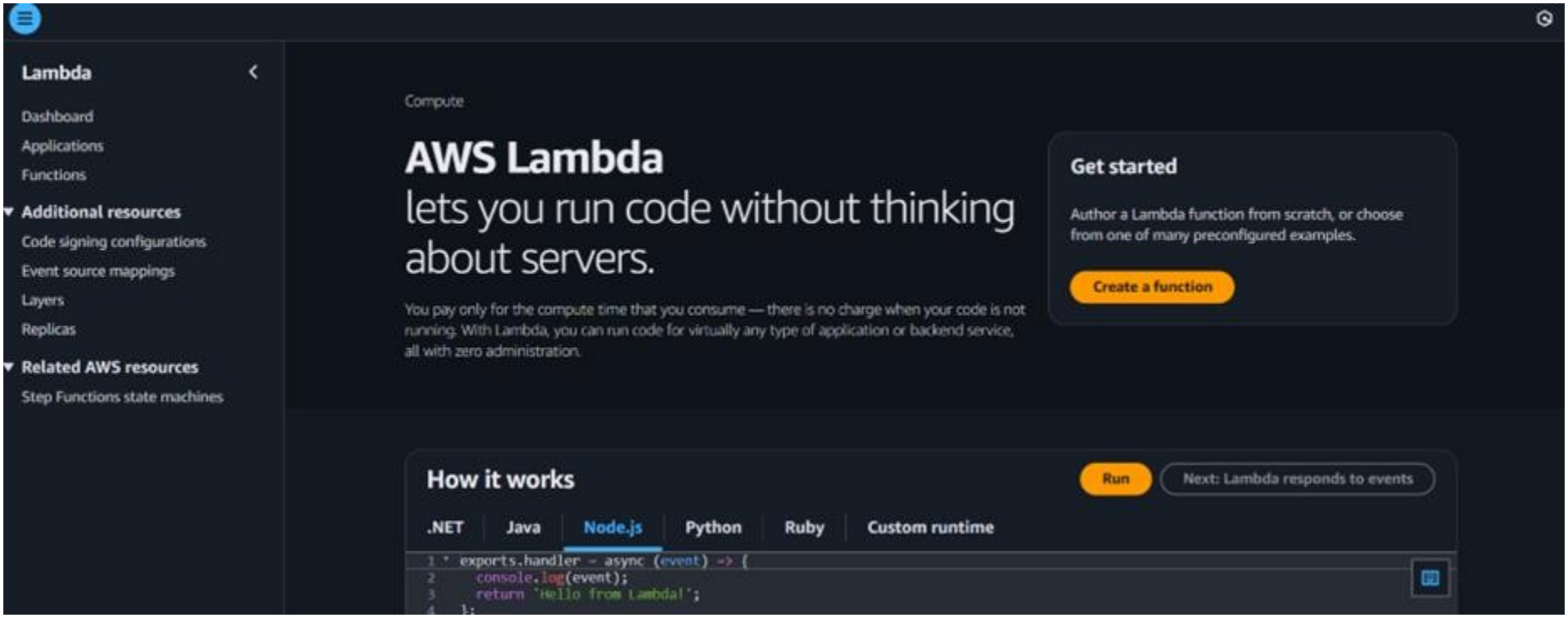


Click Deploy

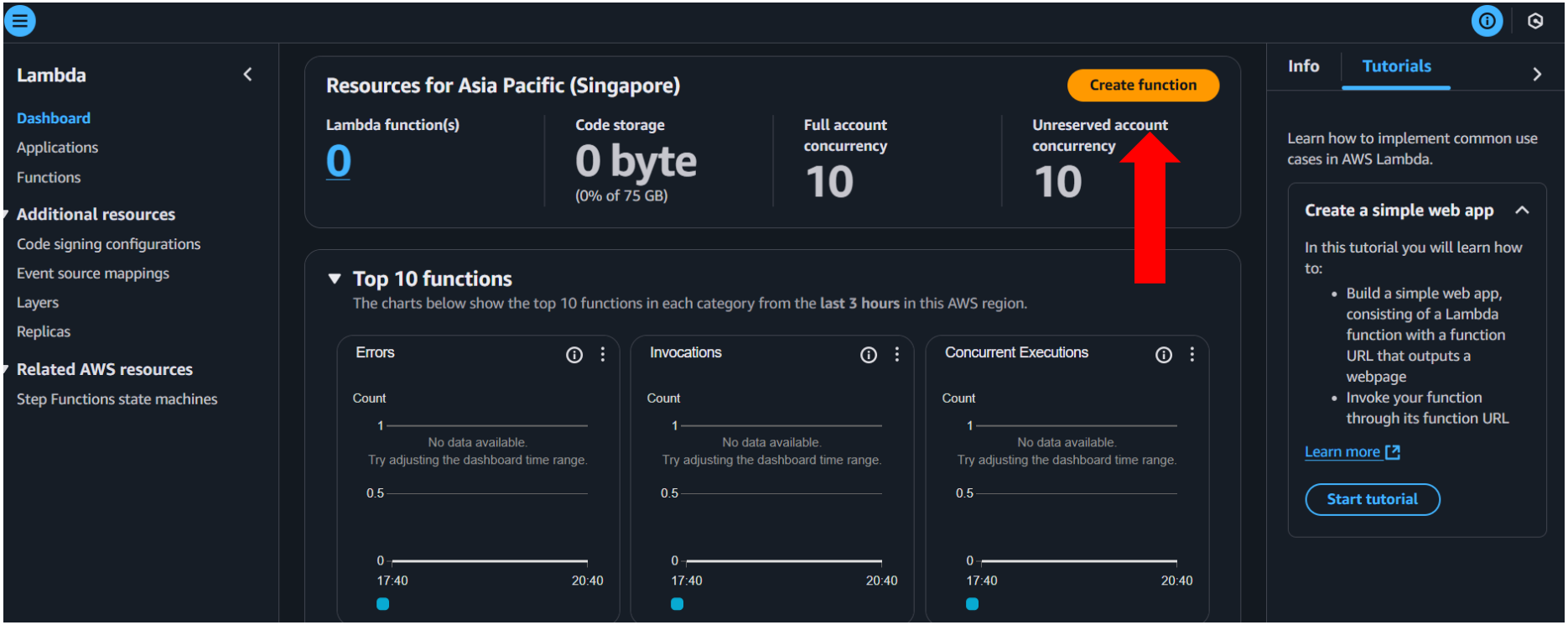


Lambda

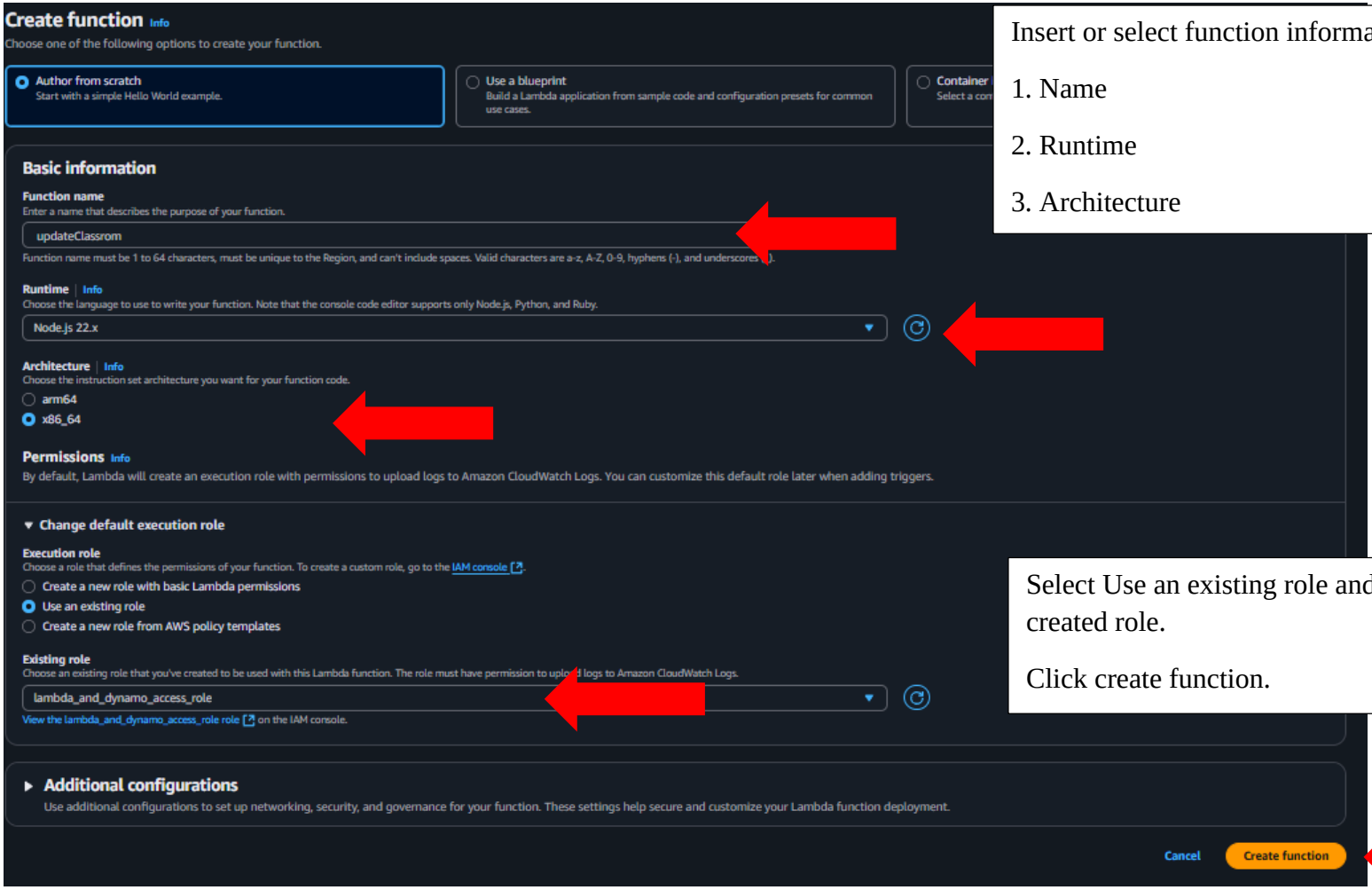
Lambda Home Page



Click Create function



Configure function



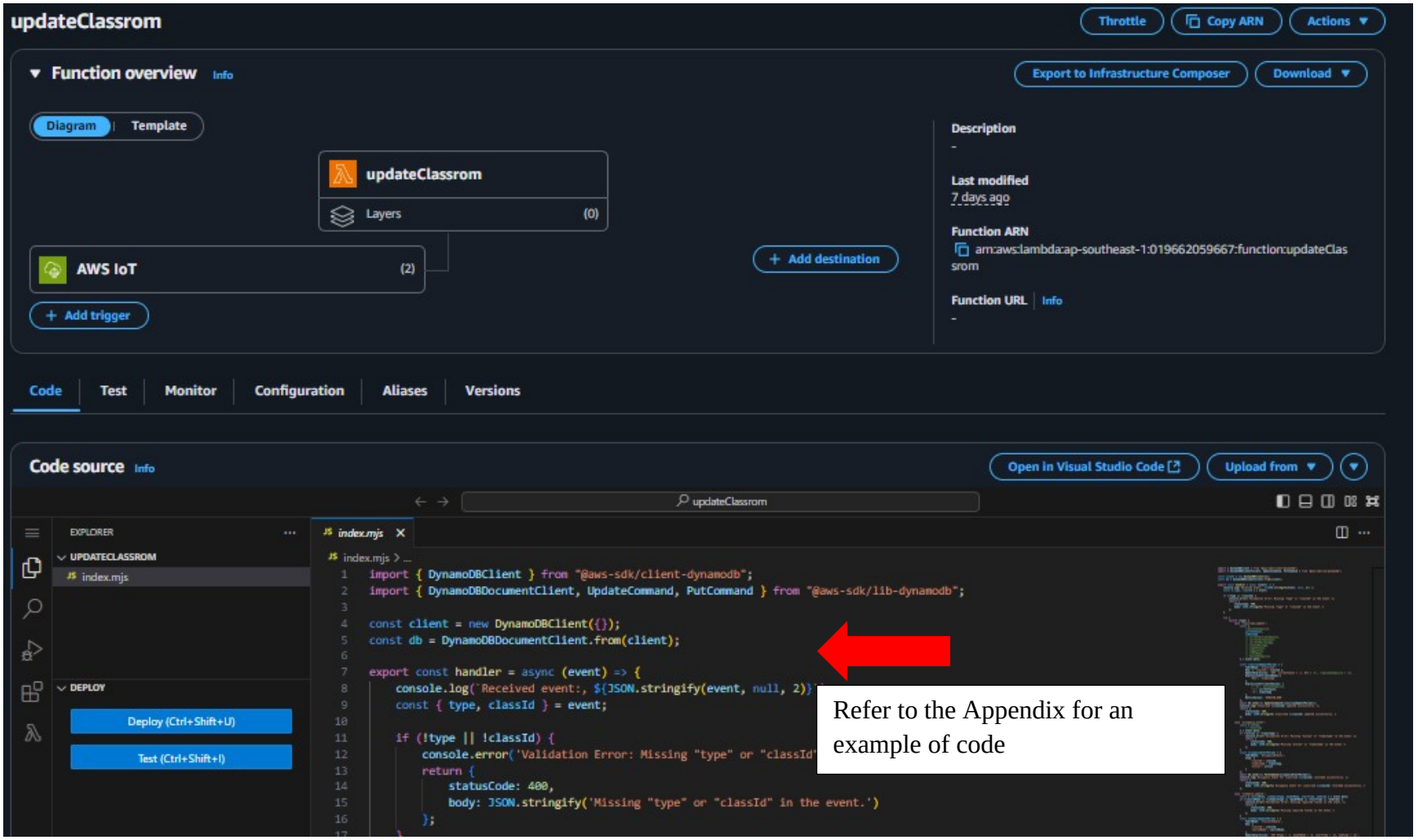
Insert or select function information:

1. Name
2. Runtime
3. Architecture

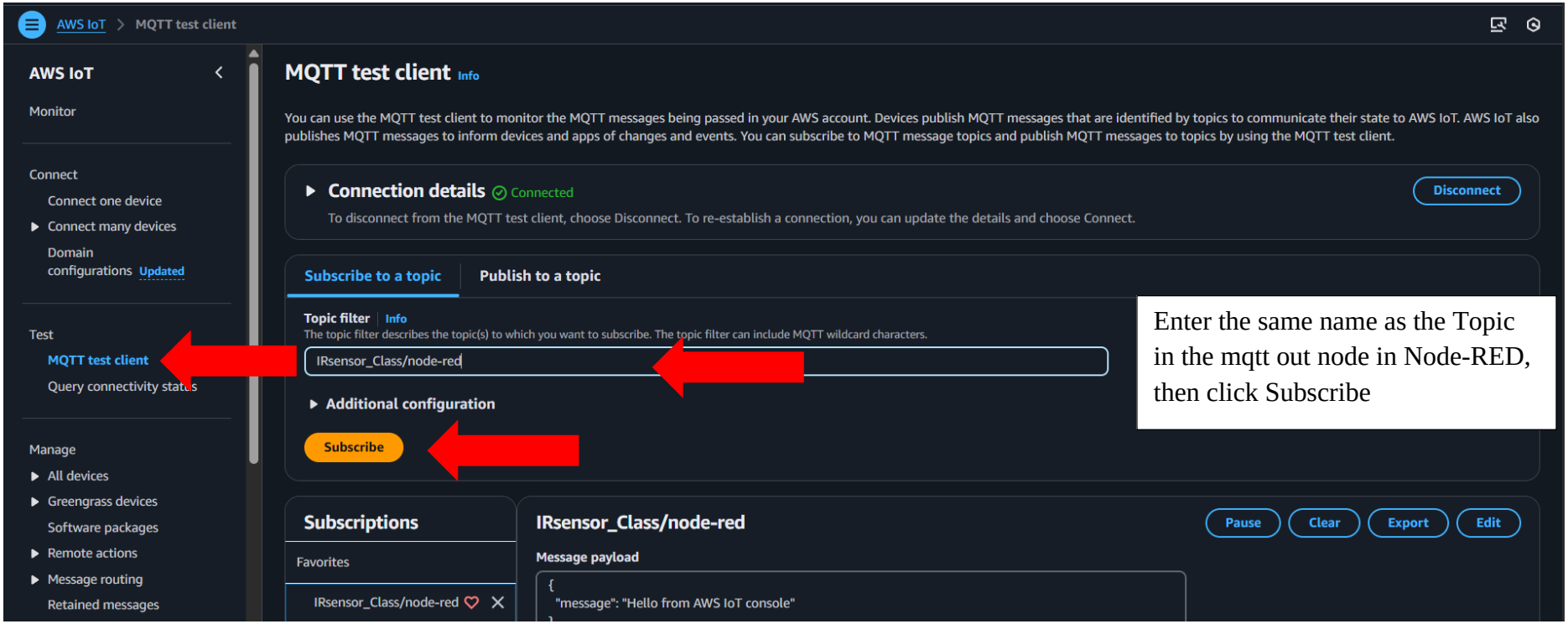
Select Use an existing role and select the newly created role.

Click create function.

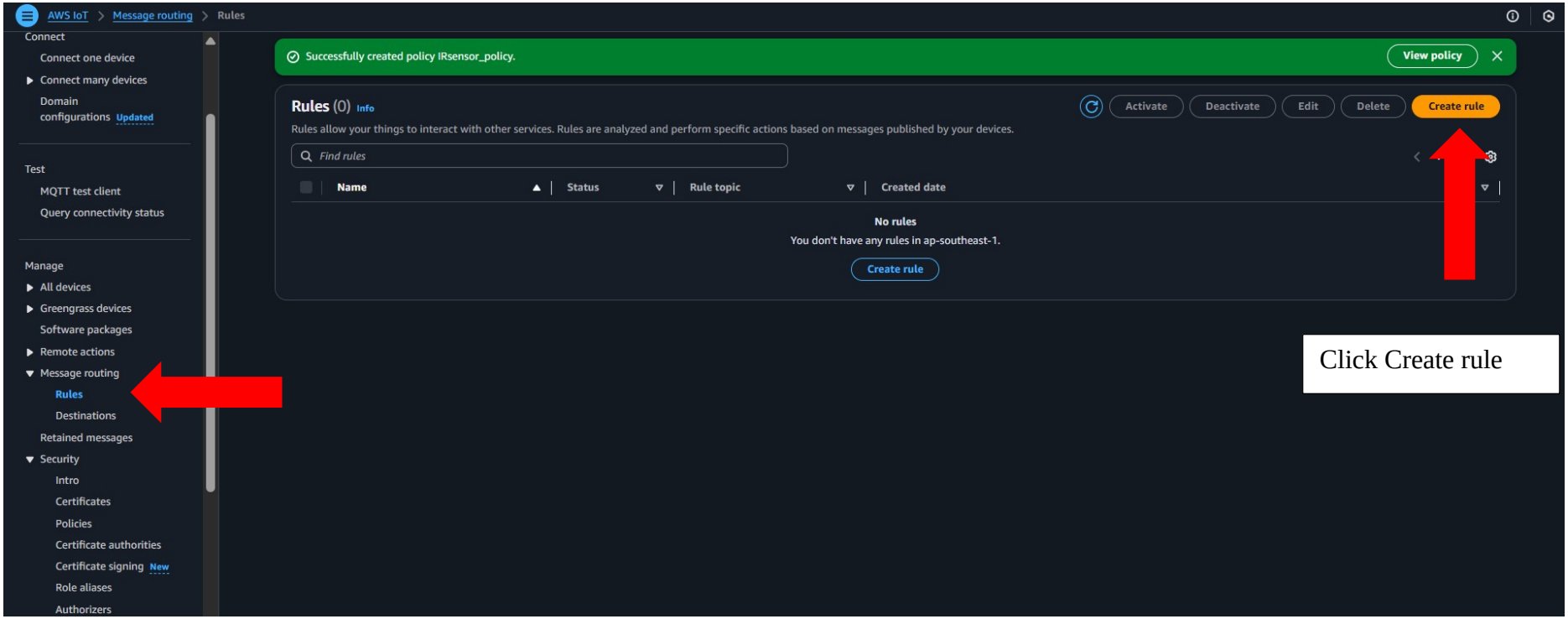
Insert the code



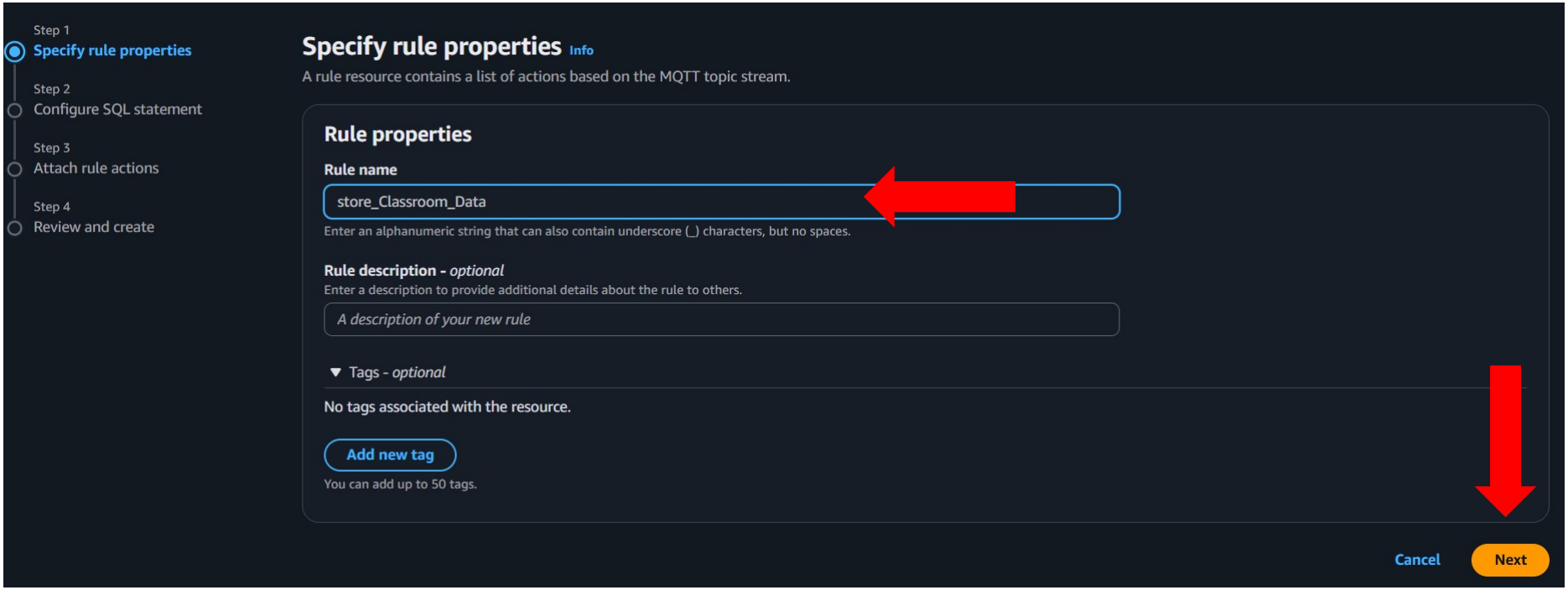
Navigate to the MQTT test client under *Test in AWS IoT Core*



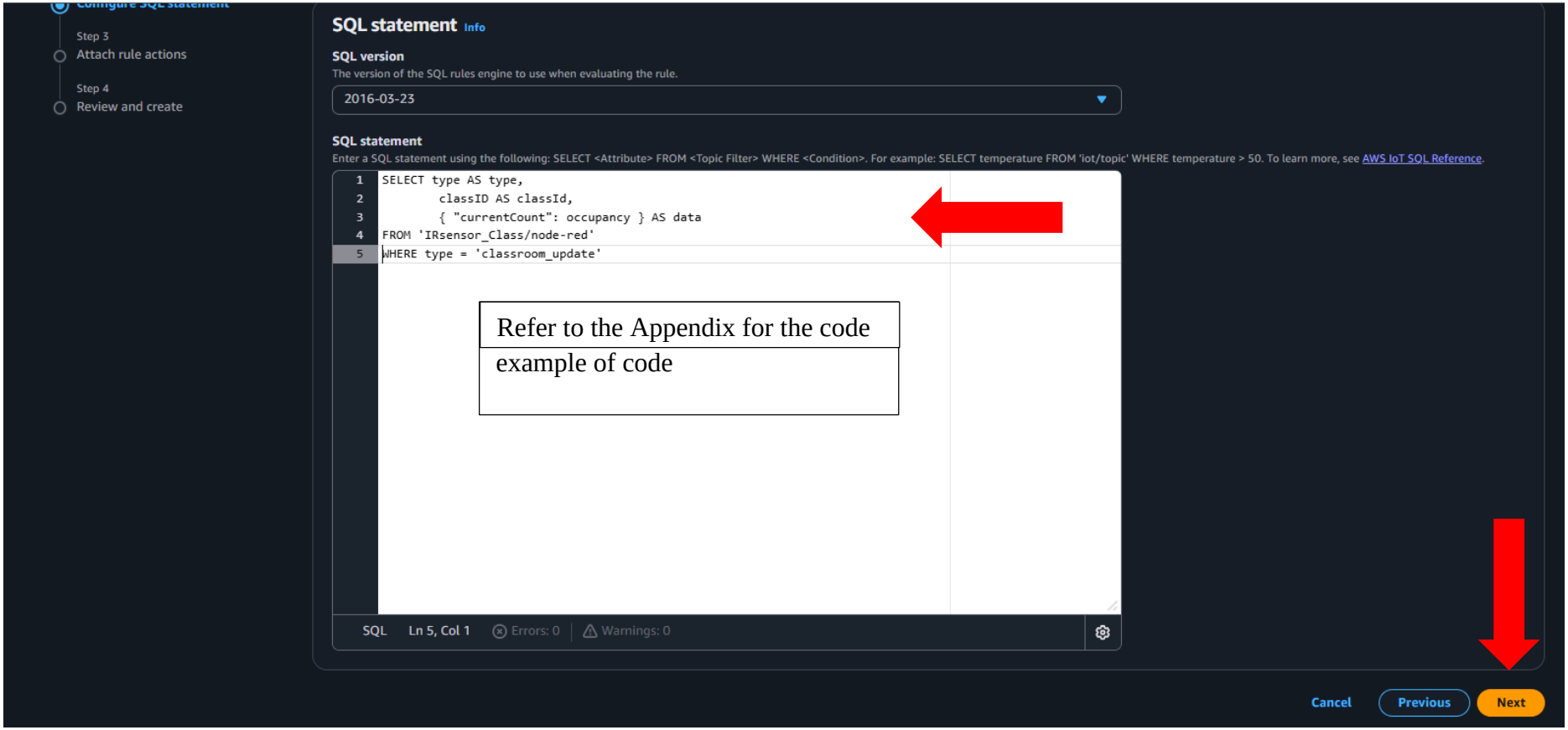
Navigate to the Rules under the *Message routing*



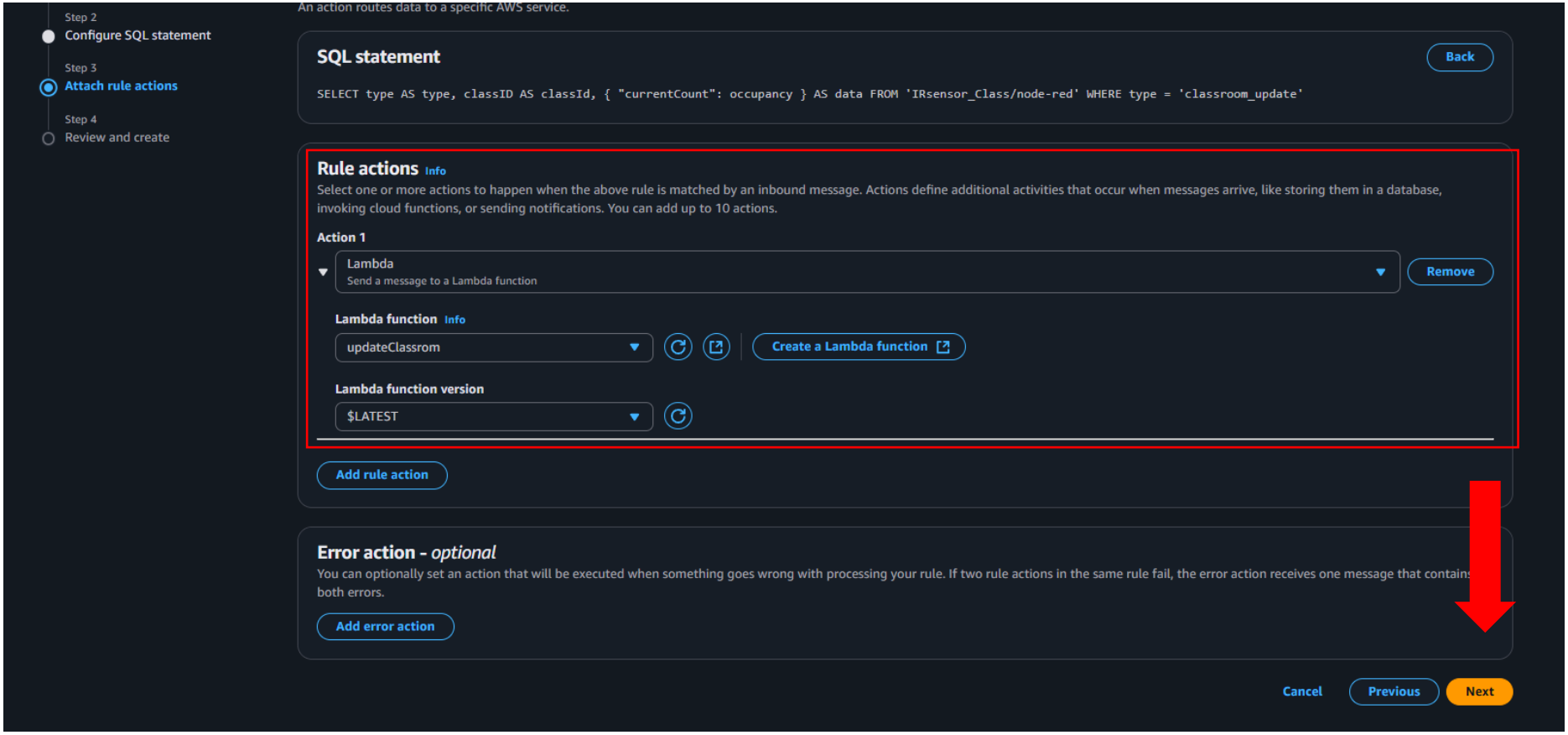
Input Rule name for the first rule



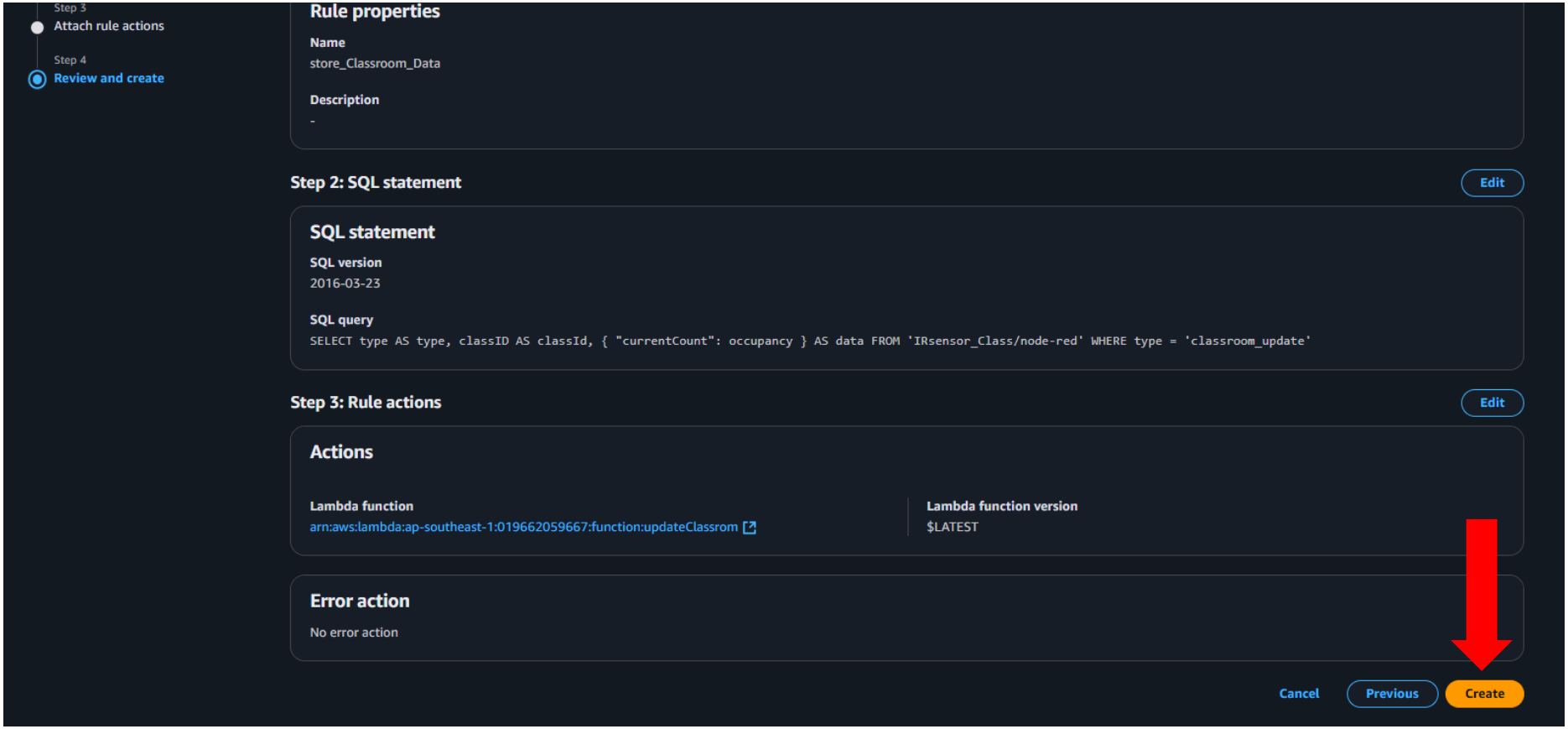
Input the SQL queries



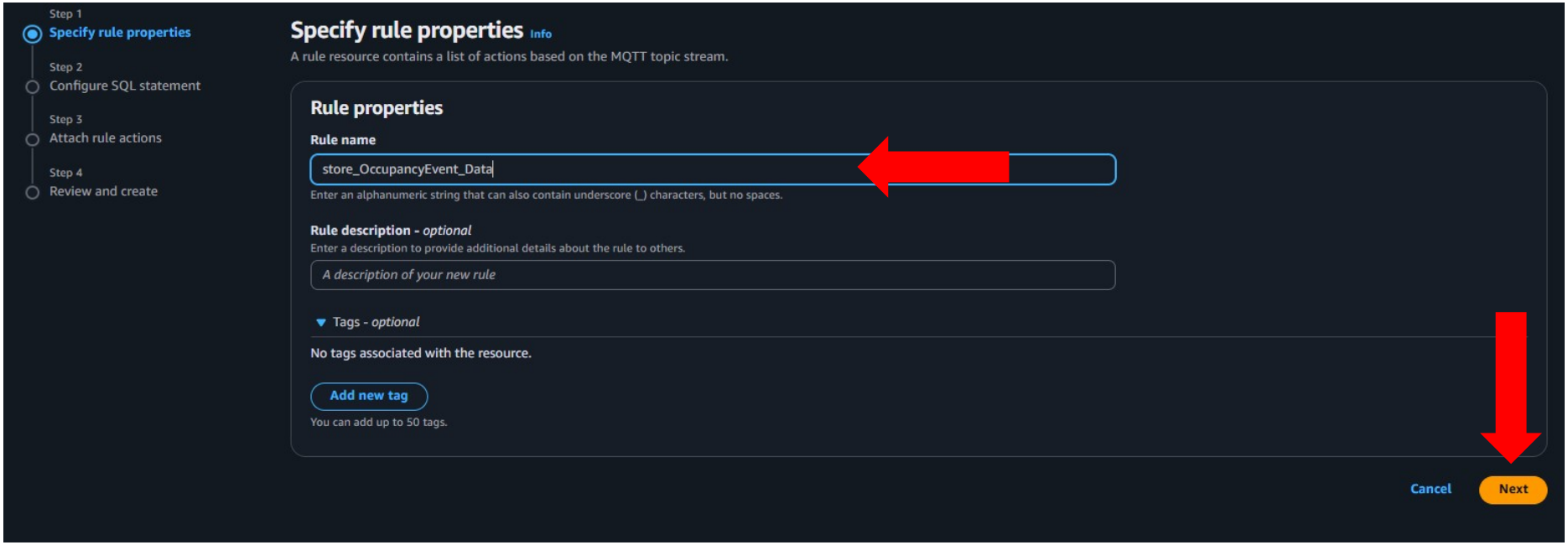
Configure the Rule actions as below



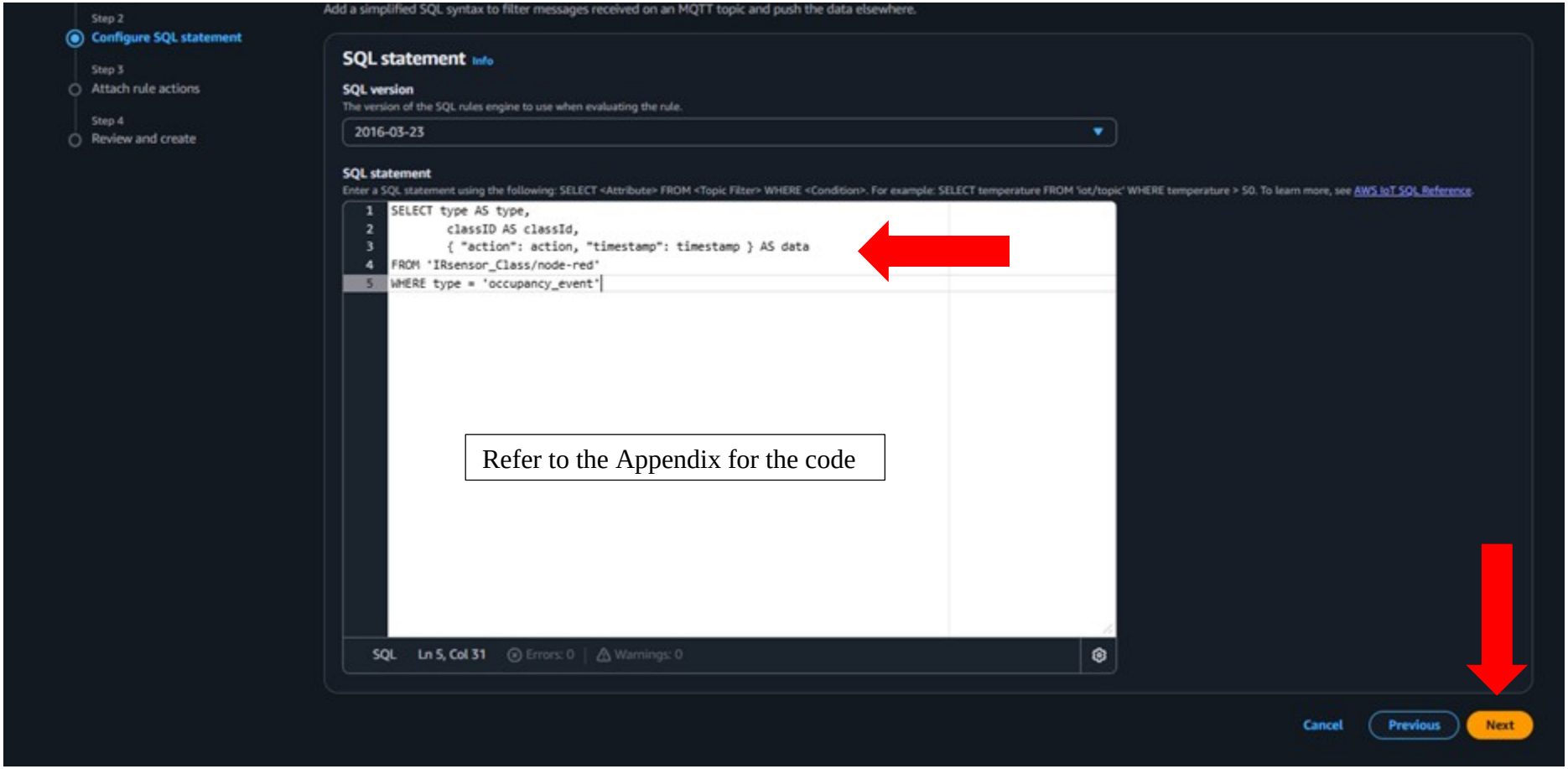
Click Create



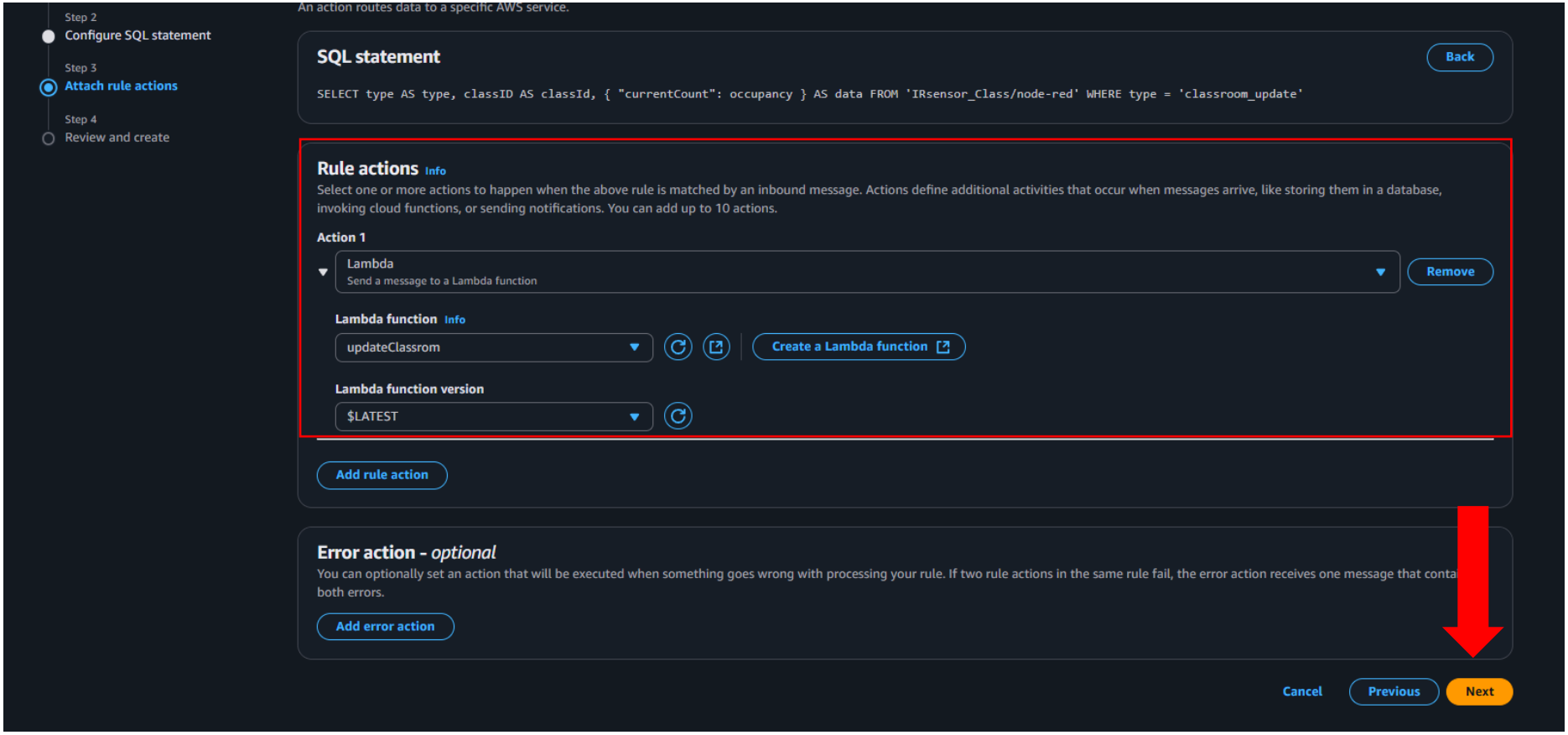
Input Rule name for the second rule



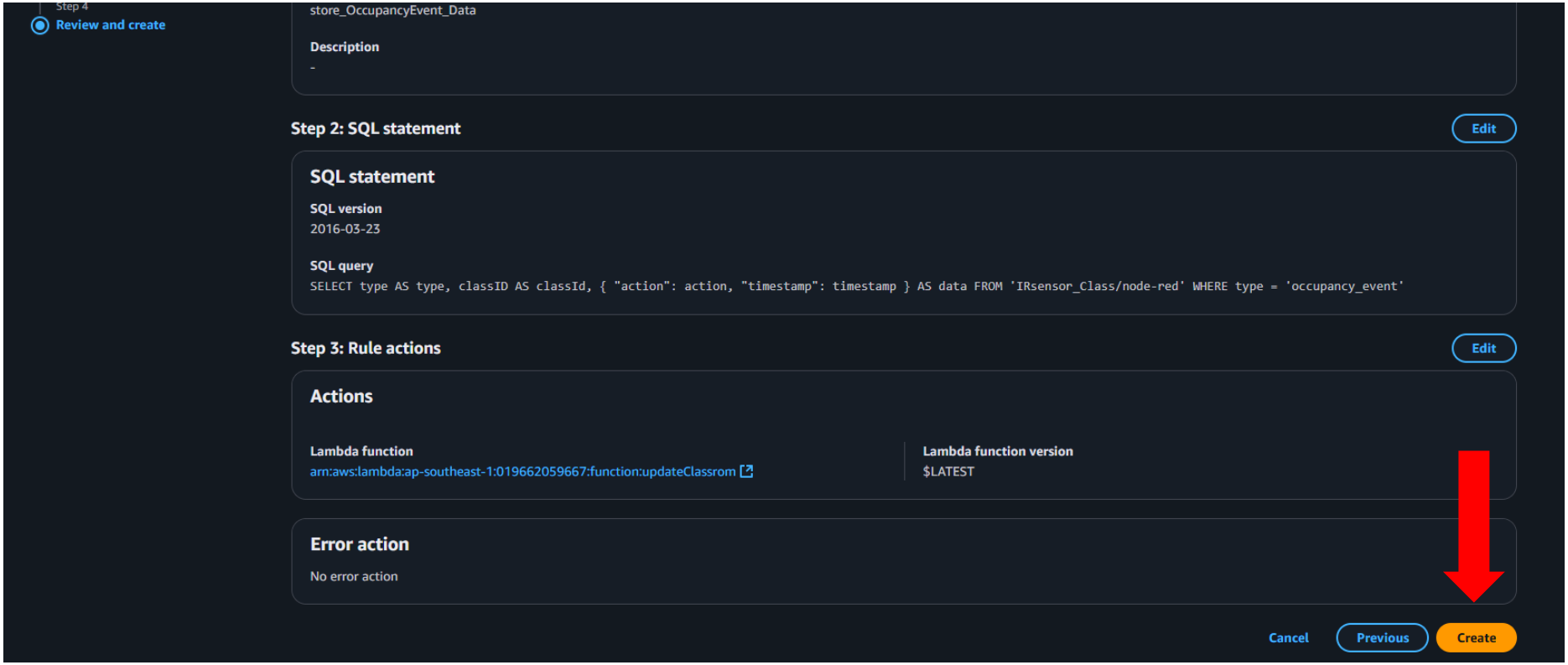
Input the SQL queries



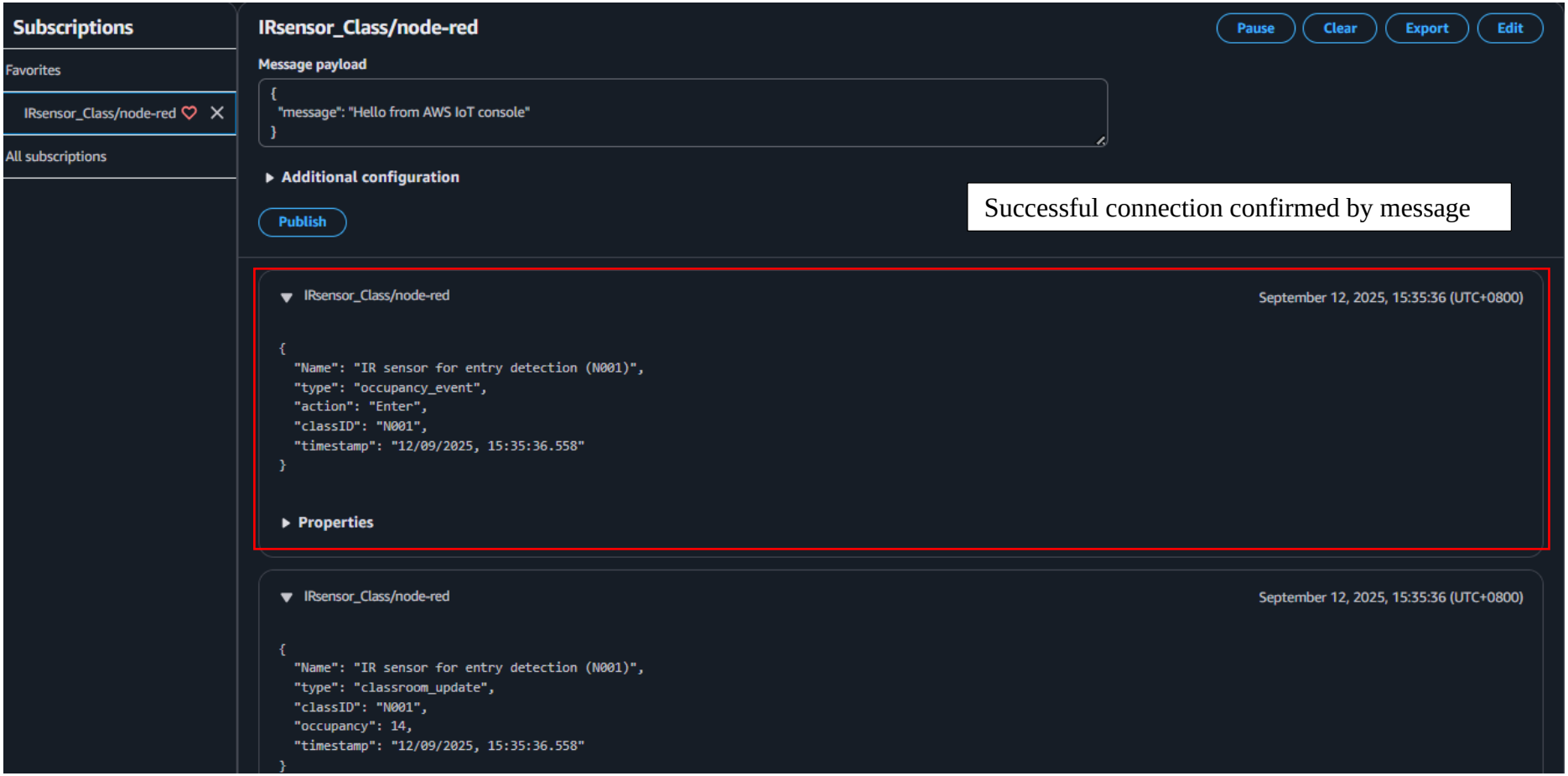
Configure the Rule actions as below



Click Create

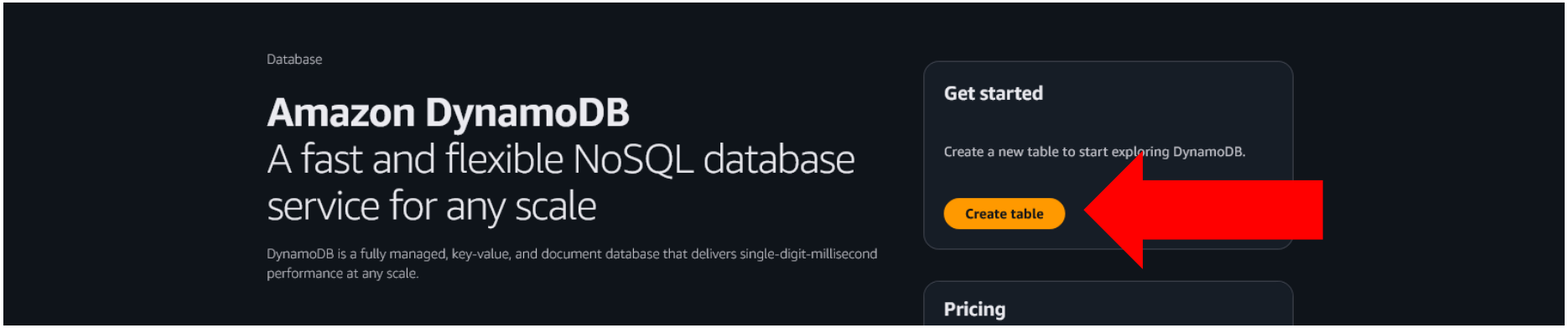


Simulate Node-RED to test its connection to IoT Core

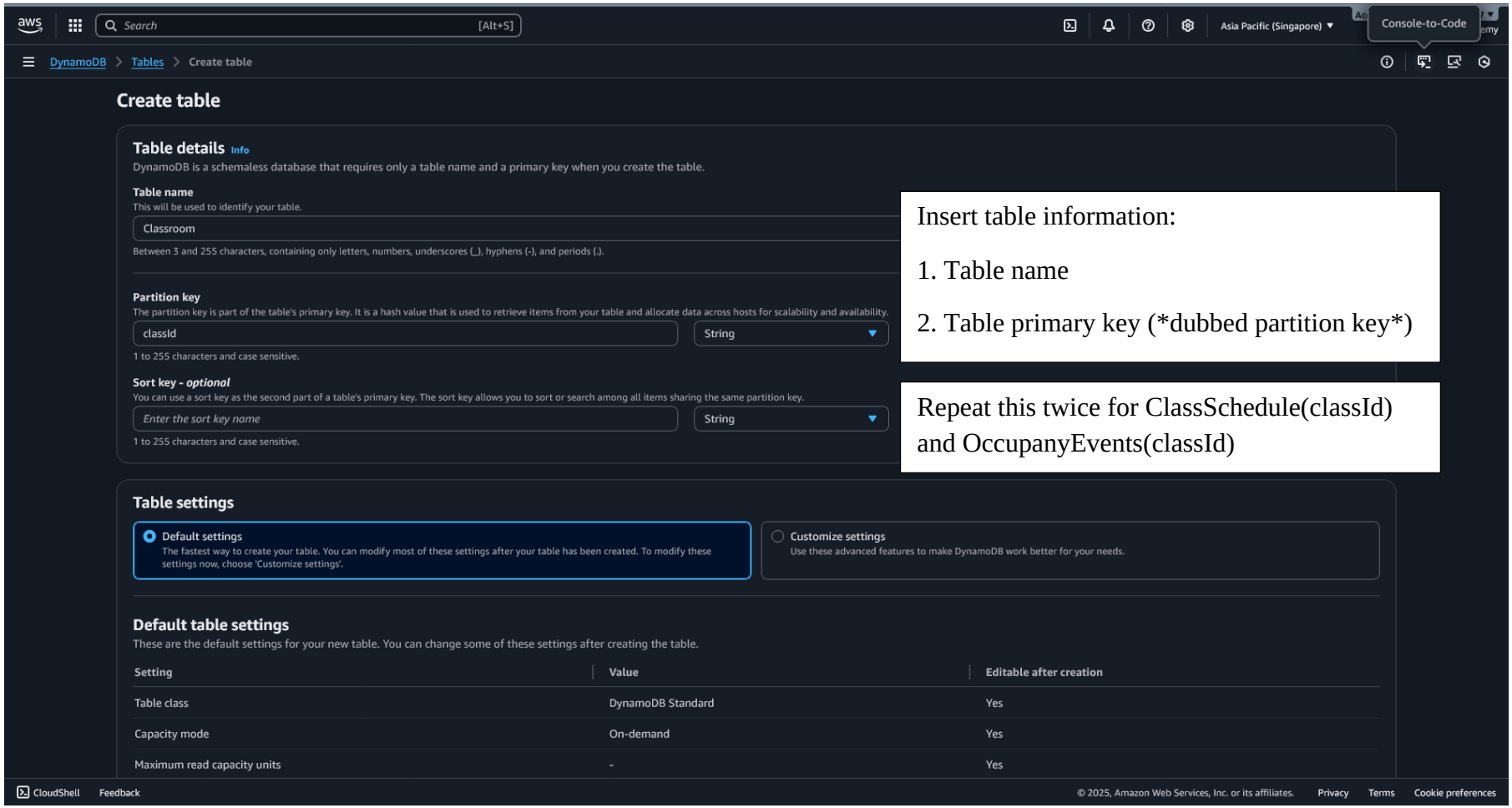


DynamoDB

Create Table



Insert Table Information



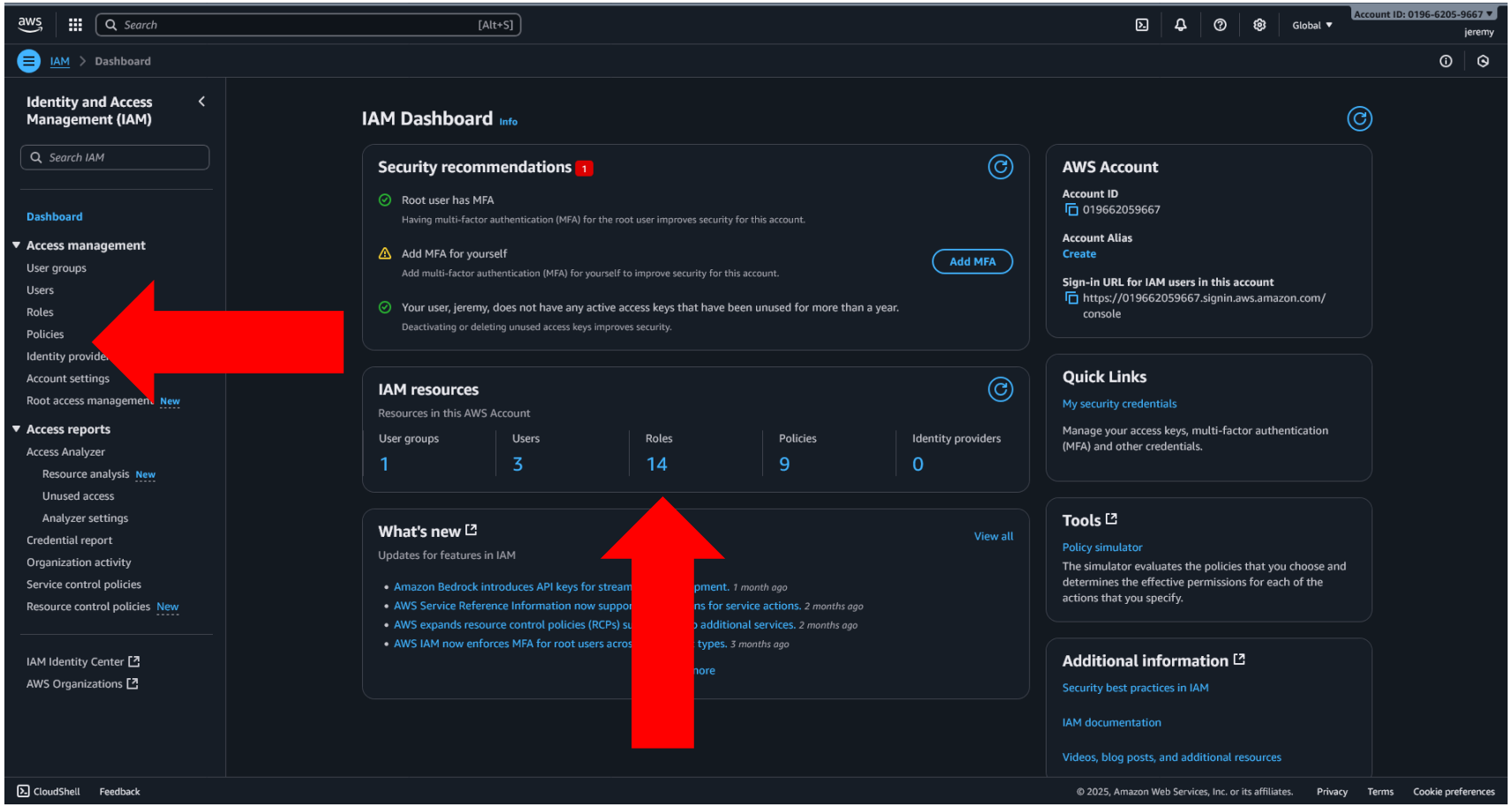
Insert table information:

1. Table name
2. Table primary key (*dubbed partition key*)

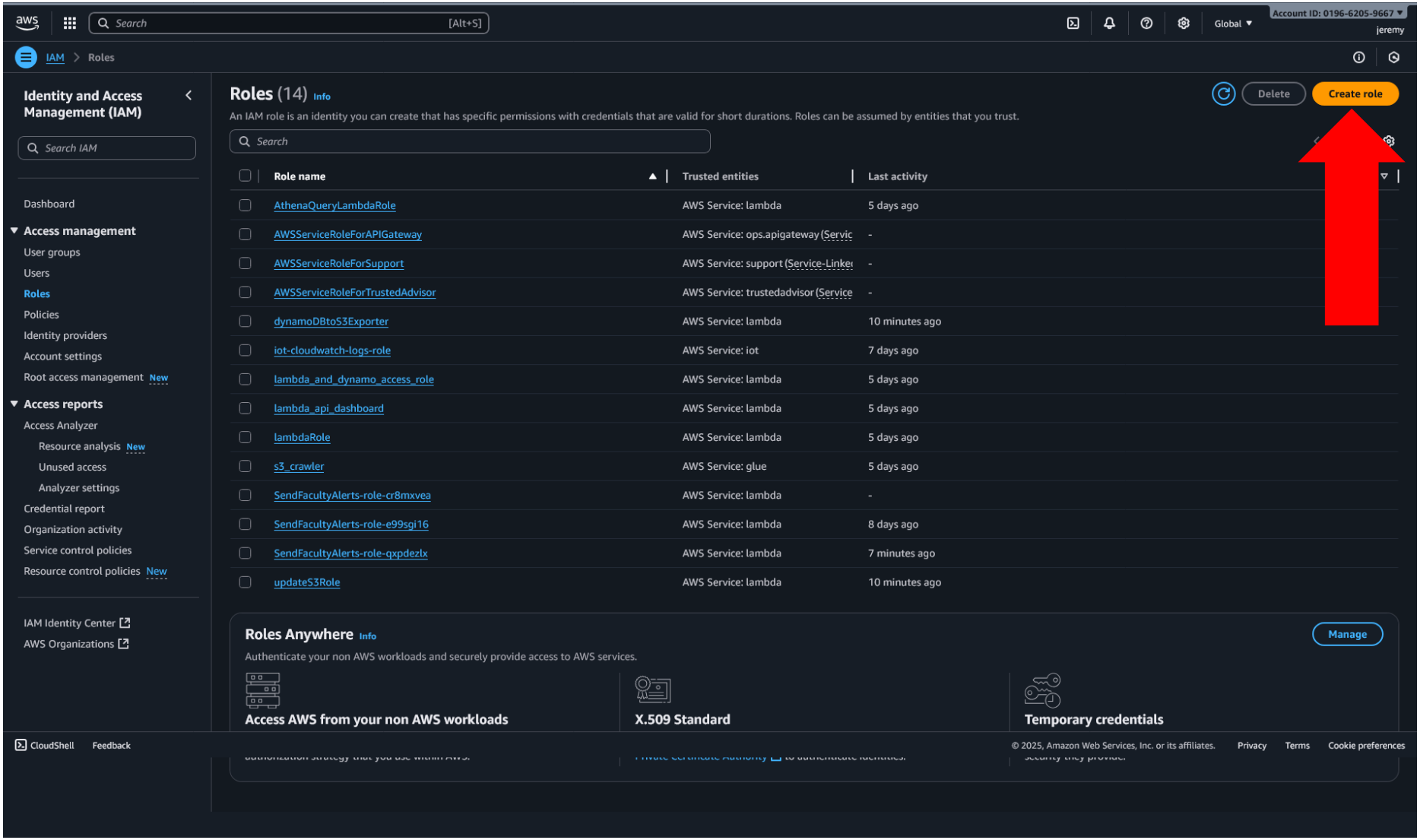
Repeat this twice for ClassSchedule(classId) and OccupanyEvents(classId)

Setup Database for Querying

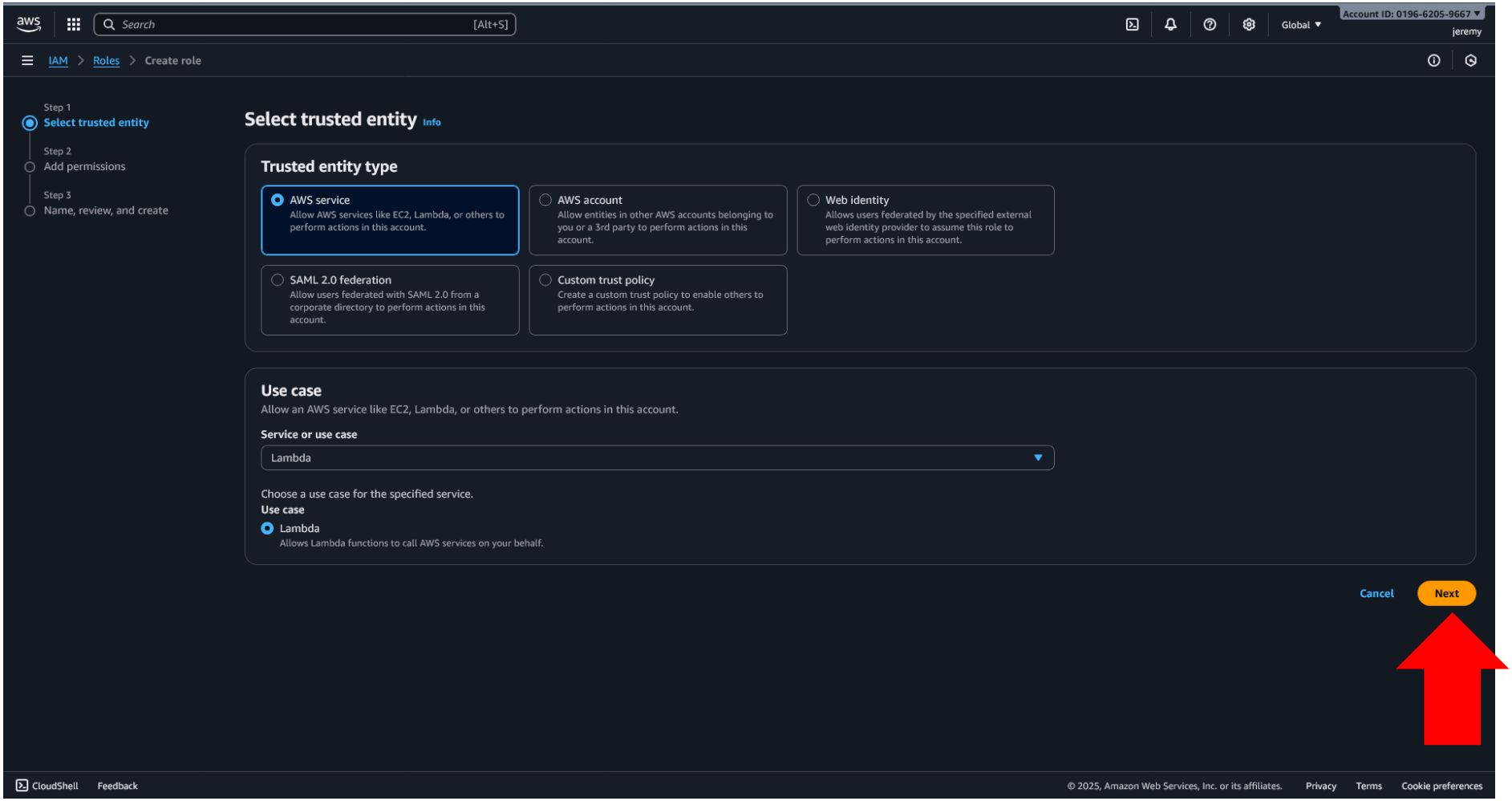
Create Role for Lambda

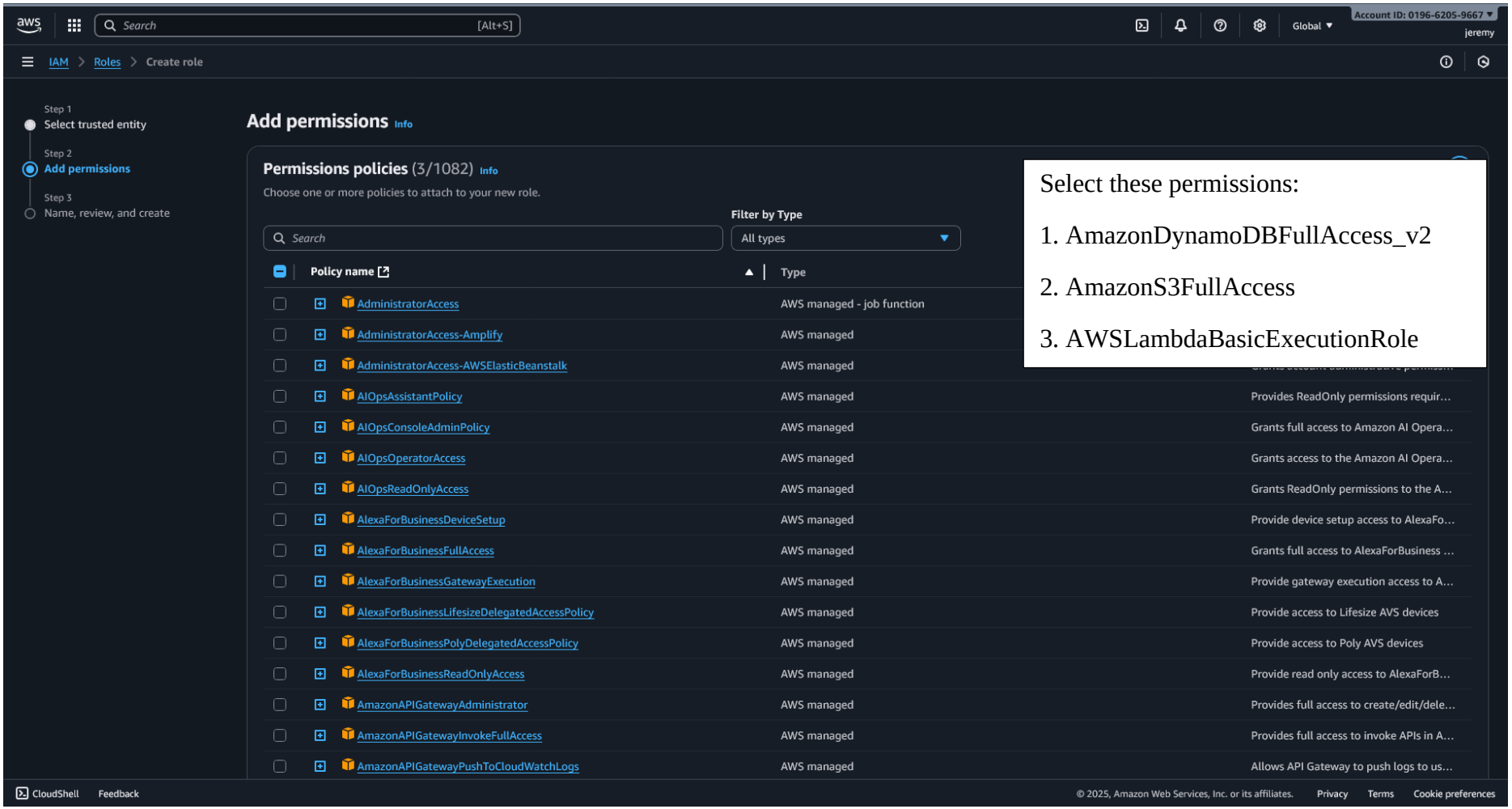


Navigate to the role section and press create role

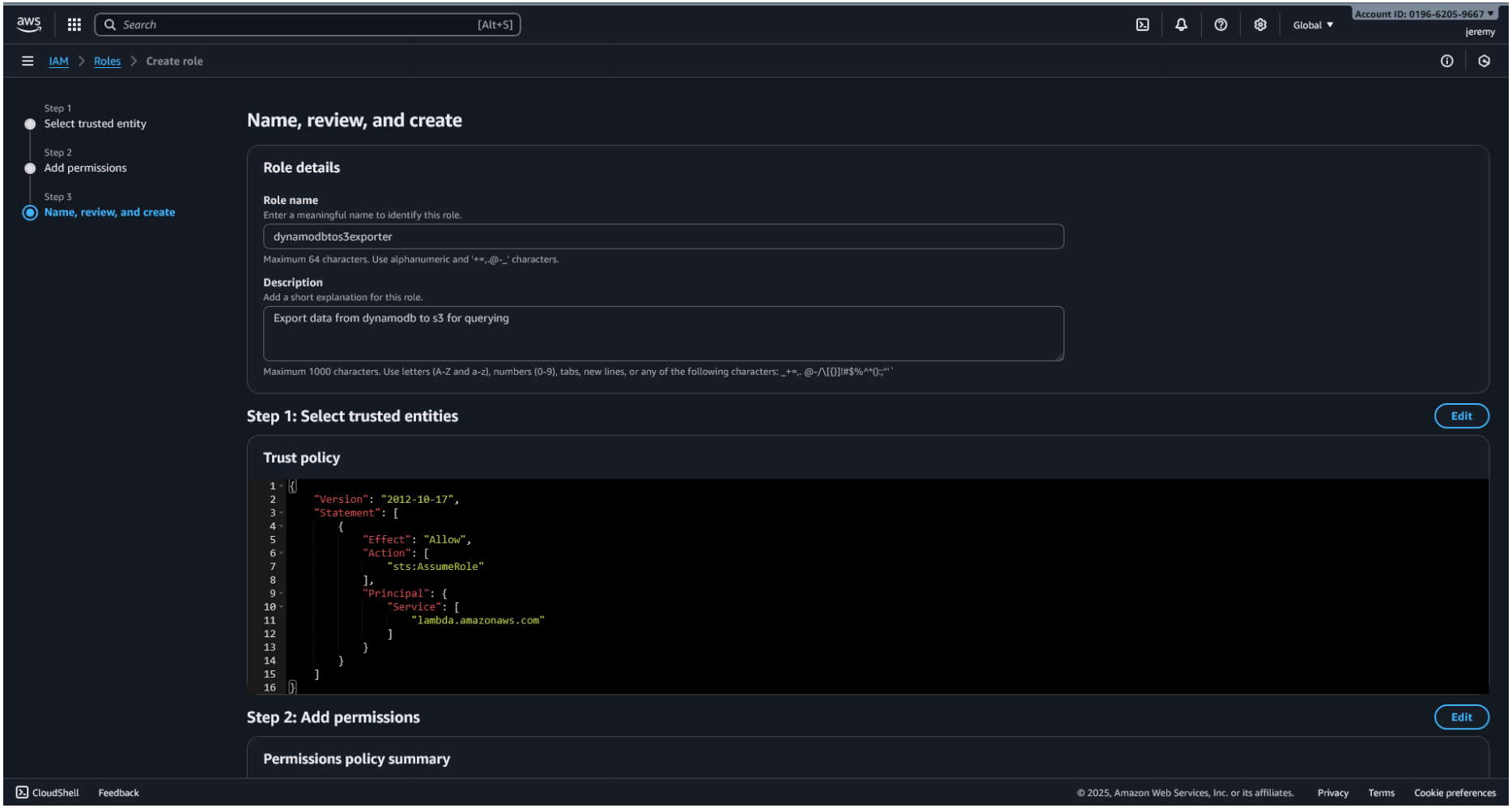


Select Lambda under use case and click next

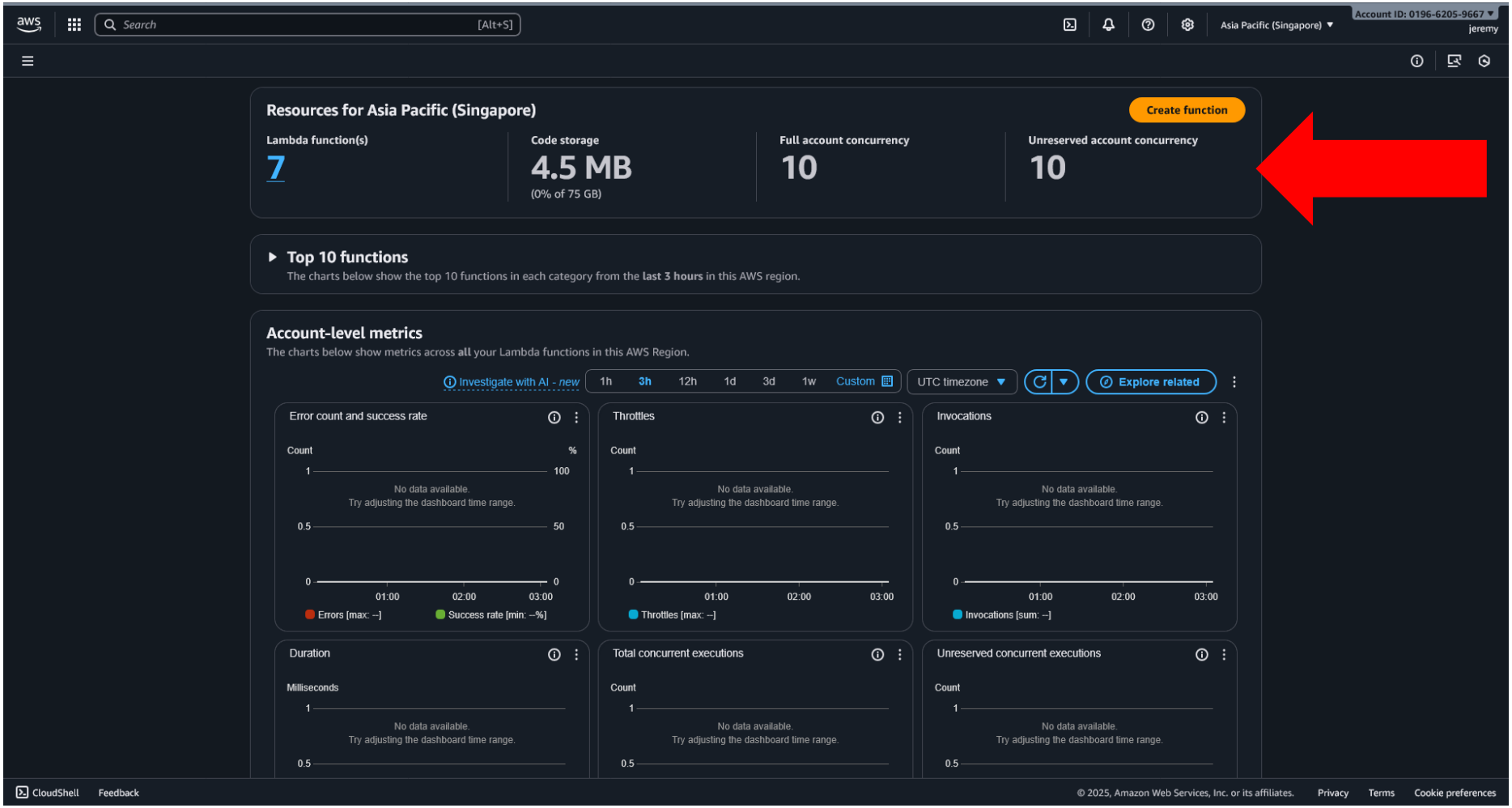


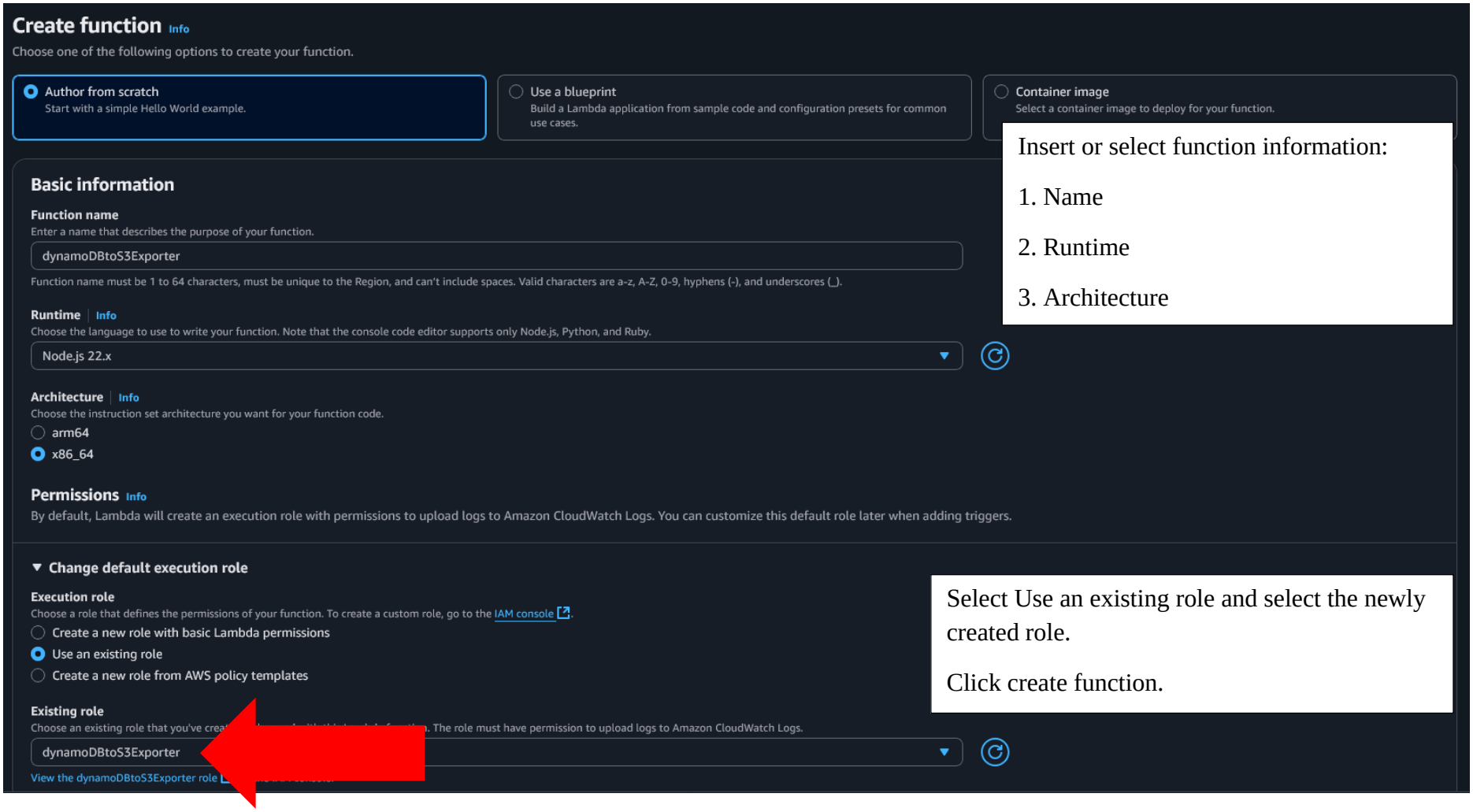

Search [Alt+S]
Global
Account ID: 0196-6205-9667
jeremy
IAM > Roles > Create role
Step 1
Select trusted entity
Step 2
Add permissions
Step 3
Name, review, and create
Add permissions Info
Permissions policies (3/1082) Info
Choose one or more policies to attach to your new role.
Filter by Type
Search
All types
Policy name
Type
AdministratorAccess AWS managed - job function
AdministratorAccess-Amplify AWS managed
AdministratorAccess-AWSElasticBeanstalk AWS managed
AIOpsAssistantPolicy AWS managed Provides ReadOnly permissions requir...
AIOpsConsoleAdminPolicy AWS managed Grants full access to Amazon AI Opera...
AIOpsOperatorAccess AWS managed Grants access to the Amazon AI Opera...
AIOpsReadOnlyAccess AWS managed Grants ReadOnly permissions to the A...
AlexaForBusinessDeviceSetup AWS managed Provide device setup access to AlexaFo...
AlexaForBusinessFullAccess AWS managed Grants full access to AlexaForBusiness ...
AlexaForBusinessGatewayExecution AWS managed Provide gateway execution access to A...
AlexaForBusinessLifesizeDelegatedAccessPolicy AWS managed Provide access to Lifesize AVS devices
AlexaForBusinessPolyDelegatedAccessPolicy AWS managed Provide access to Poly AVS devices
AlexaForBusinessReadOnlyAccess AWS managed Provide read only access to AlexaForB...
AmazonAPIGatewayAdministrator AWS managed Provides full access to create/edit/dele...
AmazonAPIGatewayInvokeFullAccess AWS managed Provides full access to invoke APIs in A...
AmazonAPIGatewayPushToCloudWatchLogs AWS managed Allows API Gateway to push logs to us...
Select these permissions:
1. AmazonDynamoDBFullAccess_v2
2. AmazonS3FullAccess
3. AWSLambdaBasicExecutionRole
CloudShell
Feedback
© 2025, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

Give the role a name and a description

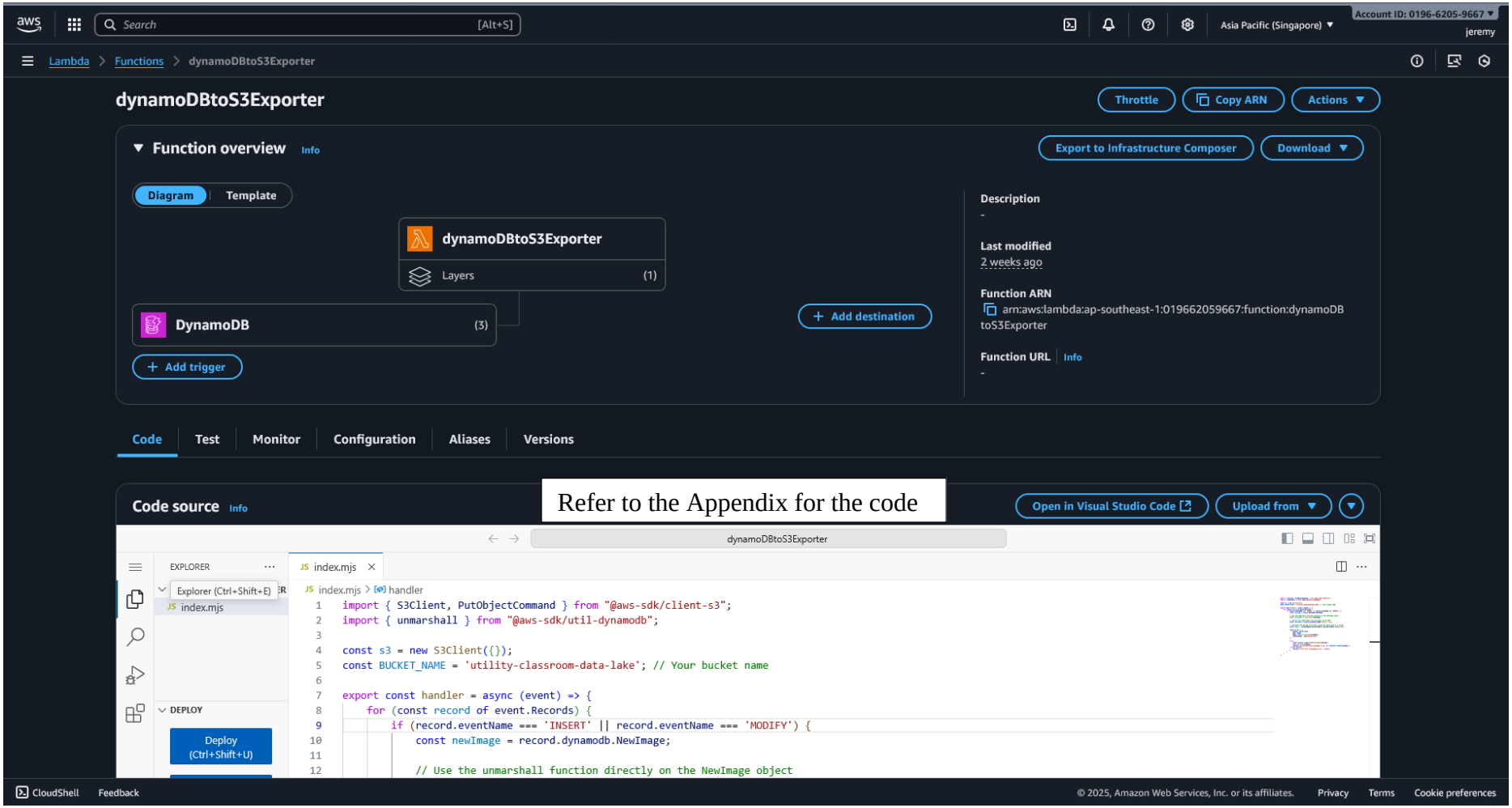


Create Lambda Function

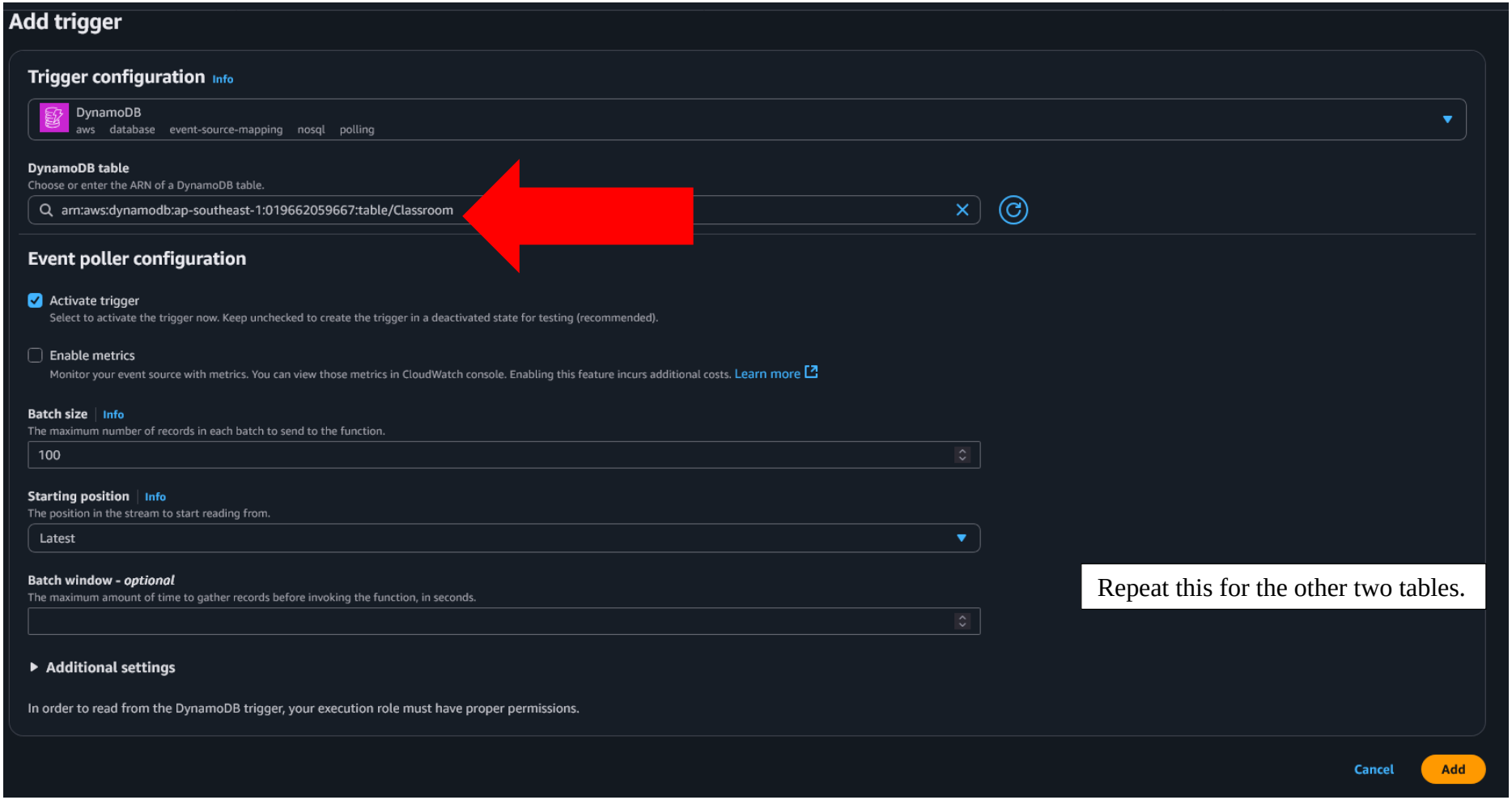



Create function Info
Choose one of the following options to create your function.
Author from scratch
Start with a simple Hello World example.
Use a blueprint
Build a Lambda application from sample code and configuration presets for common use cases.
Container image
Select a container image to deploy for your function.
Basic information
Function name
Enter a name that describes the purpose of your function.
dynamoDBtoS3Exporter
Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).
Runtime Info
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.
Node.js 22.x
Architecture Info
Choose the instruction set architecture you want for your function code.
arm64
x86_64
Permissions Info
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.
Change default execution role
Execution role
Choose a role that defines the permissions of your function. To create a custom role, go to the IAM console.
Create a new role with basic Lambda permissions
Use an existing role
Create a new role from AWS policy templates
Existing role
Choose an existing role that you've crea
. The role must have permission to upload logs to Amazon CloudWatch Logs.
dynamoDBtoS3Exporter
View the dynamoDBtoS3Exporter role
Insert or select function information:
1. Name
2. Runtime
3. Architecture
Select Use an existing role and select the newly created role.
Click create function.

Navigate to the newly created function and insert the code.



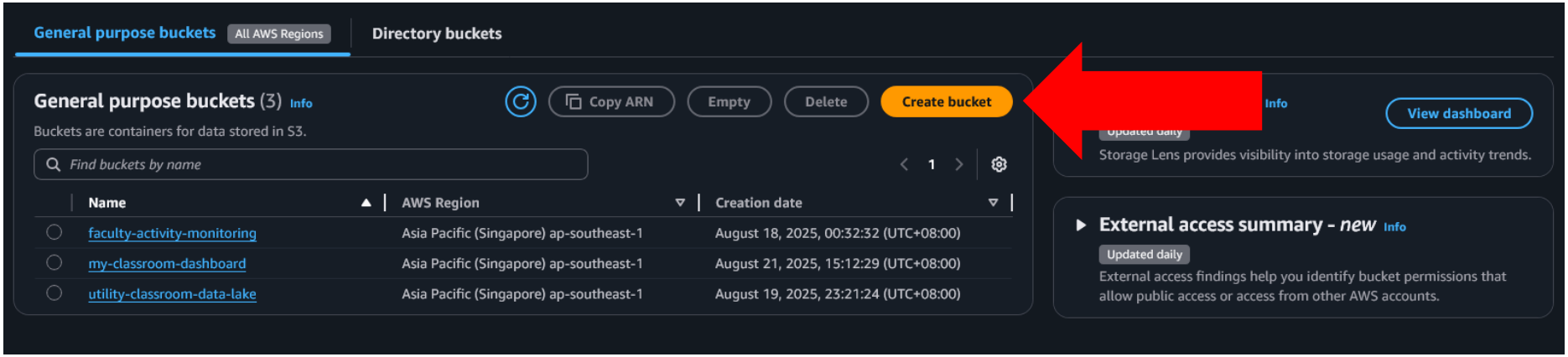
Click on the add trigger button and select trigger to be DynamoDB.

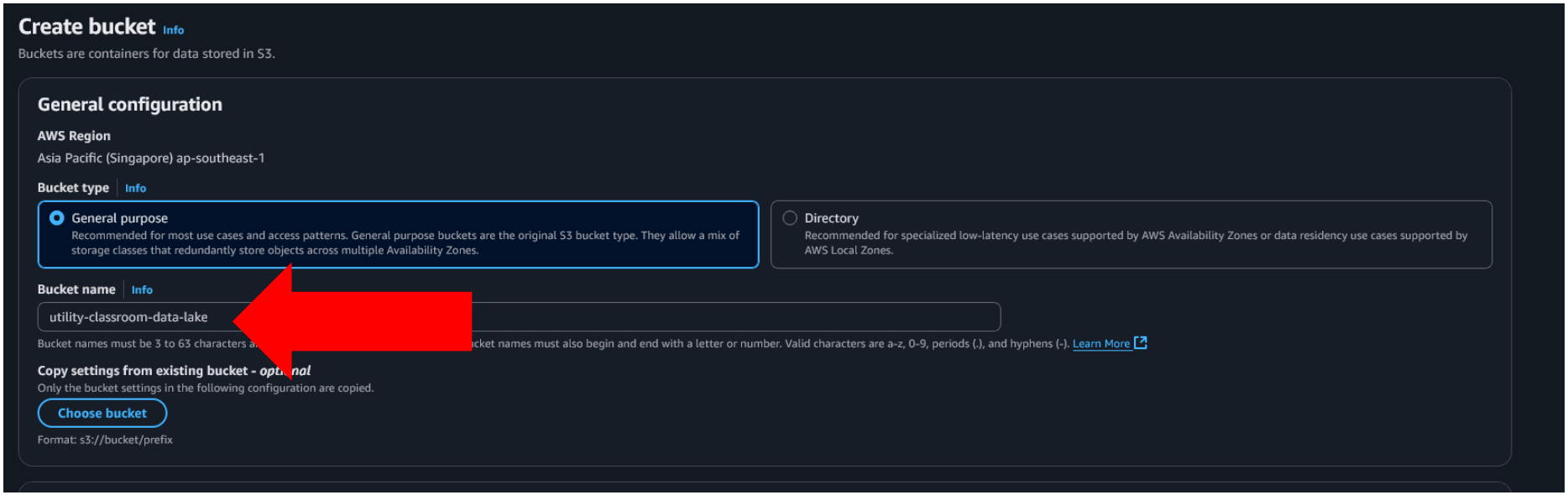


Repeat this for the other two tables.

S3

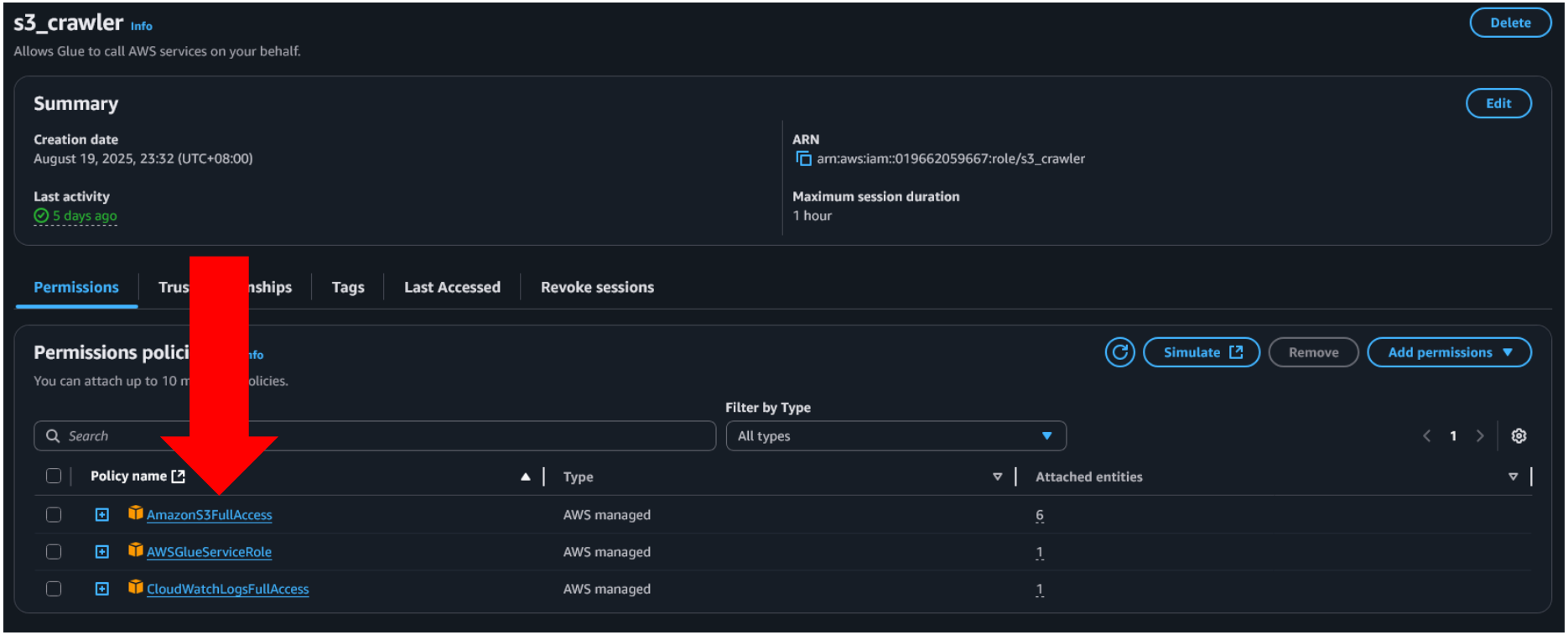
Navigate to s3 bucket and press create bucket.



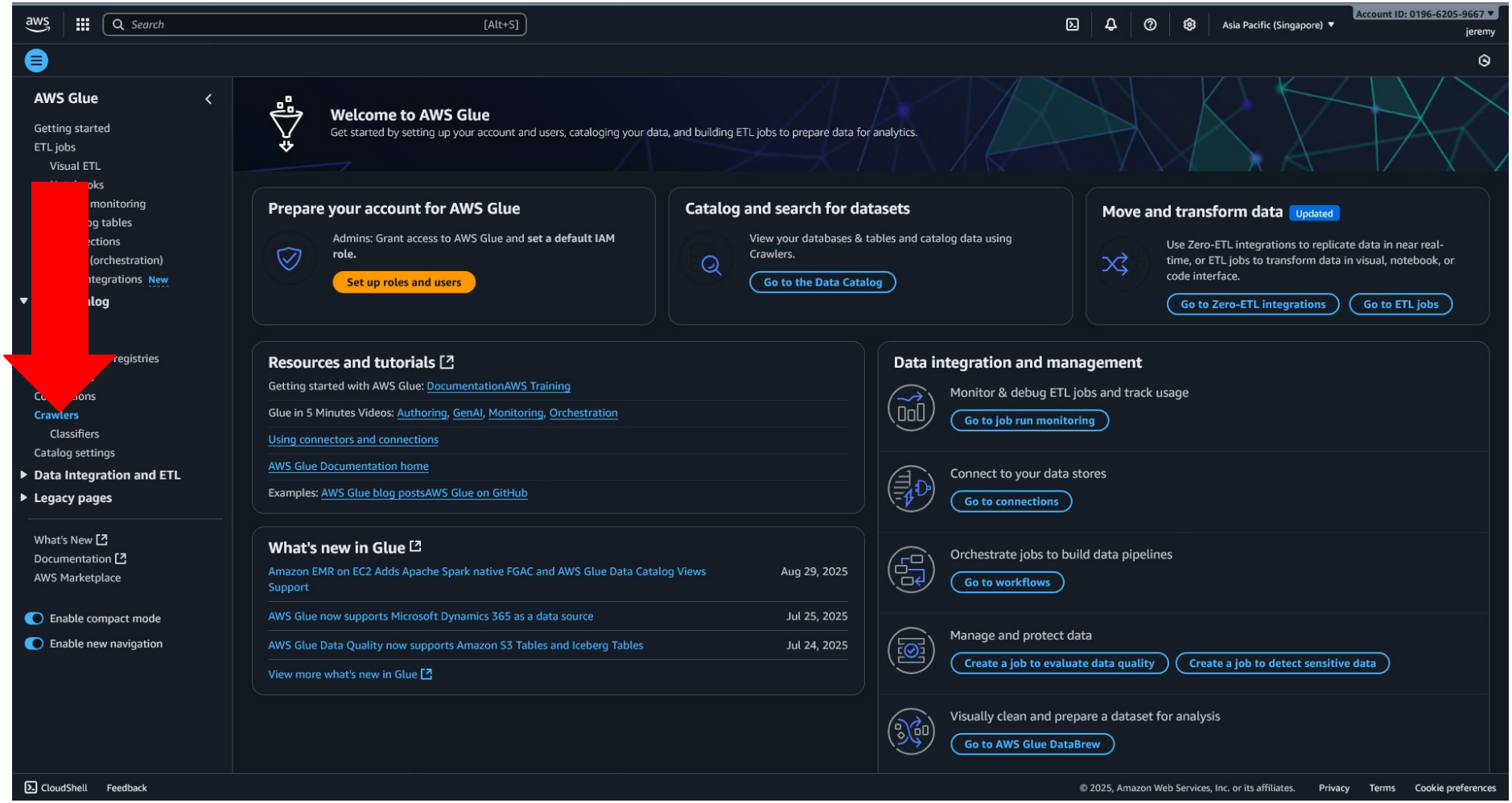


AWS Glue

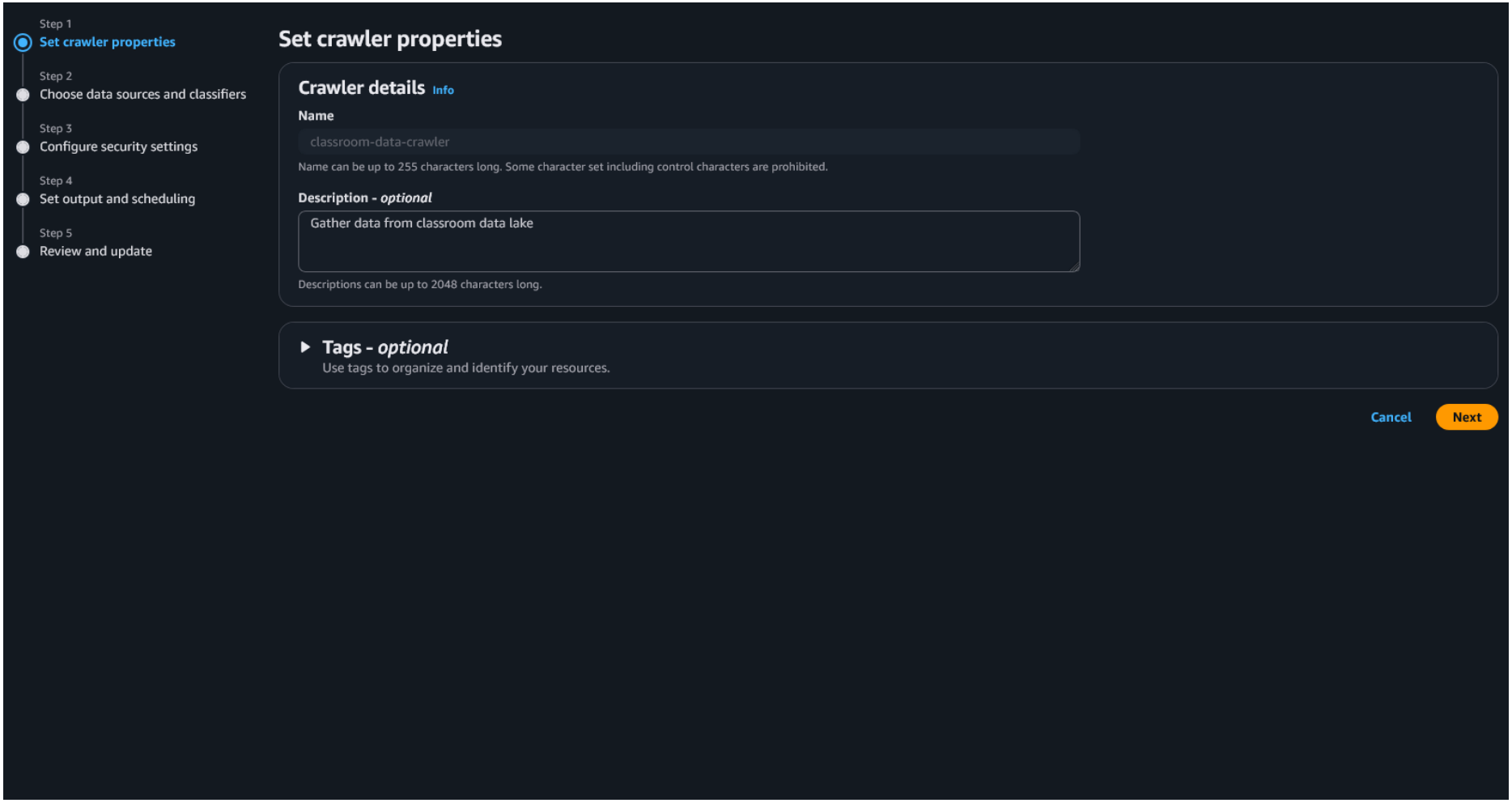
Create an IAM role



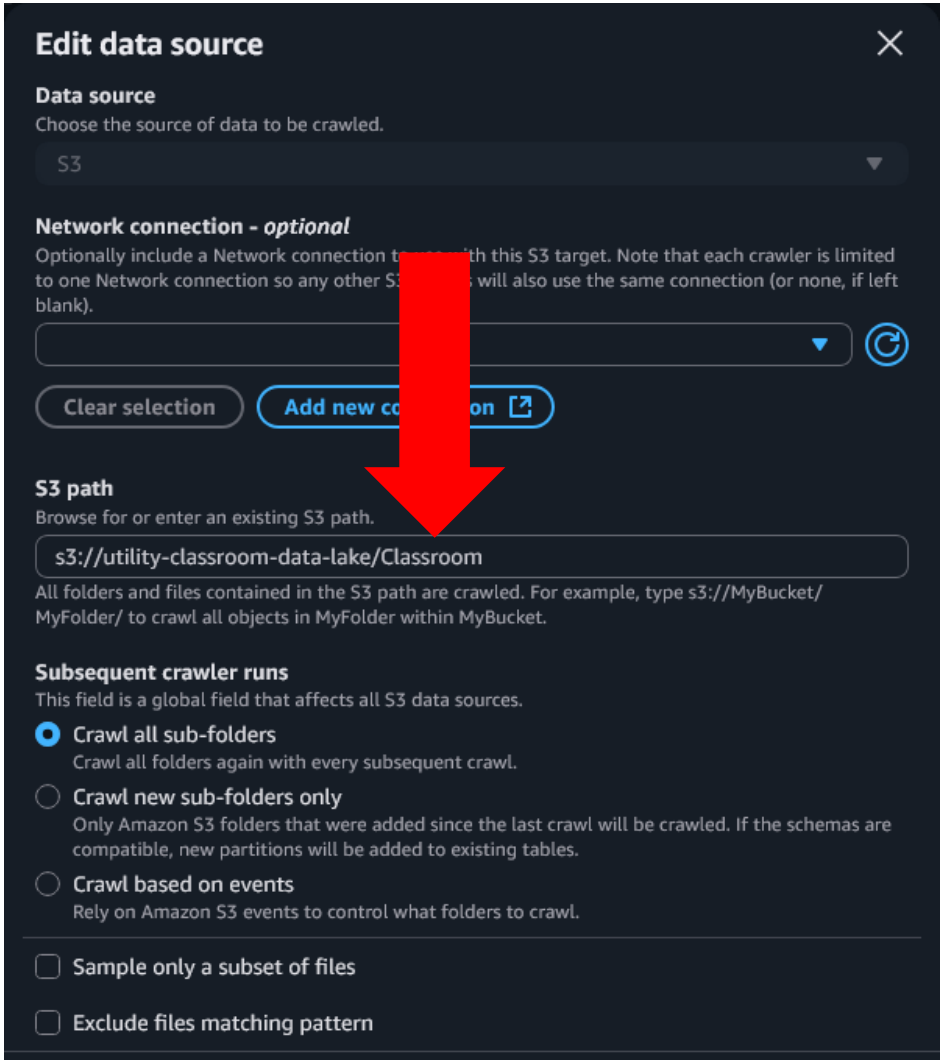
Navigate to the crawler section of the AWS Glue



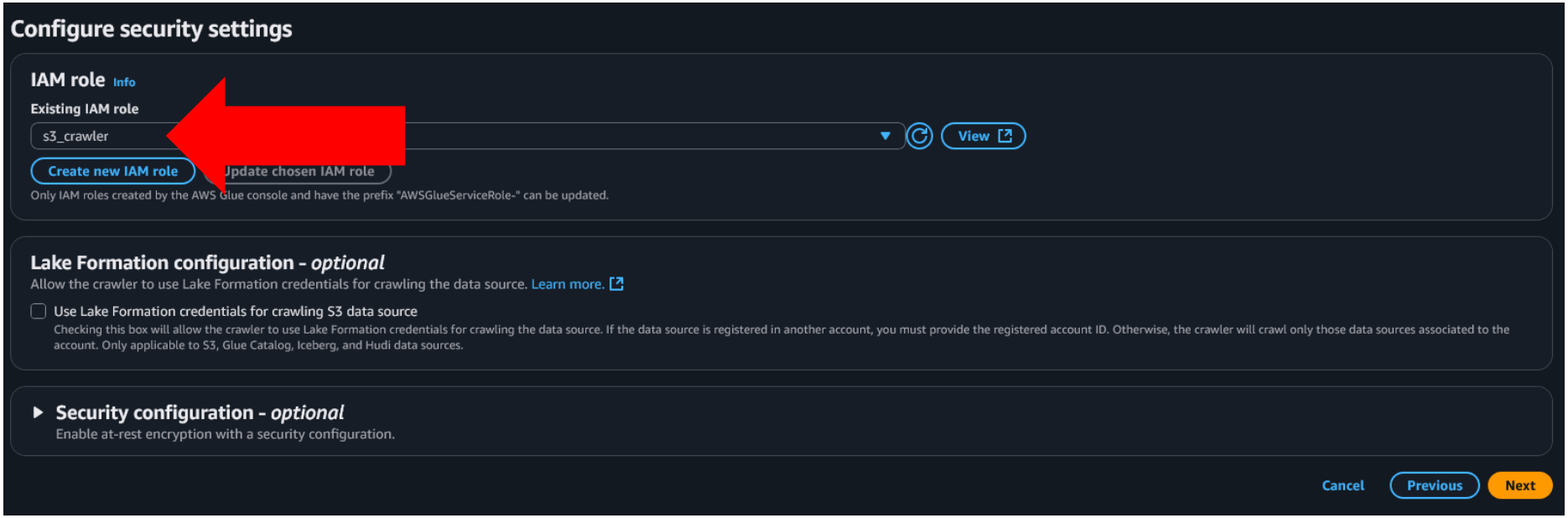
Create a crawler



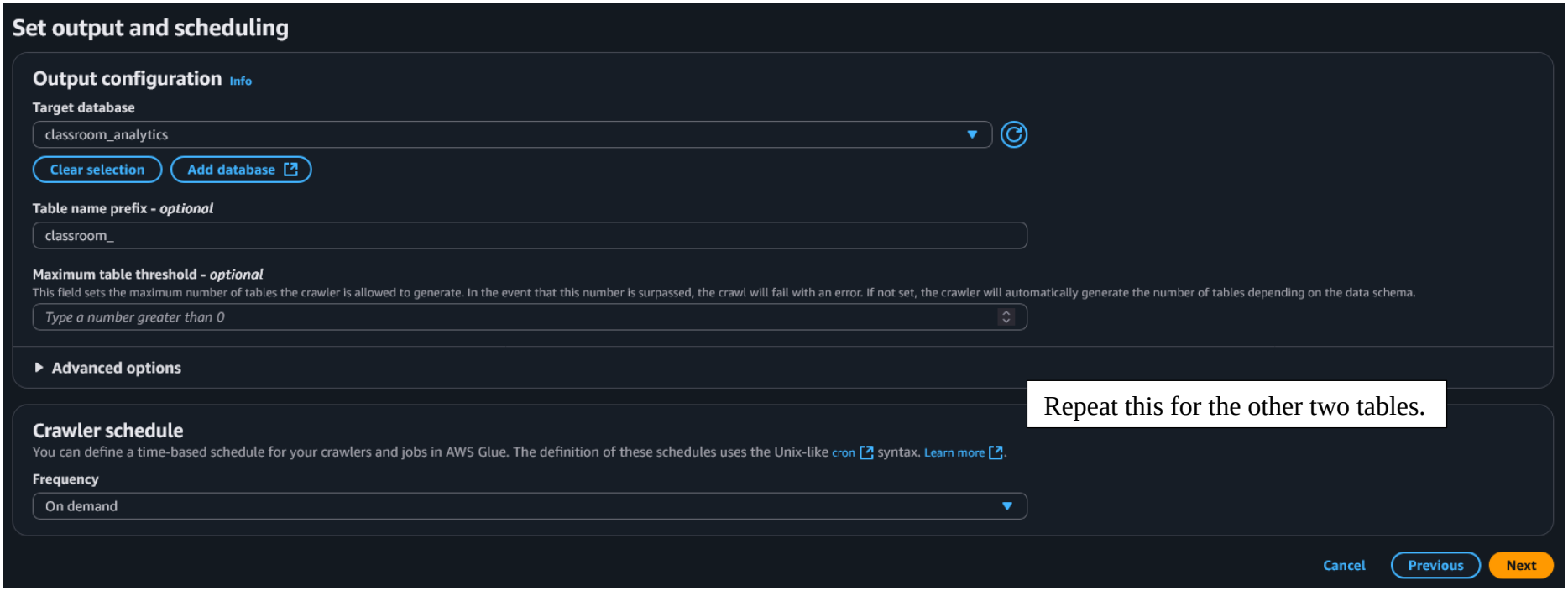
Select the Classroom data lake as the S3 source.



Select the created IAM role

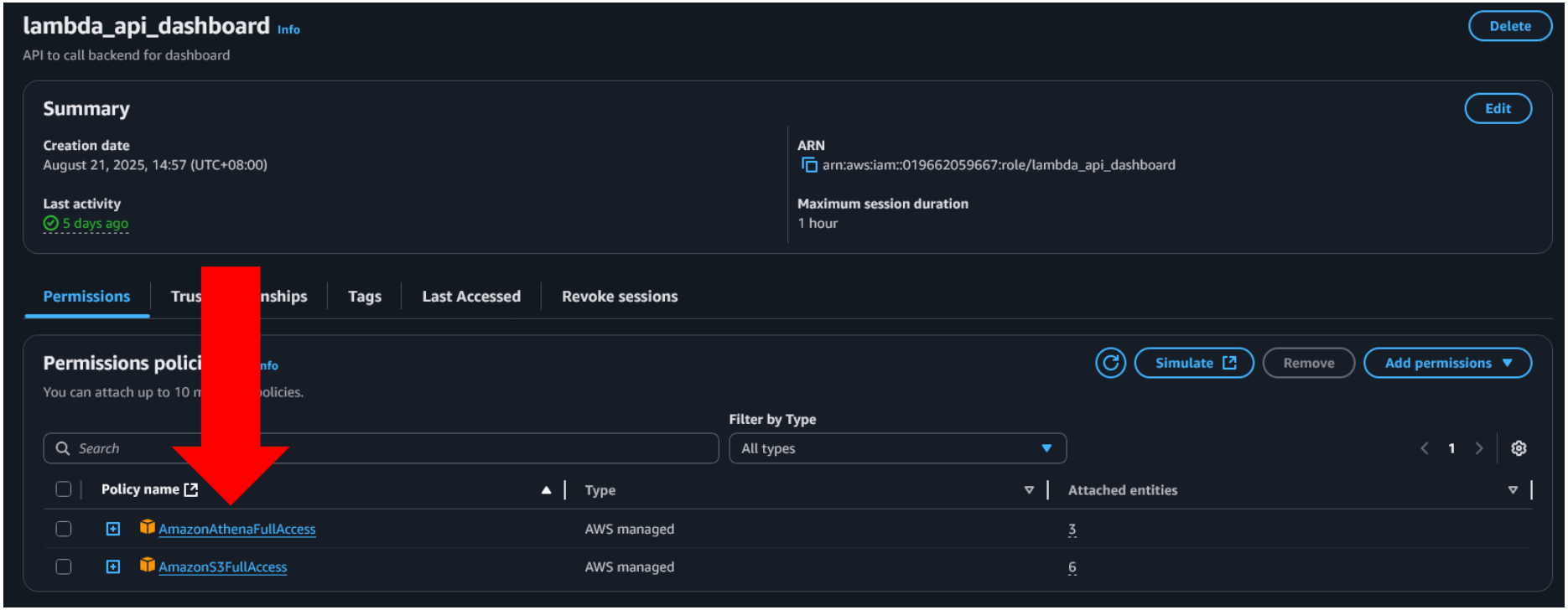


Create a database and table name prefix to store the output.
Set the schedule according to preference.

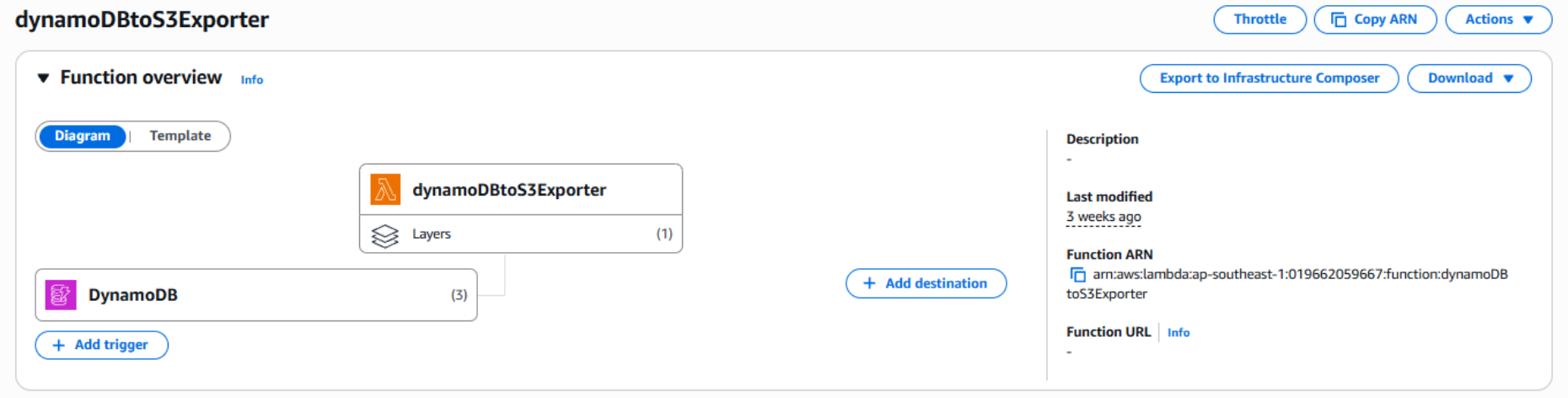


Querying the Database

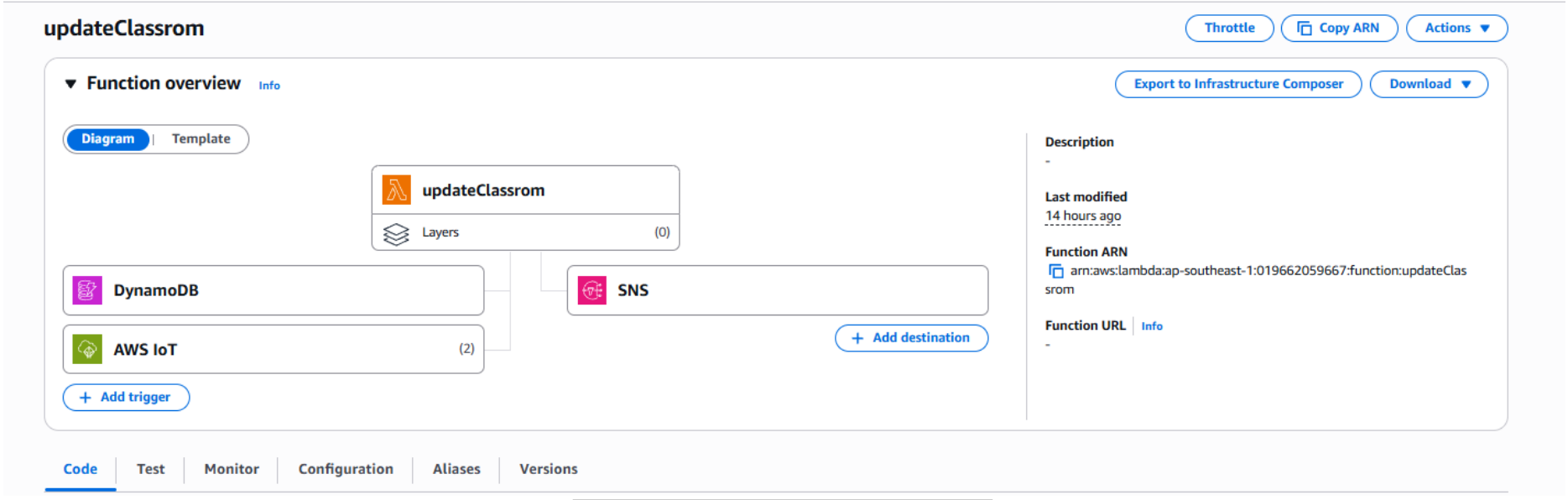
Create an IAM role



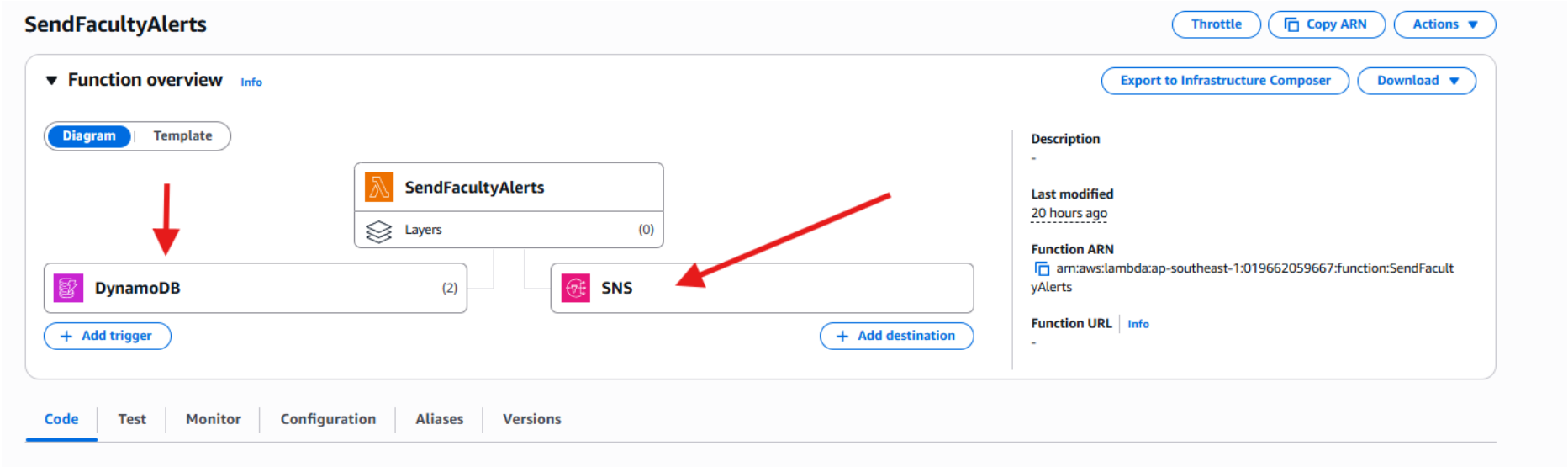
Create a lambda function and insert the necessary code.



Refer to the Appendix for the code



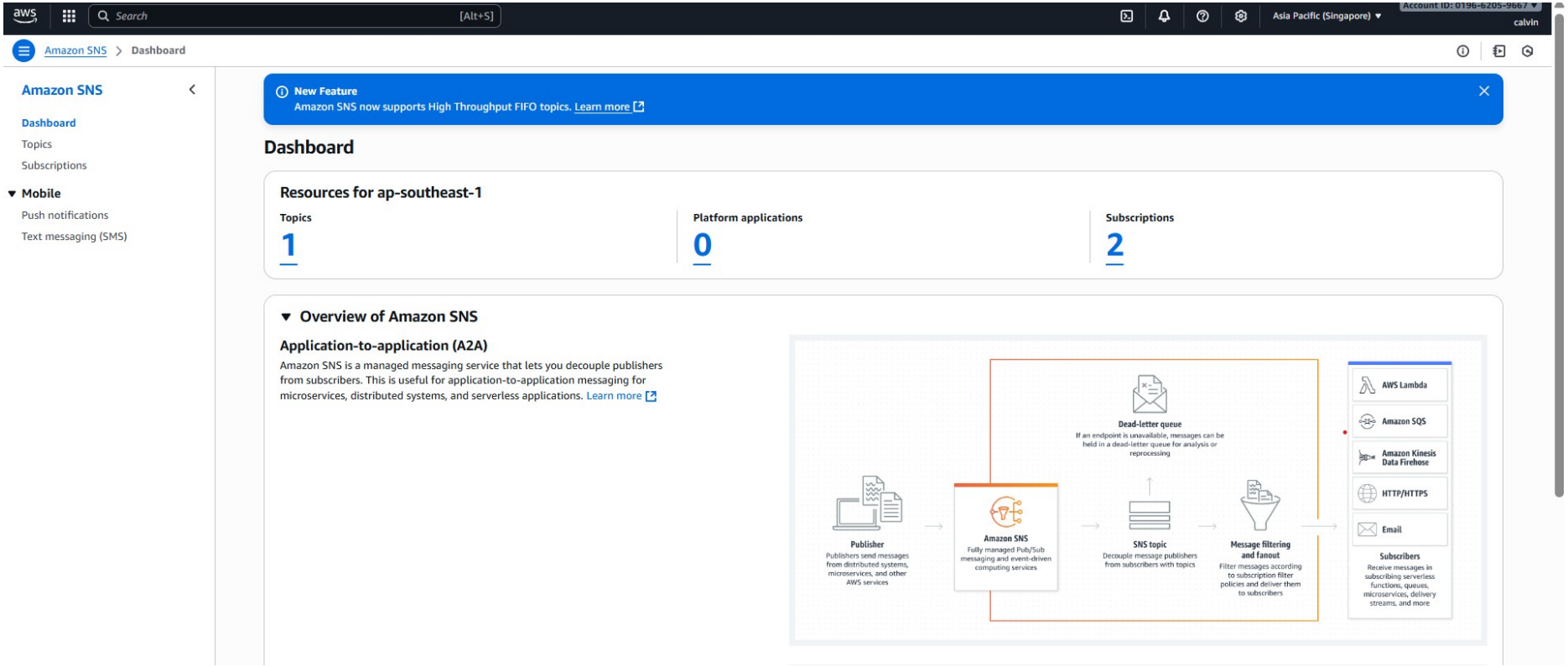
Refer to the Appendix for the code



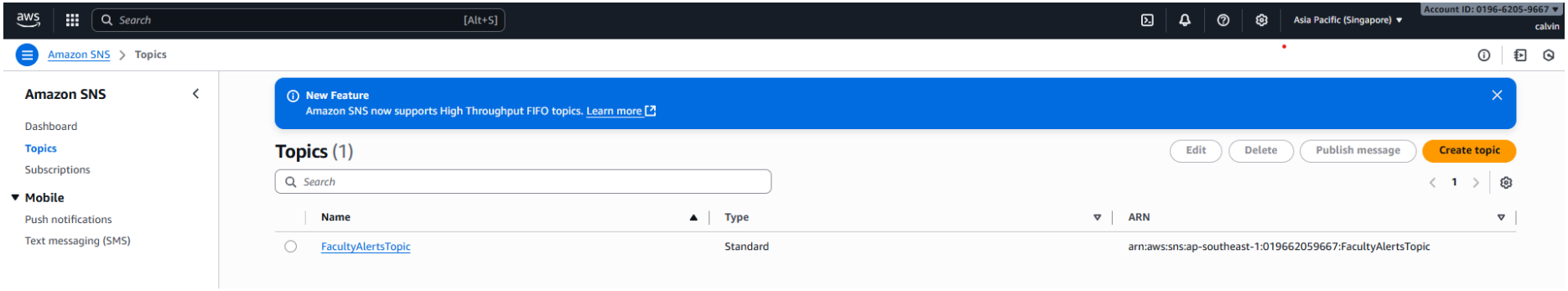
Refer to the Appendix for the code

Simple Notification Service

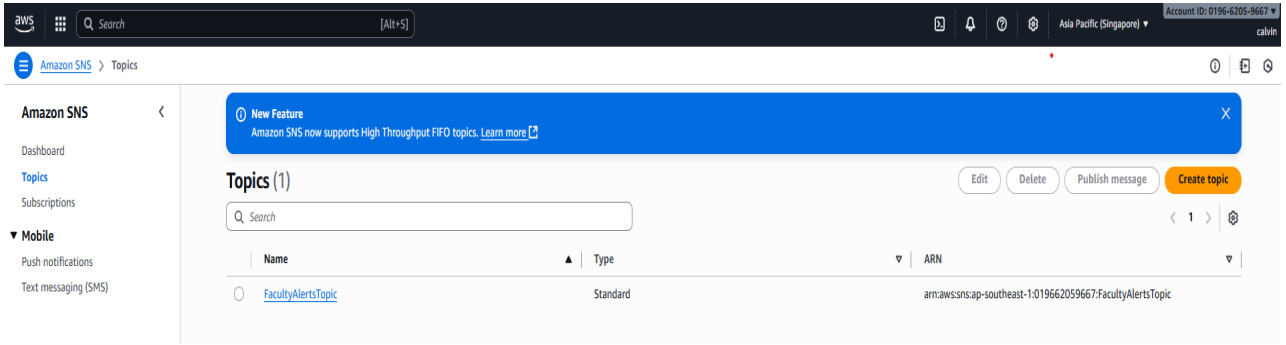
Navigate to the Amazon SNS Dashboard



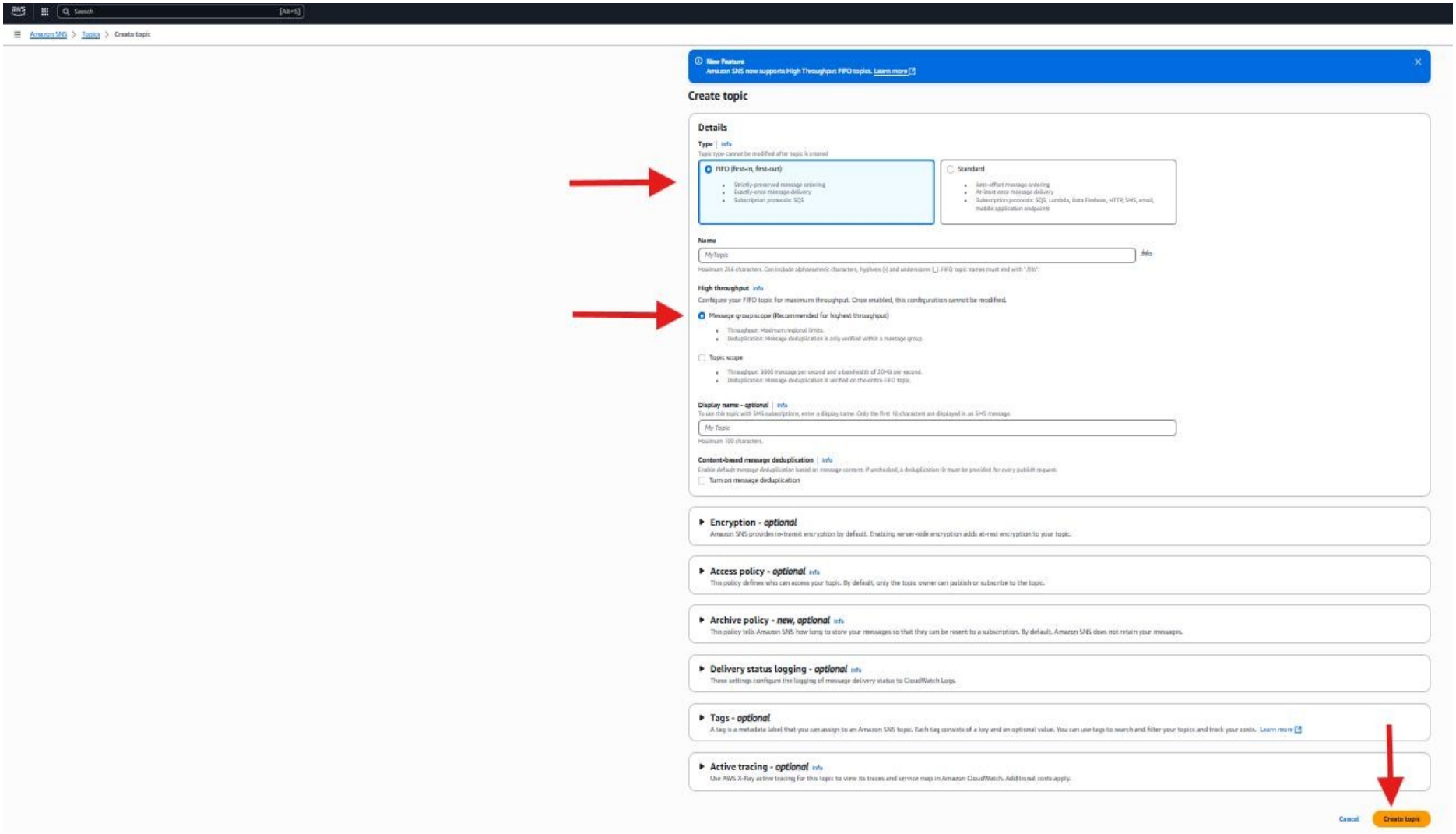
Select the Topic



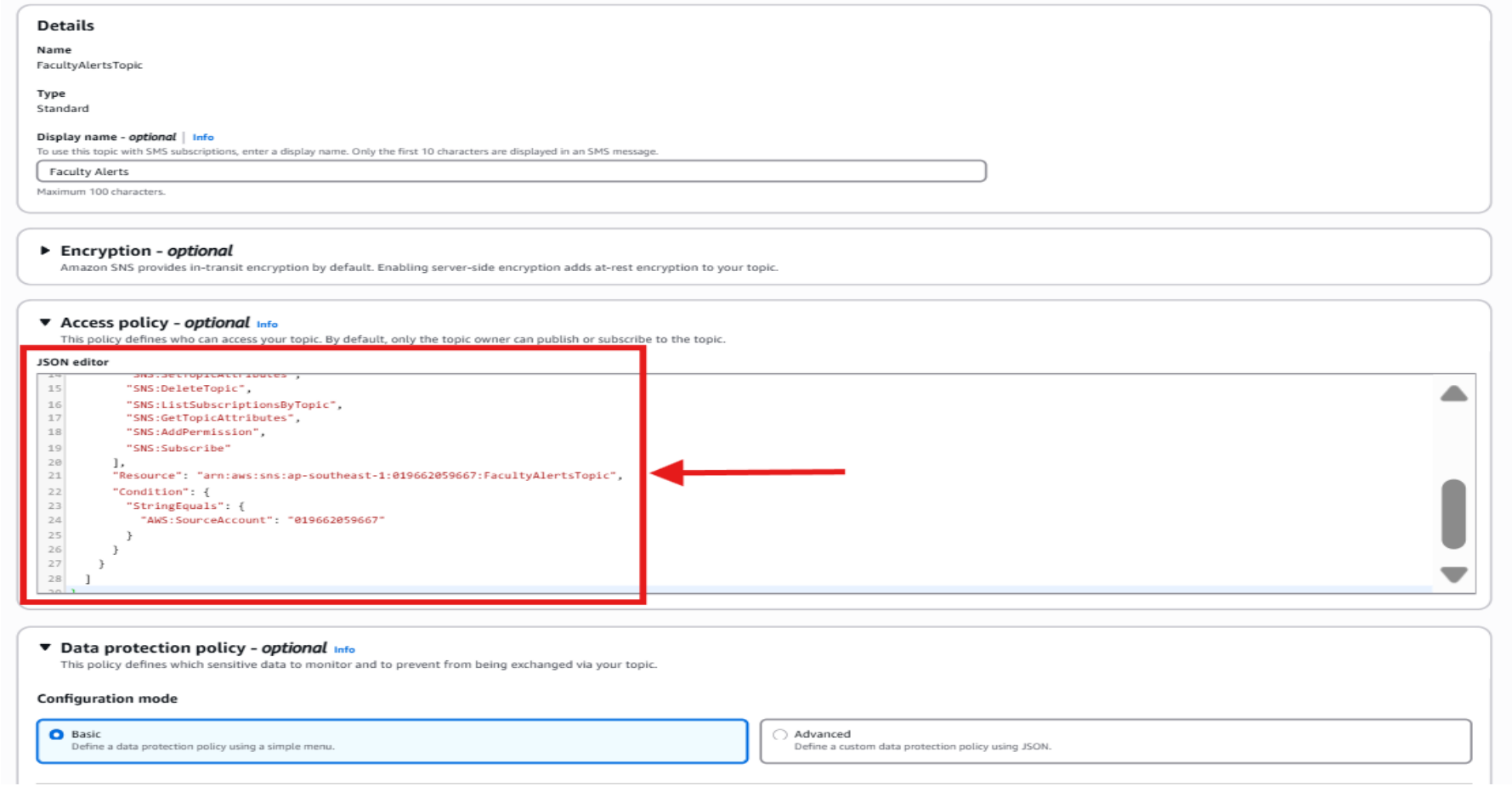
Create Topic



Insert Topic name and details

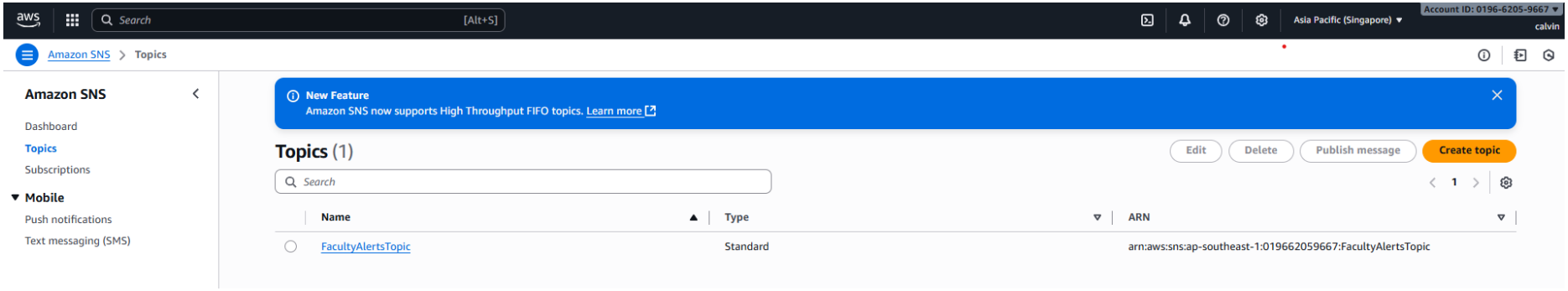


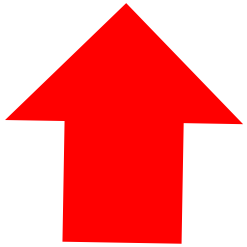
Update the Access Policy



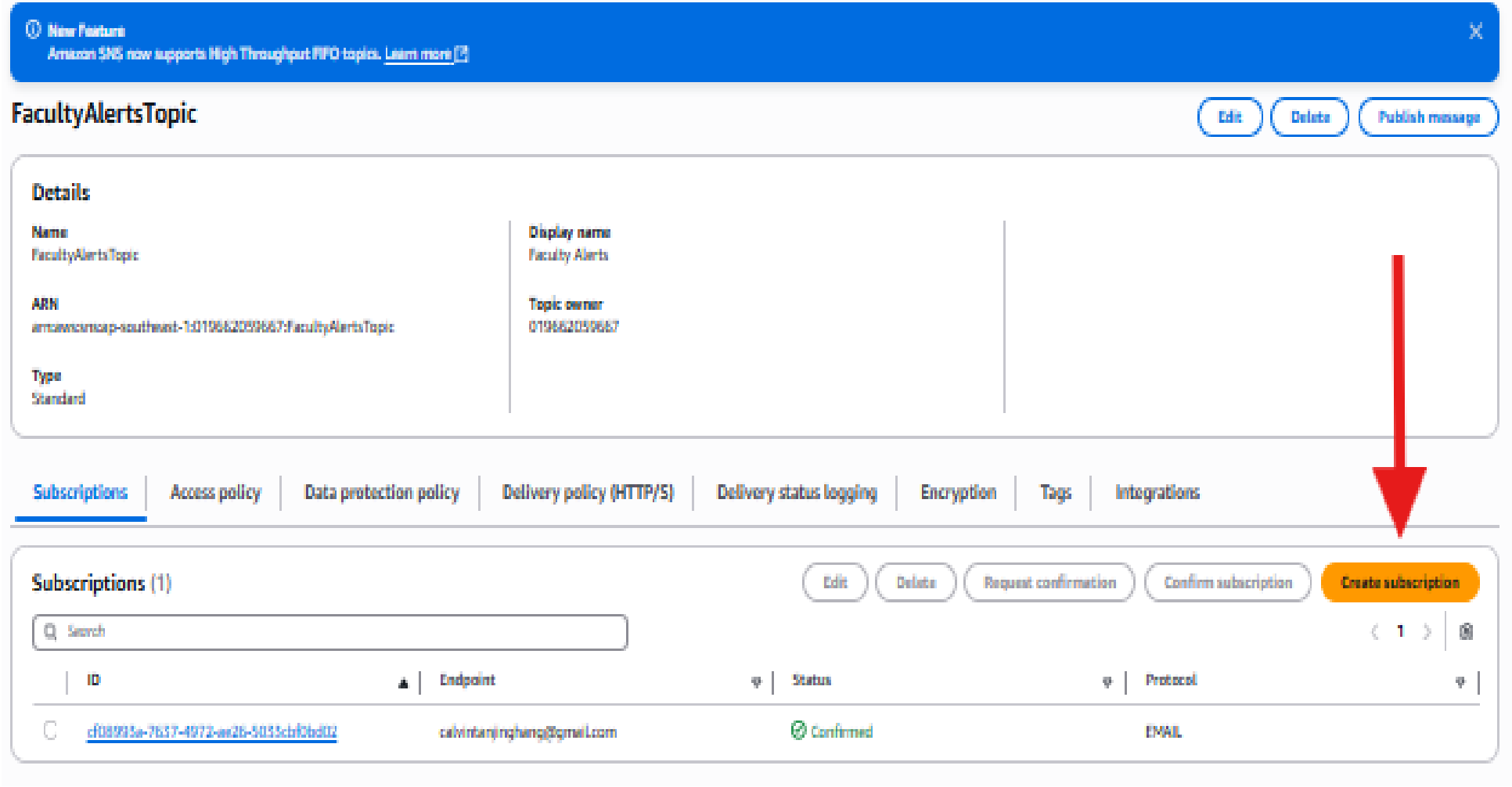
Refer to the Appendix for the code

Select the FacultyArletsTopic

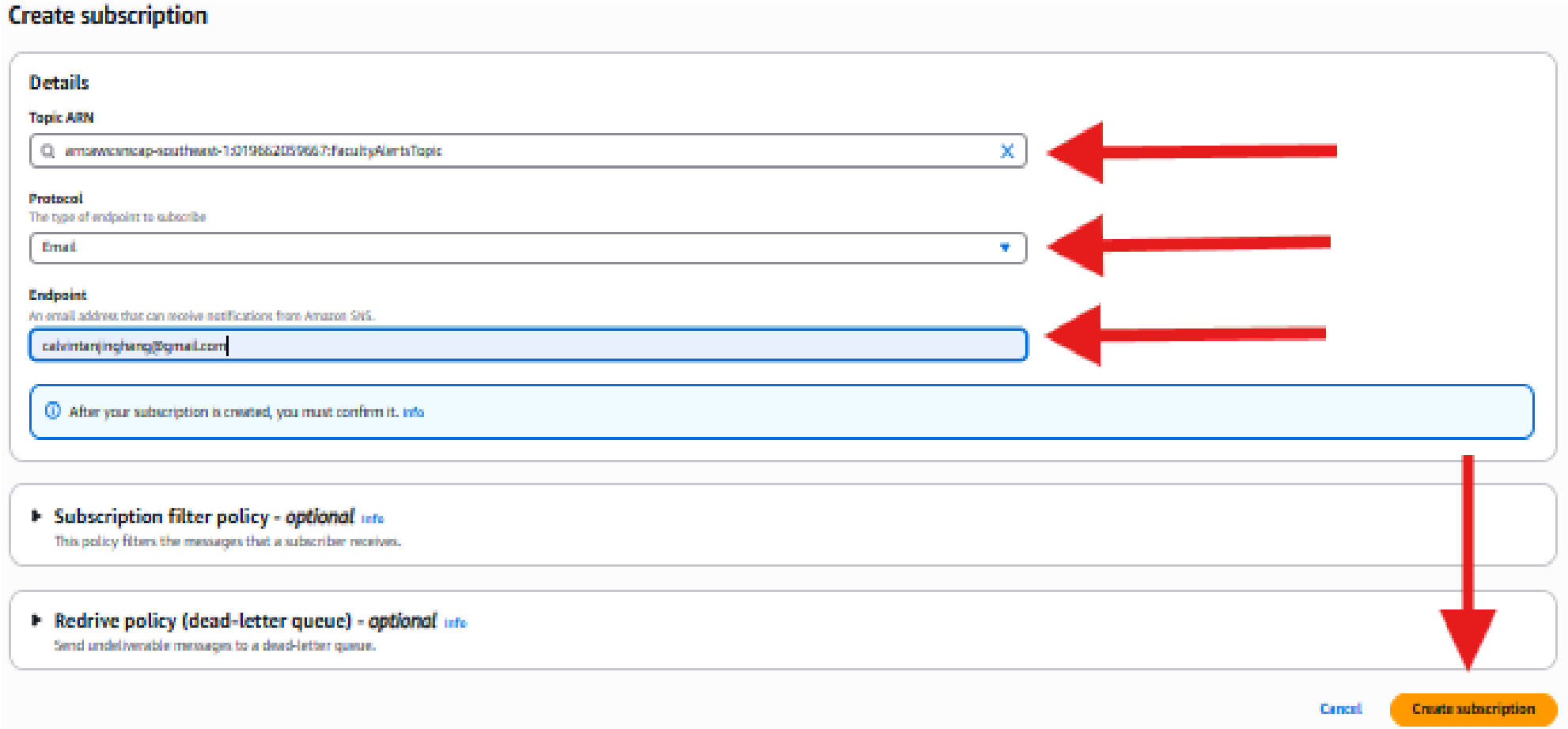




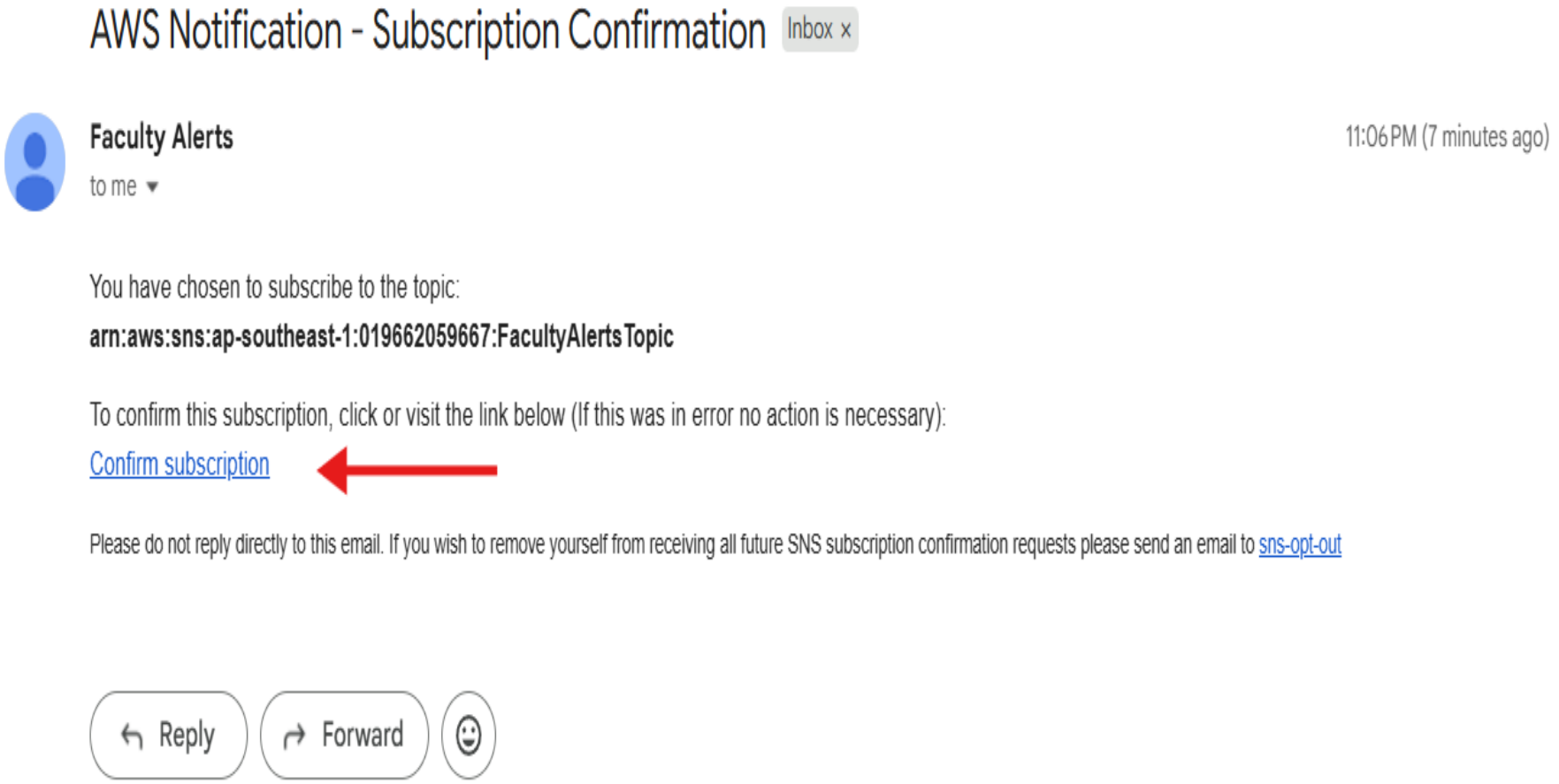
Click Create Subscription



Insert Topic ARN, Select ‘Email’ Protocol, Insert Subscription Email, Click the ‘Create Subscription’ Button



Open the Email to comfirm subscription



Subscription comfirmation massage

Simple Notification Service

Subscription confirmed!

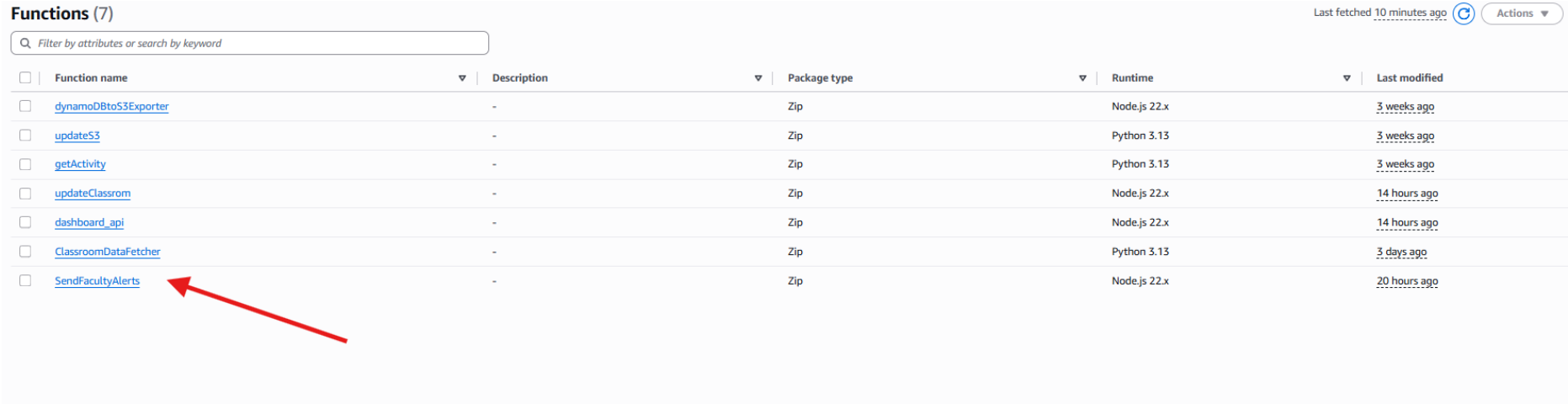
You have successfully subscribed.

Your subscription's id is:
`arn:aws:sns:ap-southeast-1:019662059667:FacultyAlertsTopic:cf08993a-7637-4972-ae26-5033cbf0bd02`

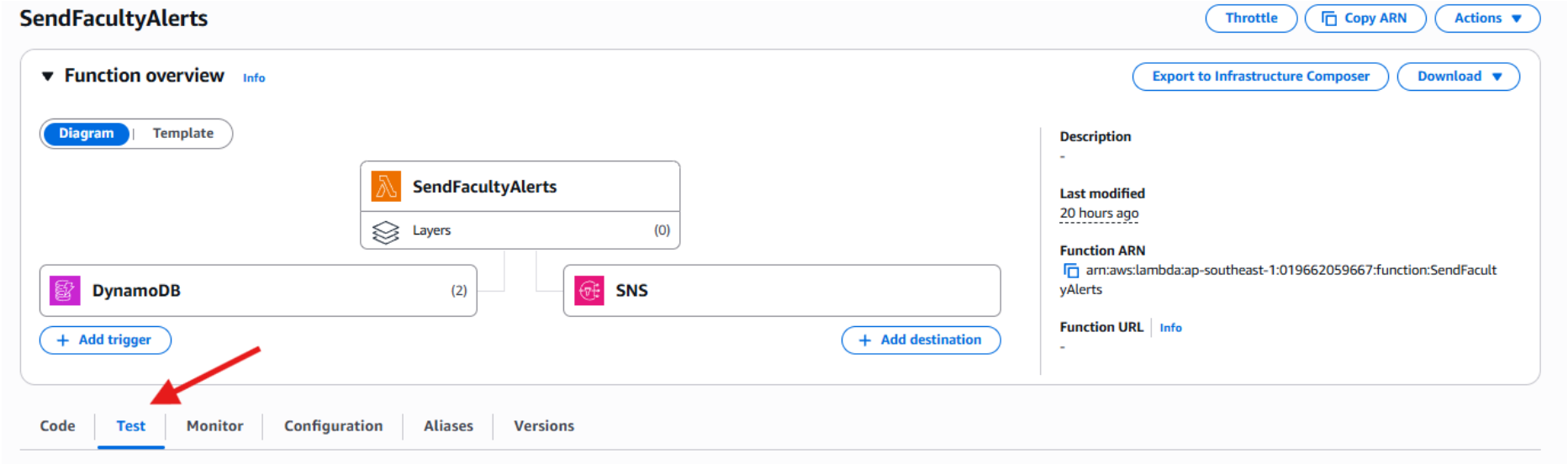
If it was not your intention to subscribe, click here to unsubscribe.

Test on the Simple Notification Service

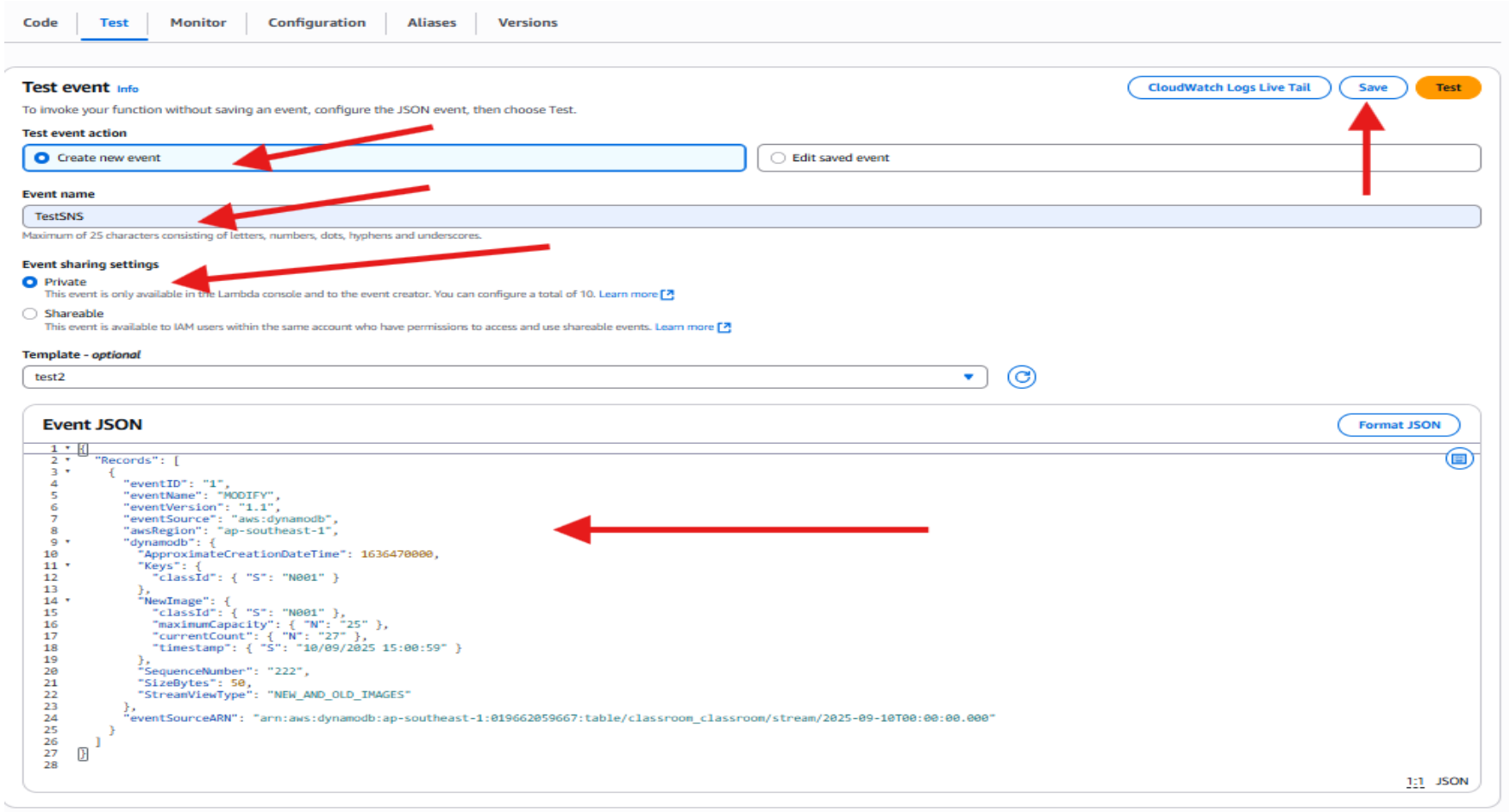
Back to lambda Function, select SendFaculty Alerts



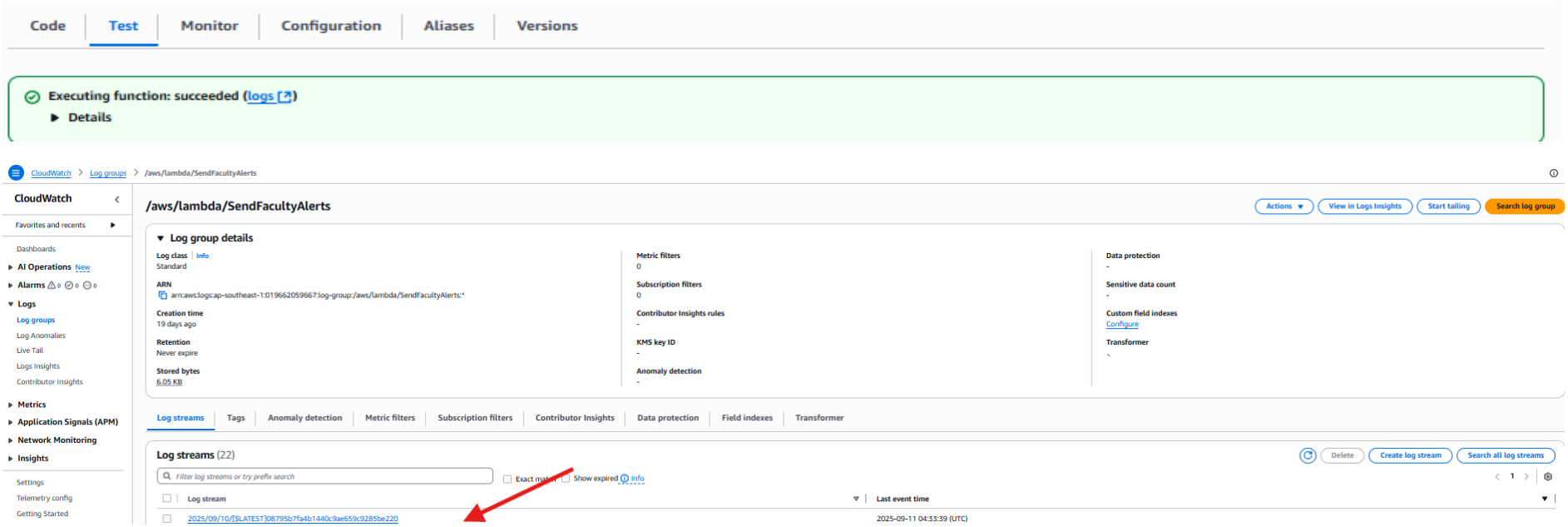
Select test


SendFacultyAlerts
Throttle
Copy ARN
Actions
Function overview
Info
Export to Infrastructure Composer
Download
Diagram
Template
SendFacultyAlerts
Layers
(0)
DynamoDB
(2)
SNS
Add trigger
Add destination
Description
-
Last modified
20 hours ago
Function ARN
arn:aws:lambda:ap-southeast-1:019662059667:function:SendFacultyAlerts
Function URL
Info
-
Code
Test
Monitor
Configuration
Aliases
Versions

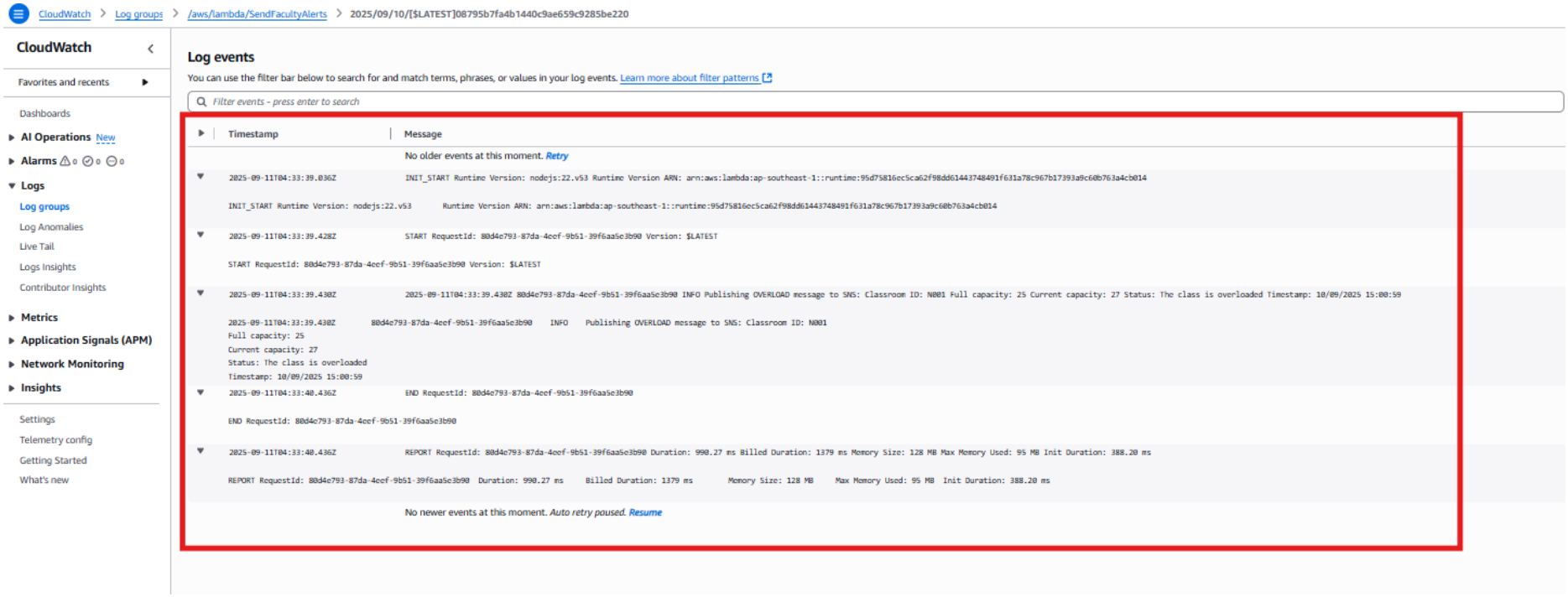
Create new Test, insert the necessary code, save and test it



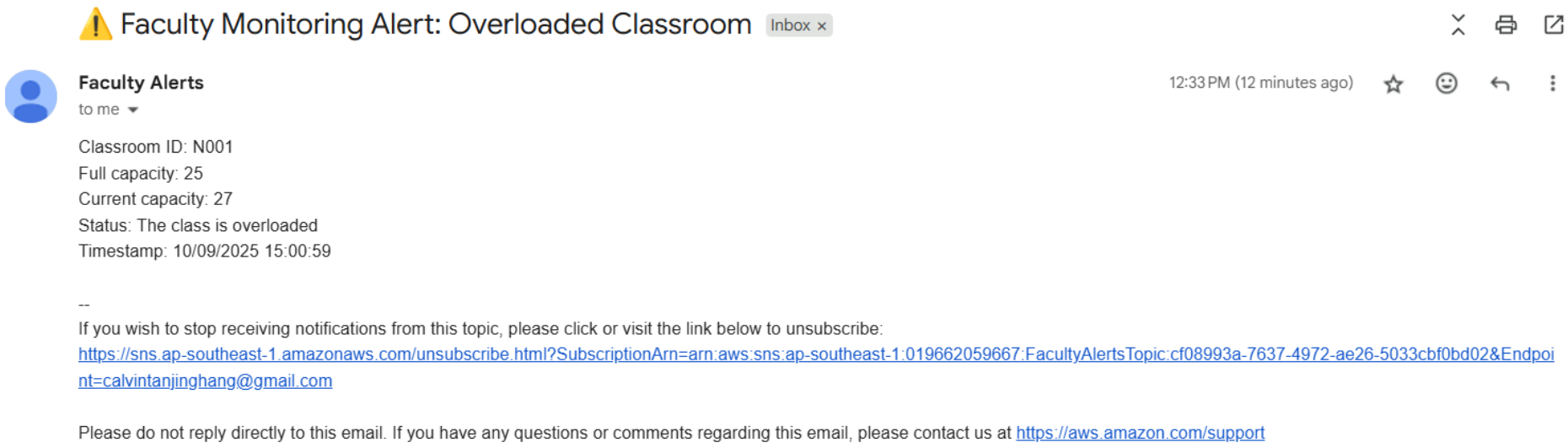
Check the Test response of SNS in Log events



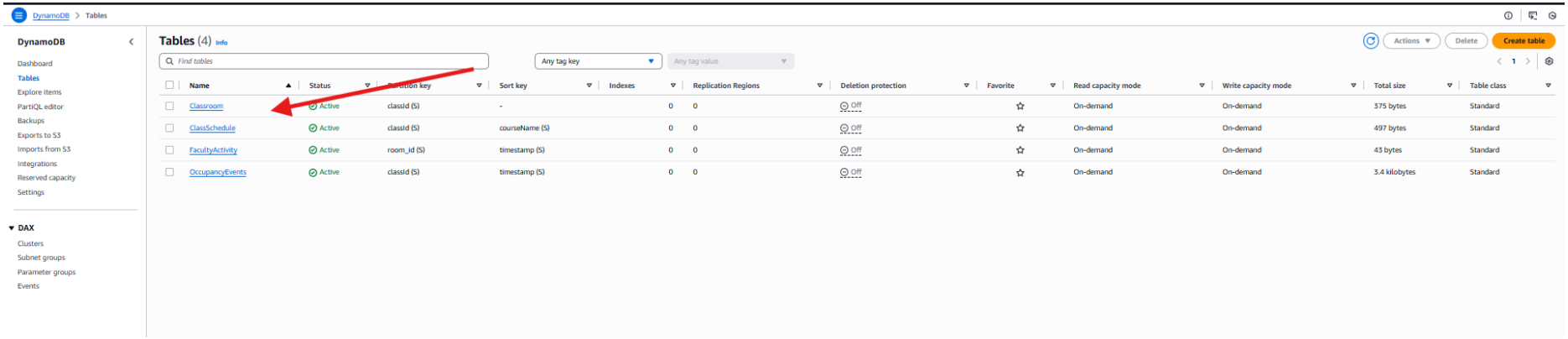
Check the Test response of SNS in Log events



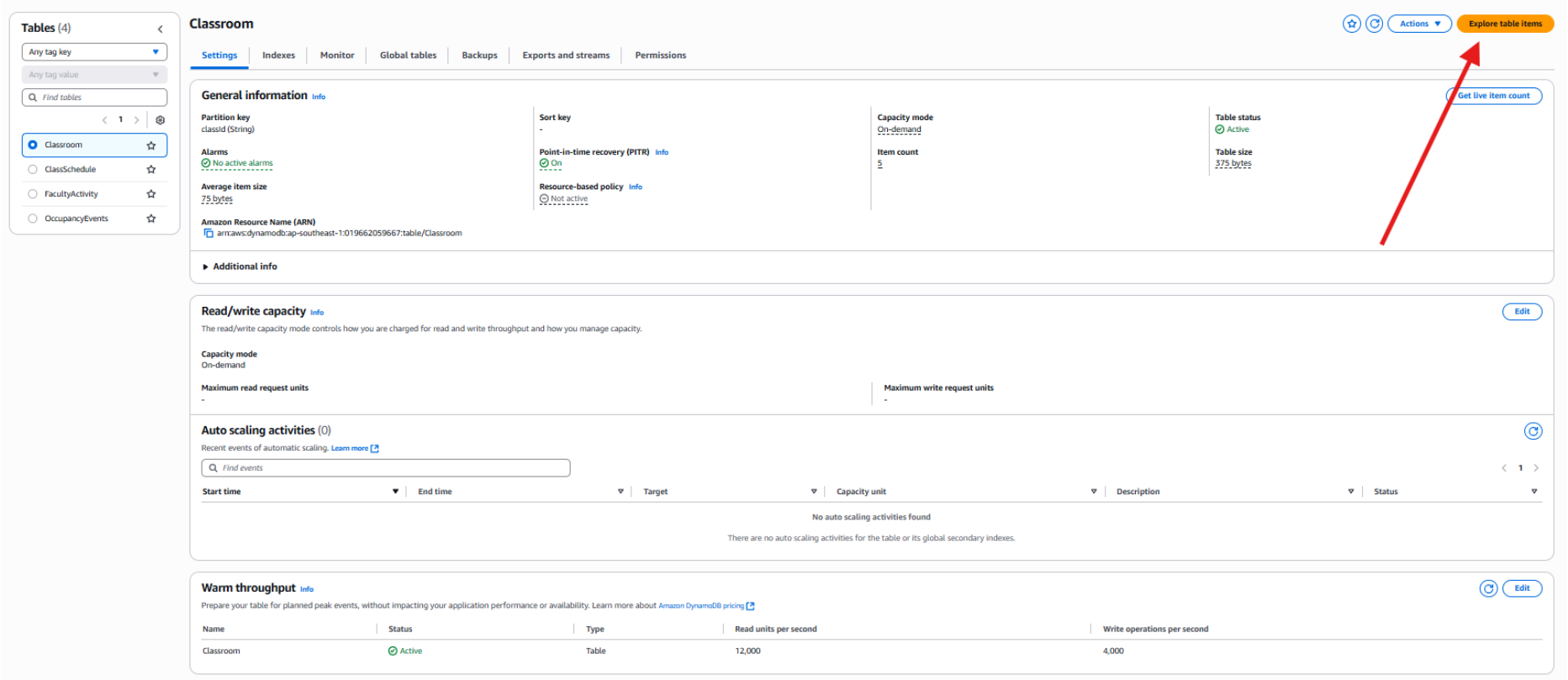
Check on email for the SNS Notification



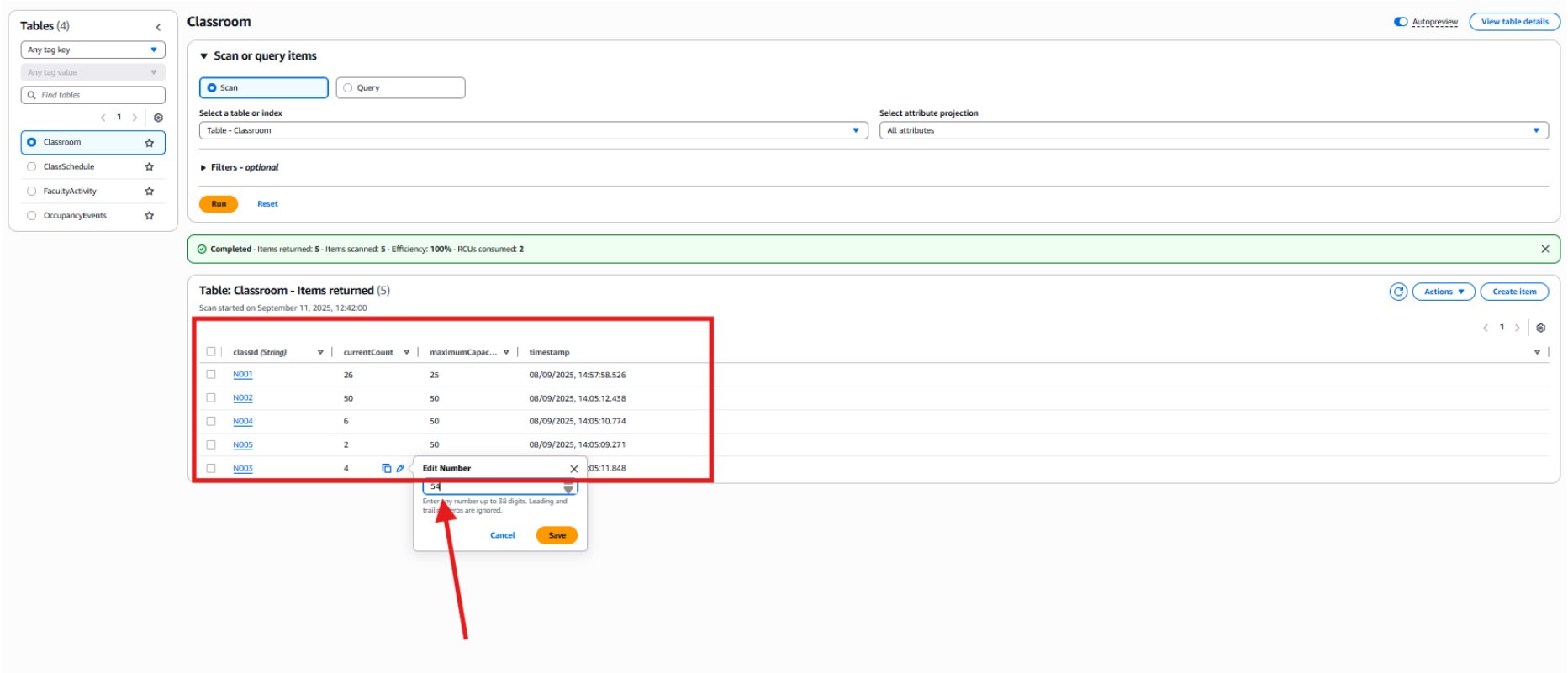
Navigate to DynamoDB, Table, then select ‘Classroom’ table



In the Classroom Table, Select ‘Explore Table Item’

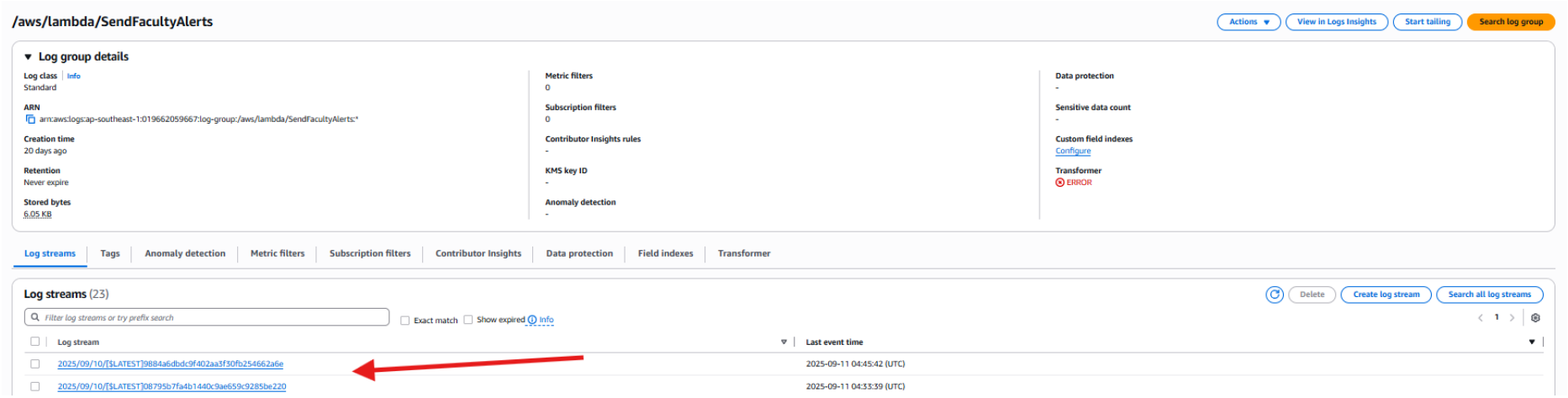


Modify the currentlyCount to exceed the maximumCapacity

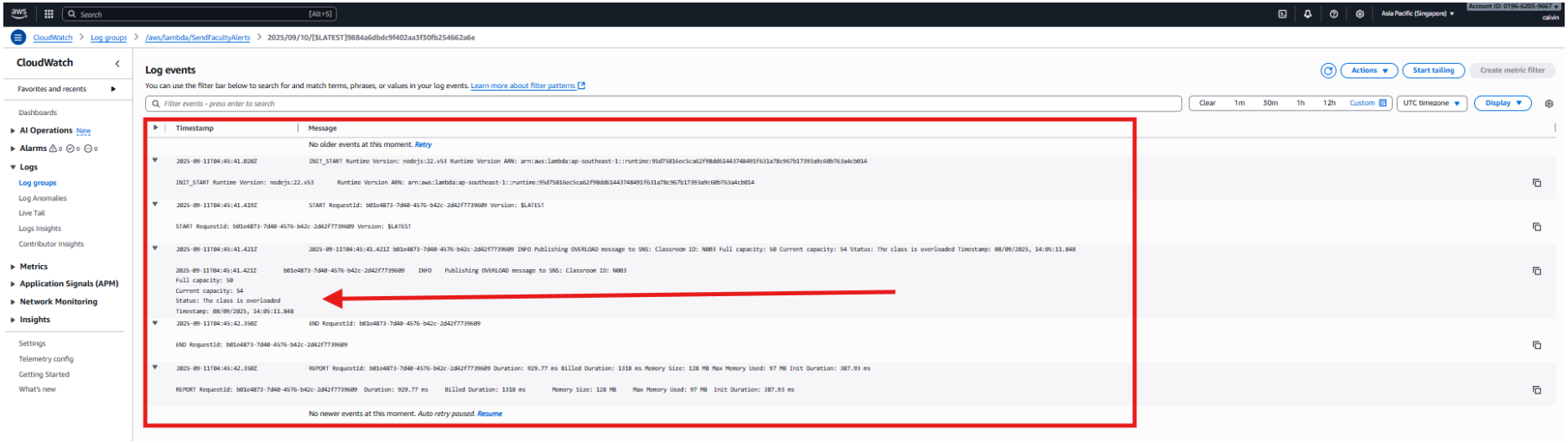


Check on email for the SNS Notification

Check the Test response of SNS in Log events



Check the Test response of SNS in Log events



Dashboards

Navigate to the API Gateway console and create an API

Networking & Content Delivery

API Gateway

Create and manage APIs at scale

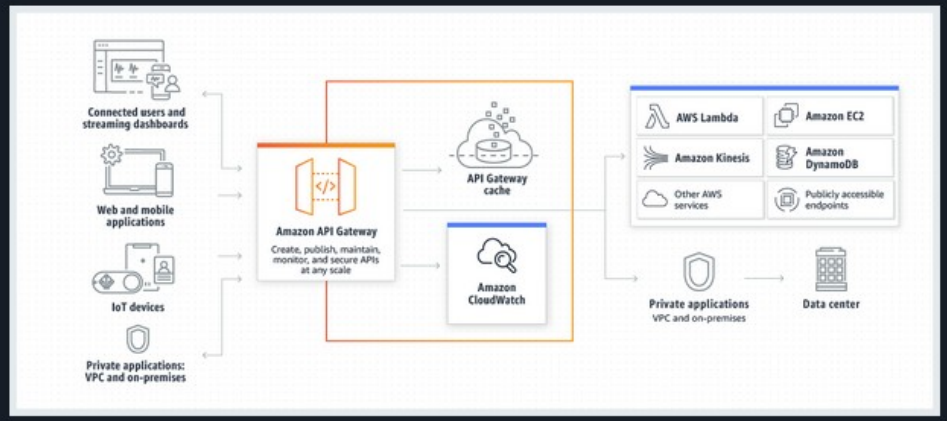
Amazon API Gateway is a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs.

Get started

Create a new API to begin exploring API Gateway. You can also import an external definition file into API Gateway.

Create an API

How it works



API Gateway enables you to connect and access data, business logic, and functionality from backend services such as workloads running on Amazon Elastic Compute Cloud (Amazon EC2), code running on AWS Lambda, any web application, or real-time communication applications.

Pric

With n API Gateway, you only pay when your APIs a se. There are no minimum fees or upfront comm s. For HTTP and REST APIs, you pay based on API calls received and amount of data transferred out. For WebSocket APIs, you pay based on number of messages and connection duration.

Learn more about pricing

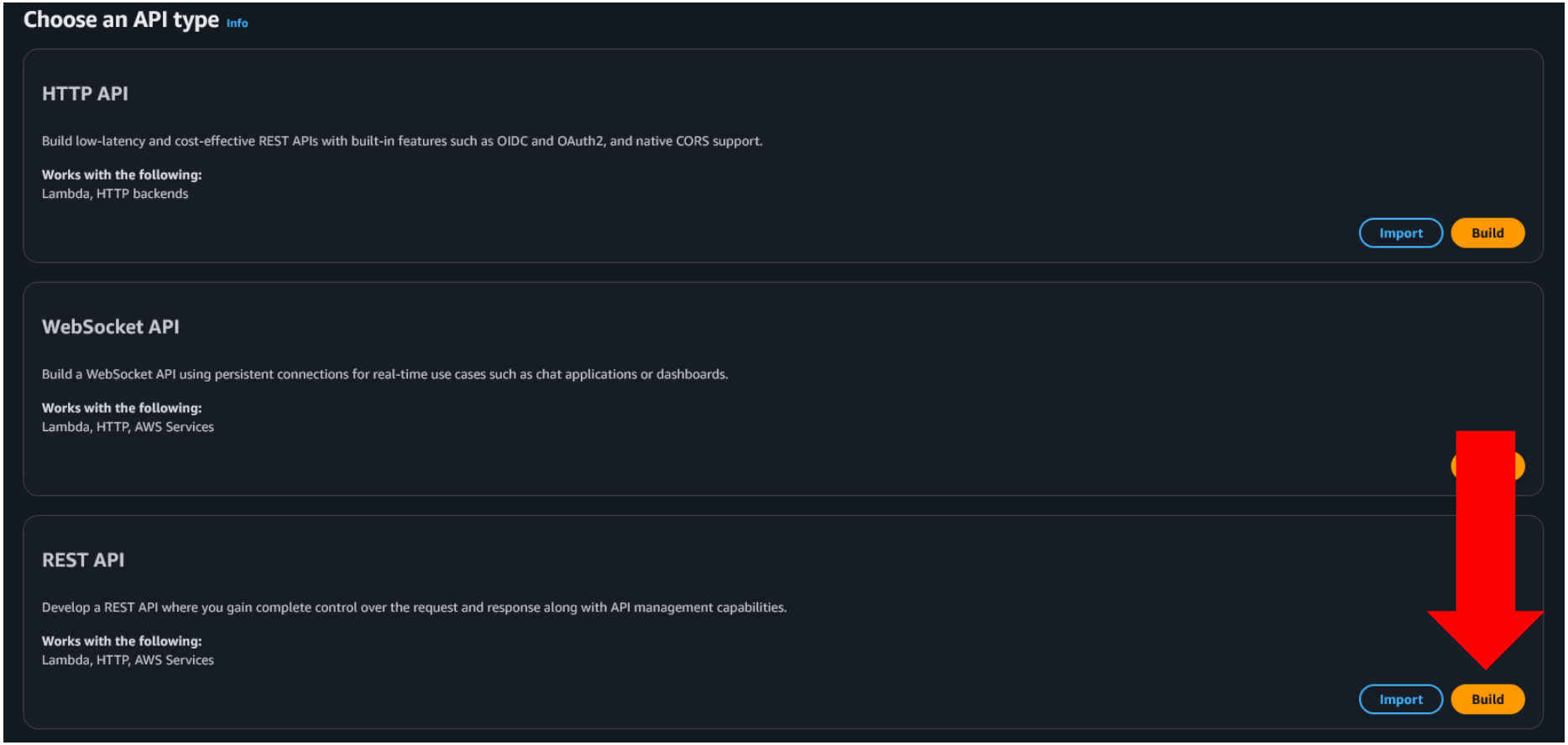
Resources

Getting Started Guide

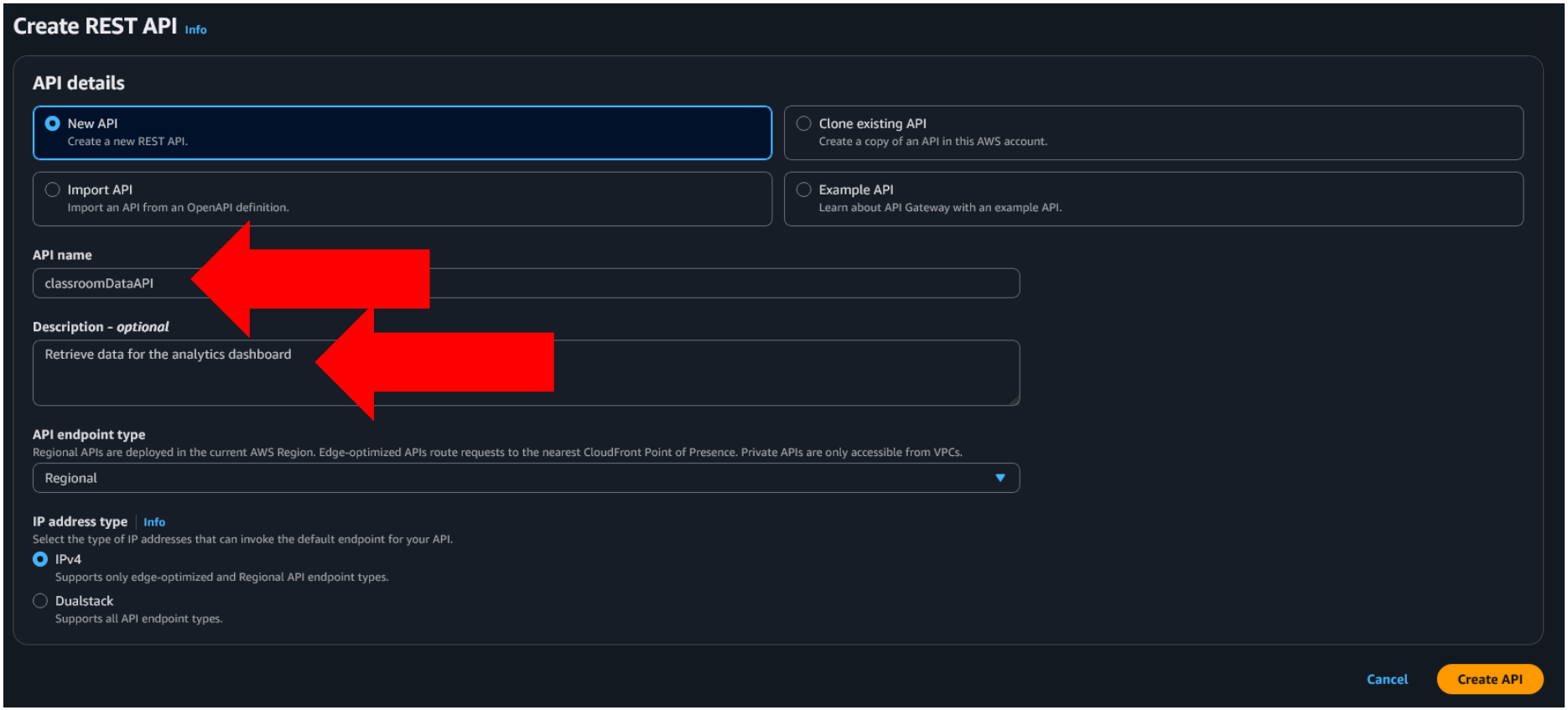
Developer Guide

API References

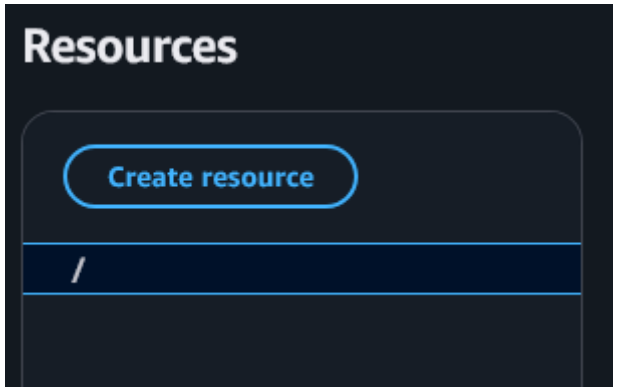
Select REST API



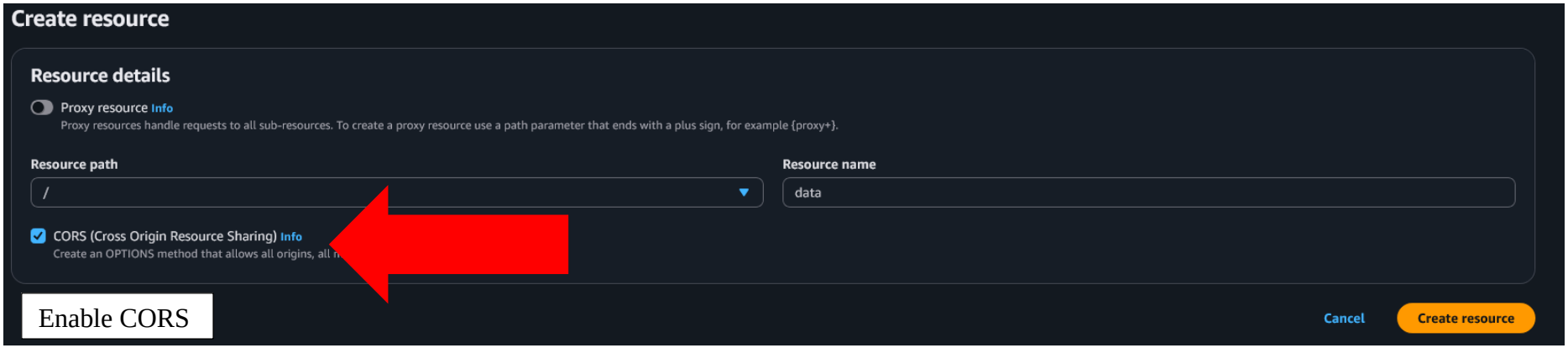
Give the API a name and a description



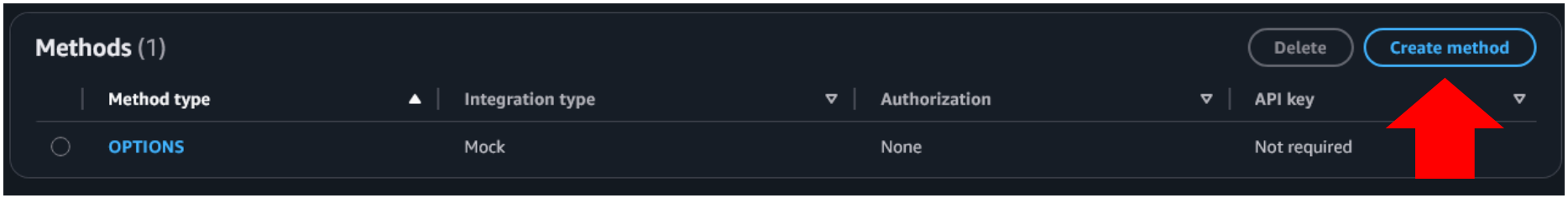
Navigate to the created API and press create resource



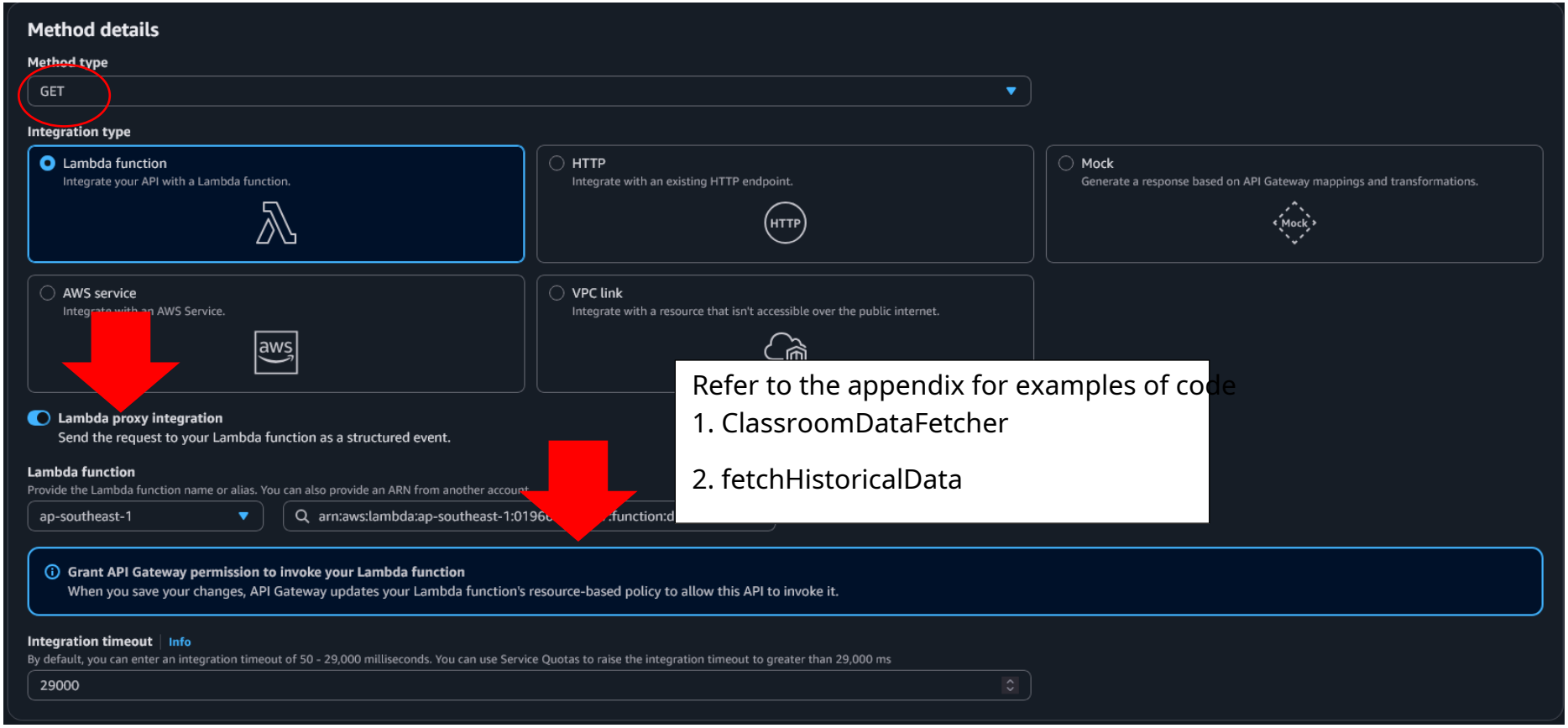
Name the resource and create resource



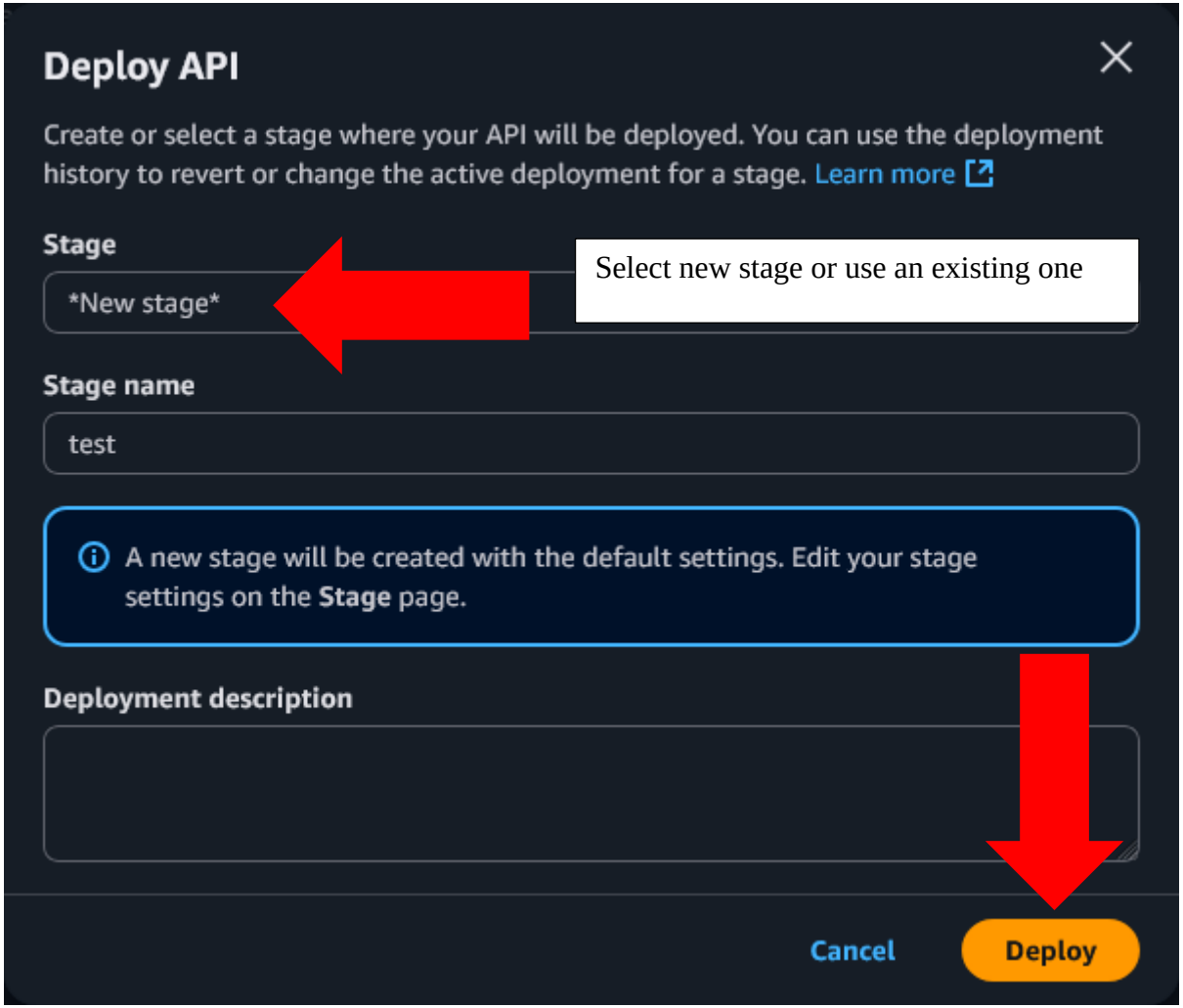
Create a method



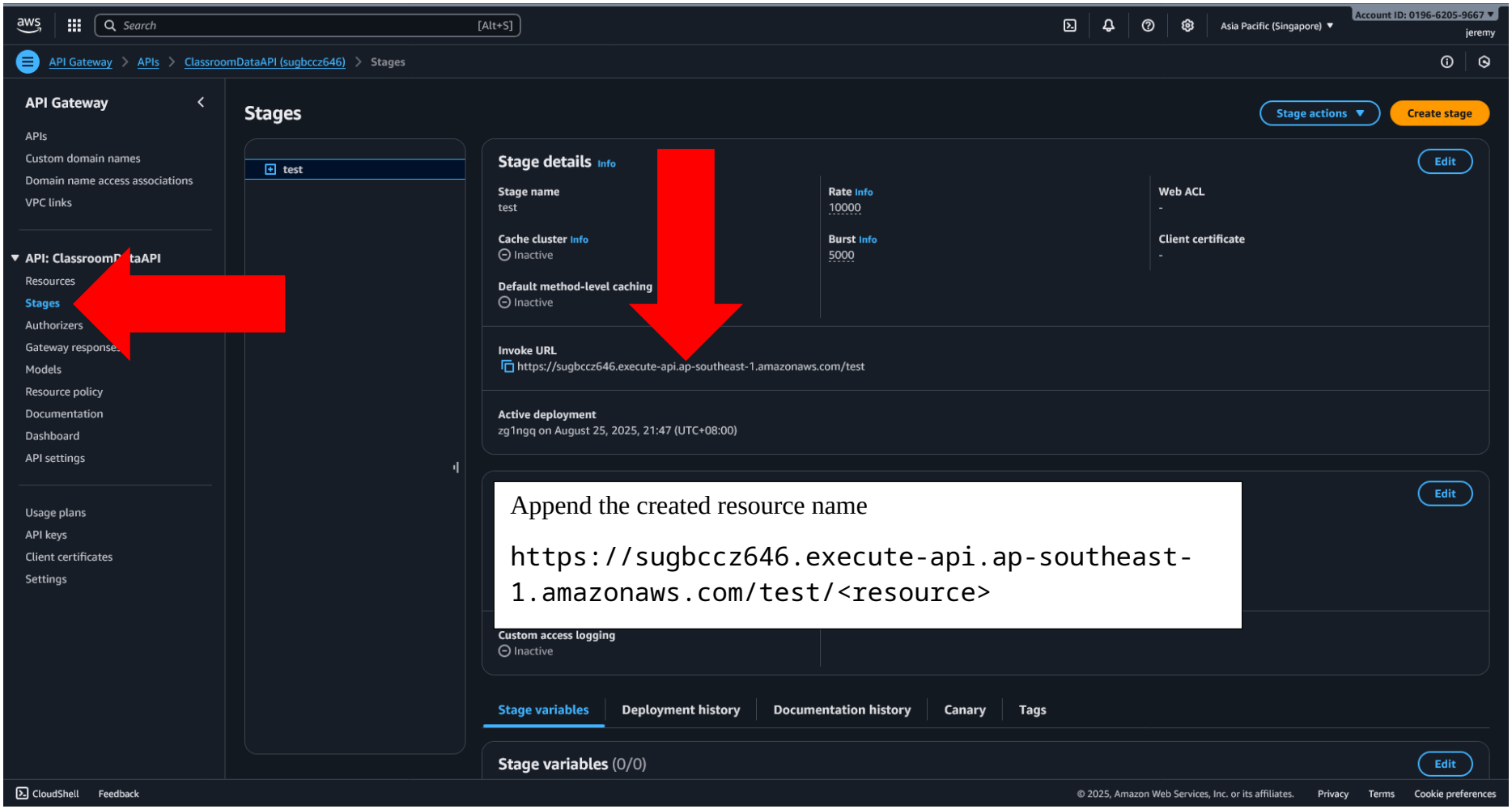
Select GET method type, choose the lambda function that queries the data with AWS Athena, and enable lambda proxy integration



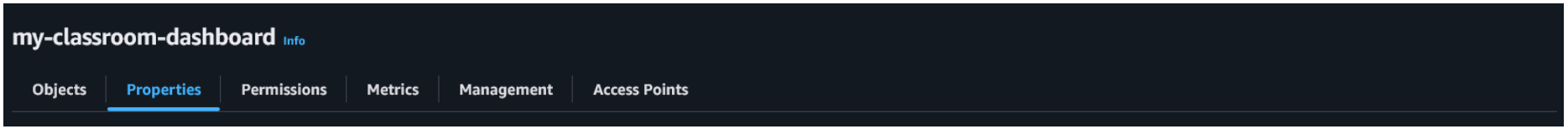
Deploy the API



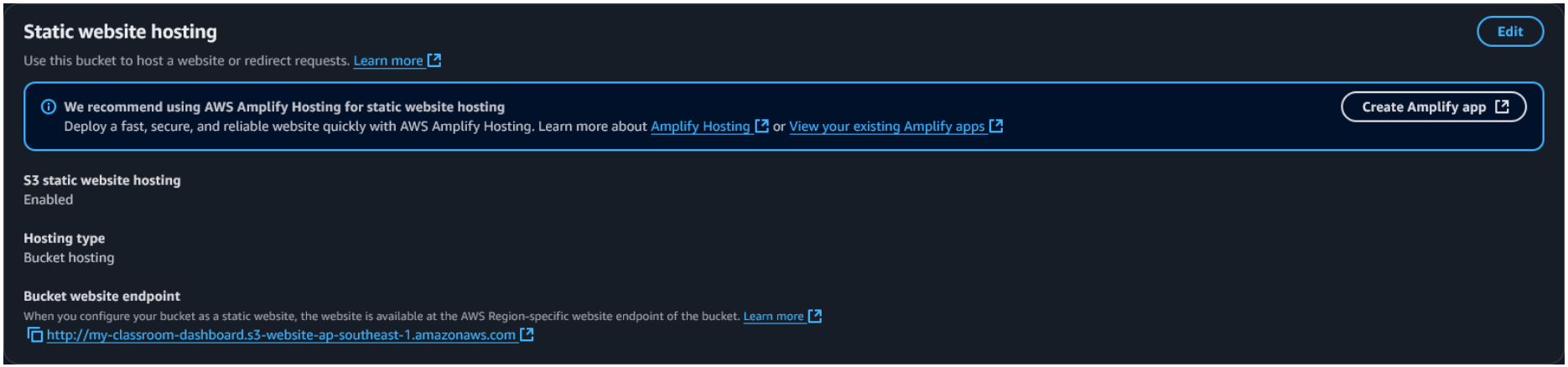
Copy the invoke URL from the Stages section



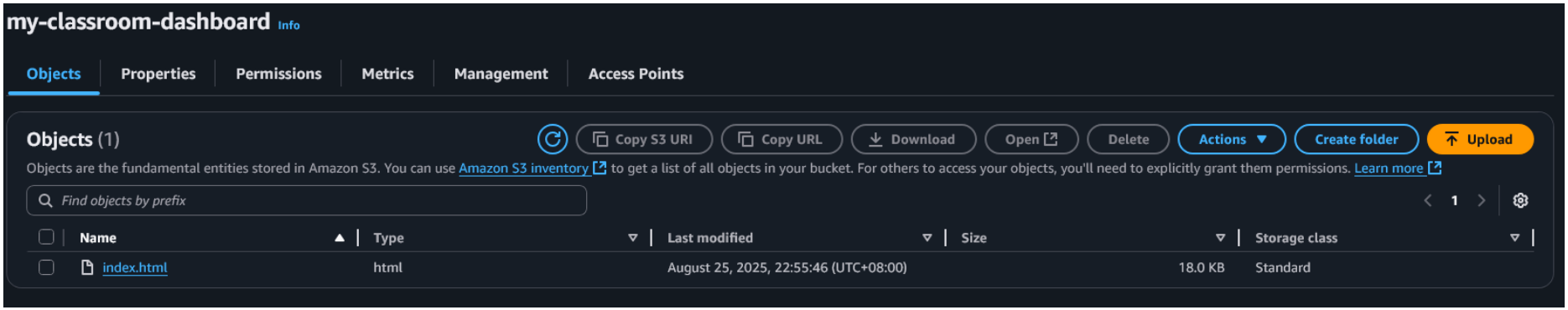
Create a bucket to host the static website



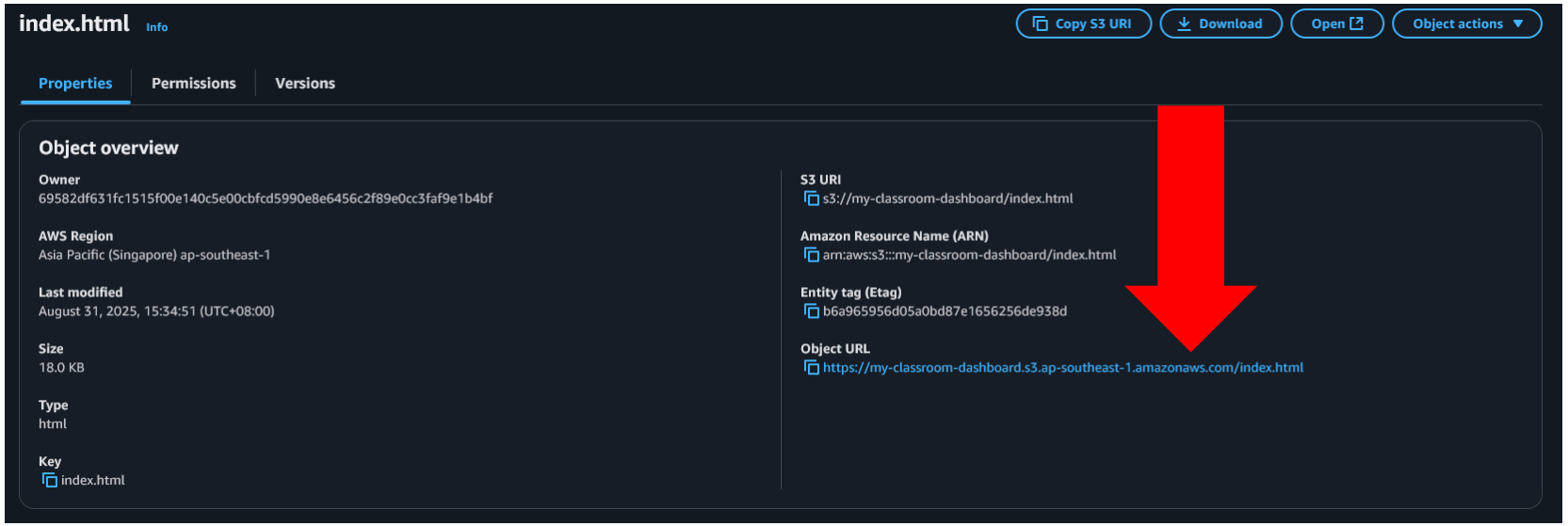
Enable s3 static website hosting



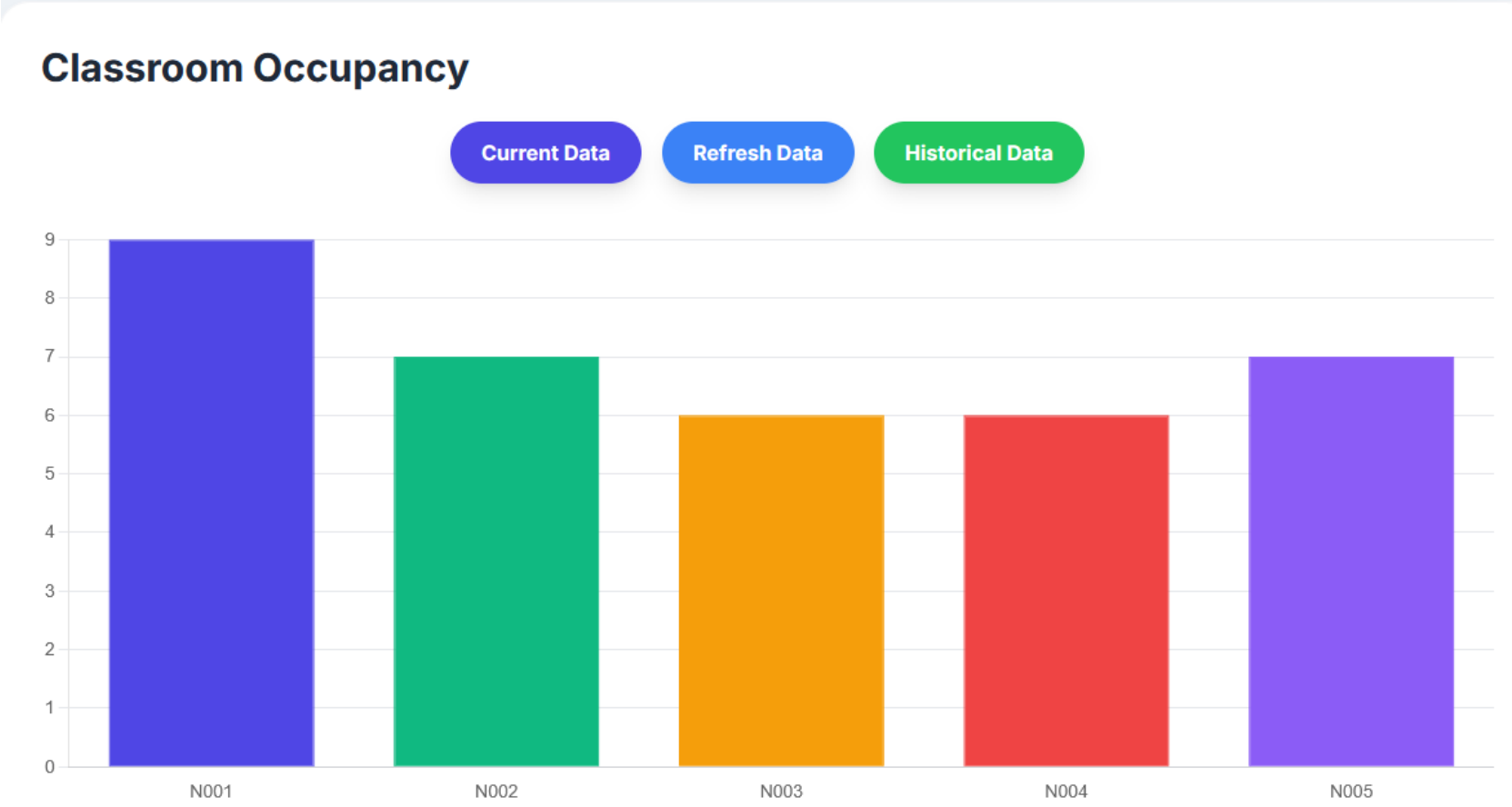
Upload the dashboard HTML code



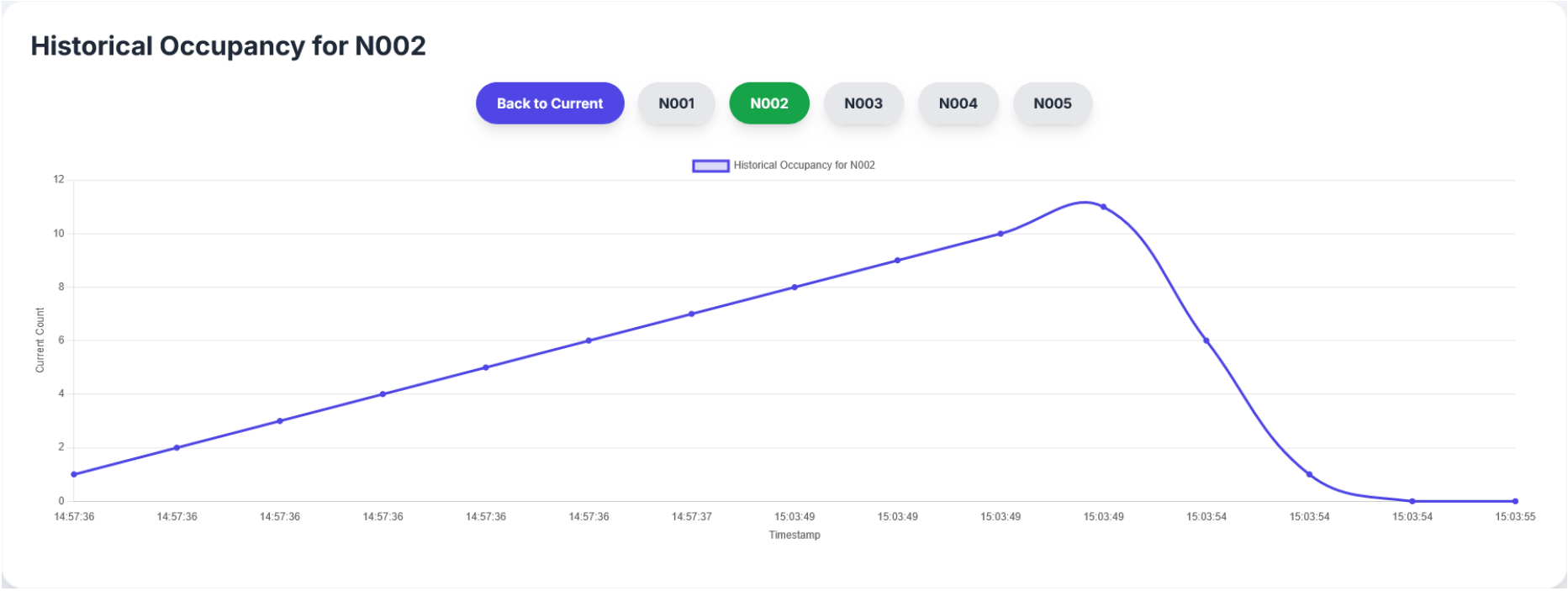
View the dashboard with the object link provided



Dashboard Live View Example 1



Dashboard Live View Example 2



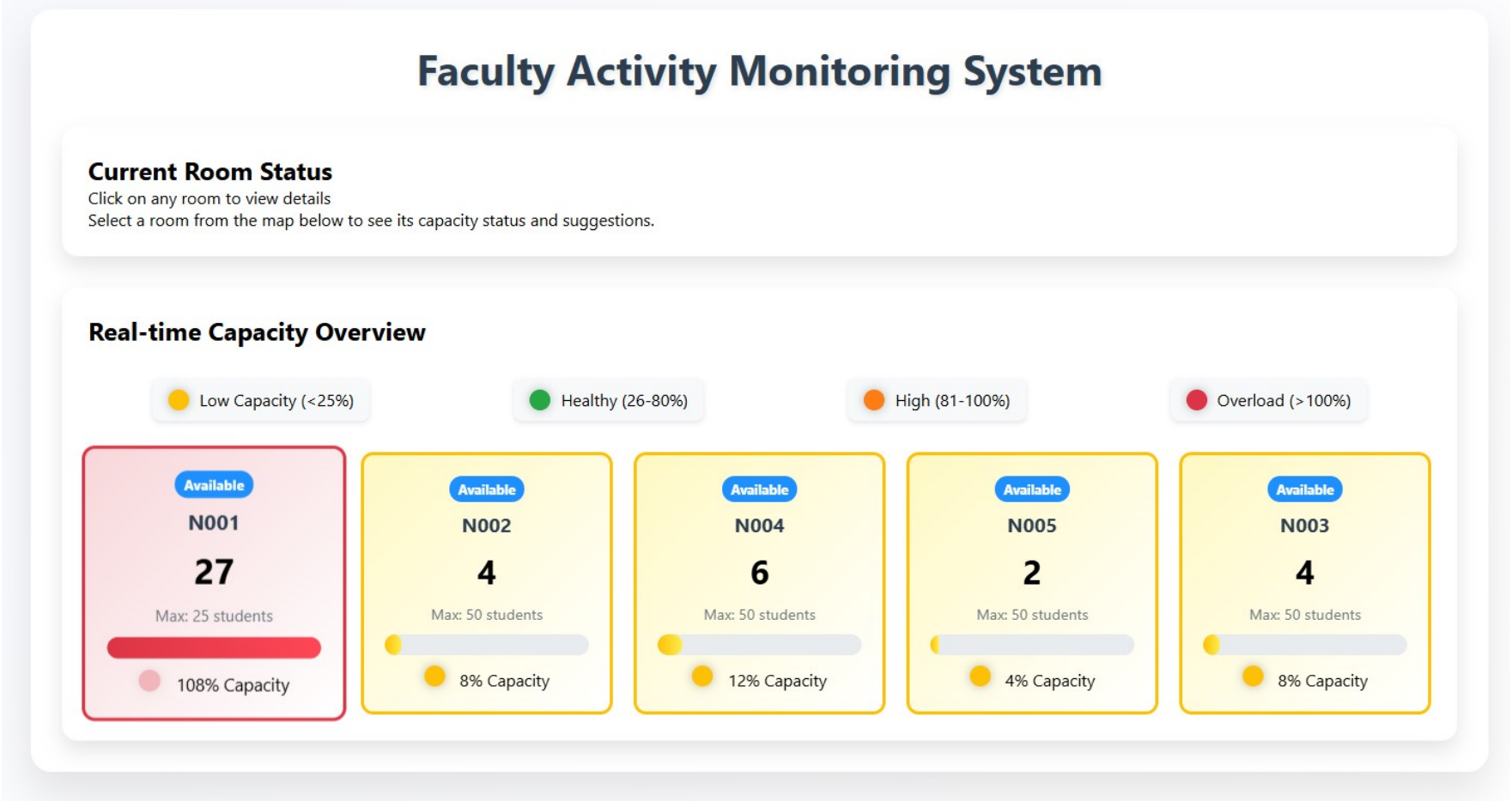
Dashboard Live View Example 3

Classroom Status Overview

CLASS ID	COUNT	MAX CAPACITY	STATUS	ALERT LIGHT	SUGGESTION
N001	50	25	OVERLOAD (200%): Capacity exceeded!	🔴	ALERT MODERATOR! Suggest another classroom with suitable capacity.
N002	0	50	Unoccupied (0%): Classroom is empty!	🟢	Switching off the lights and air-conditioner
N003	0	50	Unoccupied (0%): Classroom is empty!	🟢	Switching off the lights and air-conditioner
N004	0	50	Unoccupied (0%): Classroom is empty!	🟢	Switching off the lights and air-conditioner
N005	0	50	Unoccupied (0%): Classroom is empty!	🟢	Switching off the lights and air-conditioner

Activity Monotoring System

Repeat above process with different lambda name and file for S3 to create another static website



Appendix

Reset Flow Context

```
// Get the class ID from the message
let classId = msg.classId || "ClassA";

// Initialize flow.classData if it doesn't exist
let classData = flow.get('classData') || {};
if (!classData[classId]) {
    classData[classId] = { occupancy: 0 };
}

classData[classId].occupancy = 0;
classData[classId].actualCount = undefined;

flow.set('classData', classData); // Save the updated classData object
msg.payload = 0;
msg.topic = 'reset_trigger';
msg.classId = classId;
return msg;
```

Occupancy Counter

```
// Initialize classroomOccupancy in flow context if it doesn't exist
let classId = msg.classId || 'ClassA';

let classData = flow.get('classData') || {};
if (!classData[classId]) {
    classData[classId] = {
        occupancy: 0,
        actualCount: 0,
        capacity: 50
    };
}

let occupancy = classData[classId].occupancy;
var action

// Check if the incoming message noOfStudent is a number (1 for entry, -1
for exit)
if (typeof msg.noOfStudent === 'number') {
    occupancy += msg.noOfStudent;
}

// Ensure occupancy never goes below zero (you can't have negative students)
if (occupancy < 0) {
    occupancy = 0;

}

if (msg.noOfStudent > 0 ) {
    action = "Enter"
}
else{
    action = "Exit"
}
```

```
    "Name": msg.payload.Name,
    "action": action,
    "classID": classId,
    "occupancy": msg.curTotalNumber
};

// Assign the newly created object to the message payload
msg.payload = newPayload;

return msg;
```

2 Msg modifier

```
var timestamp = new Date().toLocaleString("en-MY", {
    timeZone: "Asia/Kuala_Lumpur",
    year: 'numeric',
    month: '2-digit',
    day: '2-digit',
    hour: '2-digit',
    minute: '2-digit',
    second: '2-digit',
    hour12: false
}) + '.' + new Date().getMilliseconds().toString().padStart(3, '0');

var classid = msg.classId
var occupancy = msg.payload.occupancy


// Create a new JavaScript object with the desired keys
var newPayload = {
    "Name": msg.payload.Name,
    "type": "classroom_update",
    "classID": classid,
    "occupancy": occupancy,
    "timestamp": timestamp,

};

msg.payload = newPayload;

return msg;
```

```
var timestamp = new Date().toLocaleString("en-MY", {
    timeZone: "Asia/Kuala_Lumpur",
    year: 'numeric',
```

```
// Create a new JavaScript object with the desired keys
var newPayload = {
    "Name": msg.payload.Name,
    "type": "occupancy_event",
    "action": msg.payload.action,
    "classID": classid,
    "timestamp": timestamp,

};

msg.payload = newPayload;

return msg;
```

updateClassroom

```
import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
import {
    DynamoDBDocumentClient,
    UpdateCommand,
    PutCommand
} from "@aws-sdk/lib-dynamodb";

const client = new DynamoDBClient({});
const db = DynamoDBDocumentClient.from(client);

export const handler = async (event) => {
    const type = event.type;
    const classId = event.classID || event.classId;

    if (!type || !classId) {
        return {
            statusCode: 400,
            body: JSON.stringify('Missing "type" or "classId/classID" in the
event.')
        };
    }

    try {
        switch (type) {
            case 'classroom_update':
                const {
                    currentCount,
                    timestamp: providedTimestamp
                } = event.data;

                if (currentCount === undefined) {
                    console.error('Validation Error: Missing "currentCount"
in the event data.');
                    return {
```

```
                    Key: {
                        'classId': classId
                    },
                    UpdateExpression: 'SET  currentCount = :c, #ts = :t',
                    ExpressionAttributeNames:{
                        '#ts': 'timestamp'
                    },
                    ExpressionAttributeValues: {
                        ':c': currentCount,
                        ':t': timestamp
                    },
                    ReturnValues: 'UPDATED_NEW'
                };

                await db.send(new UpdateCommand(classroomUpdateParams));
                return {
                    statusCode: 200,
                    body: JSON.stringify(`Classroom ${classId} updated
successfully.`)
                };

            case 'occupancy_event':
                const { action } = event.data;
                const eventTimestamp = event.data.timestamp;

                if (!action || !eventTimestamp) {
                    console.error('Validation Error: Missing "action" or
"timestamp" in the event.');
                    return {
                        statusCode: 400,
                        body: JSON.stringify('Missing "action" or
"timestamp" in the event.')
                    };
                }

                const occupancyEventParams = {
                    TableName: 'OccupancyEvents'
```

```
                    body: JSON.stringify(`Occupancy event for classroom
${classId} recorded successfully.`)
                };

            case 'schedule_update':
                const {
                    courseName,
                    scheduleType,
                    dayOfWeek,
                    startTime,
                    endTime
                } = event.data;

                if (!courseName || !scheduleType || !dayOfWeek || !startTime
|| !endTime) {
                    console.error('Validation Error: Missing required fields
in the event.');
                    return {
                        statusCode: 400,
                        body: JSON.stringify('Missing required fields in the
event.')
                    };
                }

                const scheduleUpdateParams = {
                    TableName: 'ClassSchedule',
                    Key: {
                        'classId': classId,
                        'courseName': courseName,
                    },
                    UpdateExpression: 'SET #type = :t, dayOfWeek = :d,
startTime = :st, endTime = :et',
                    ExpressionAttributeNames: {
                        '#type': 'type'
                    },
                    ExpressionAttributeValues: {
                        ':t': scheduleType,
```

```
                    body: JSON.stringify(`Schedule for classroom ${classId}
updated successfully.`)
                };

            default:
                return {
                    statusCode: 400,
                    body: JSON.stringify(`Unknown event type: ${type}`)
                };
        }
    } catch(error) {
        return {
            statusCode: 500,
            body: JSON.stringify(`DynamoDB Operation Error for type
"${type}": ${error.message}`)
        };
    }
};
```

store_Classroom_Data

```
SELECT type AS type,
    classID AS classId,
    { "currentCount": occupancy } AS data
FROM 'IRsensor_Class/node-red'
WHERE type = 'classroom_update'
```

store_OccupancyEvent_Data

```
SELECT type AS type,
```

DynamoDBtoS3Exporter

```
import {
    S3Client,
    PutObjectCommand
} from "@aws-sdk/client-s3";

import {
    unmarshall
} from "@aws-sdk/util-dynamodb";

const s3 = new S3Client({});
const BUCKET_NAME = 'utility-classroom-data-lake';

export const handler = async (event) => {
    for (const record of event.Records) {
        if (record.eventName === 'INSERT' || record.eventName === 'MODIFY')
{
            const newImage = record.dynamodb.NewImage;
            const recordData = unmarshall(newImage);
            const tableName = record.eventSourceARN.split('/')[1];
            const s3Key =
`${tableName}/${recordData.classId}/${Date.now()}.json`;
            const params = {
                Bucket: BUCKET_NAME,
                Key: s3Key,
                Body: JSON.stringify(recordData),
                ContentType: 'application/json'
            };

            try {
                const command = new PutObjectCommand(params);
                await s3.send(command);
                console.log(`Successfully uploaded to S3:
s3://${BUCKET_NAME}/${s3Key}`);
            } catch (error) {
```

ClassroomDataFetcher

```
import boto3
import time
import json

def lambda_handler(event, context):
    athena_client = boto3.client('athena')
    s3_bucket = 'utility-classroom-data-lake'
    athena_database = 'classroom_analytics'
    athena_table = 'classroom_classroom'

    sql_query = f"""
        WITH ranked_data AS (
            SELECT
                classId,
                currentCount,
                maximumCapacity,
                ROW_NUMBER() OVER (
                    PARTITION BY classId
                    ORDER BY
                        CASE
                            -- ISO 8601 format
                            WHEN timestamp LIKE '%T%' THEN
from_iso8601_timestamp(timestamp)
                            -- Custom format with slashes
                            WHEN timestamp LIKE '%/%' THEN
date_parse(timestamp, '%d/%m/%Y, %H:%i:%s.%f')
                            -- Custom format with dashes
                            WHEN timestamp LIKE '%-%' THEN
date_parse(timestamp, '%d-%m-%Y, %H:%i:%s.%f')
                            -- NEW: Explicitly handle and ignore the
malformed data format
                            WHEN timestamp LIKE '("(%)"%' THEN NULL
                            -- Fallback for any other bad data
                            ELSE NULL
```

```
            classId,
            currentCount,
            maximumCapacity
        FROM
            ranked_data
        WHERE
            rn = 1;

    """

    try:
        response = athena_client.start_query_execution(
            QueryString=sql_query,
            QueryExecutionContext={'Database': athena_database},
            ResultConfiguration={'OutputLocation': f's3://{s3_bucket}/query-
results/'}
        )

        query_execution_id = response['QueryExecutionId']

        while True:
            query_status =
athena_client.get_query_execution(QueryExecutionId=query_execution_id)
            state = query_status['QueryExecution']['Status']['State']

            if state in ['SUCCEEDED', 'FAILED', 'CANCELLED']:
                break
            time.sleep(1)

        if state == 'FAILED':
            error_message =
query_status['QueryExecution']['Status']['StateChangeReason']
            print(f"Athena query failed: {error_message}")
            return {
                'statusCode': 500,
                'body': json.dumps({'error': 'Athena query failed'}),
                'headers': {
```

```
            for row in results_response['ResultSet']['Rows'][1:]:
                data_list = [d.get('VarCharValue') for d in row['Data']]

                if len(data_list) >= 3 and all(d is not None for d in
data_list[:3]):
                    try:
                        class_id = data_list[0]
                        current_count = int(data_list[1])
                        maximum_capacity = int(data_list[2])
                        results.append({'classId': class_id, 'currentCount':
current_count, 'maximumCapacity': maximum_capacity})
                    except (ValueError, TypeError) as e:
                        print(f"Error parsing row: {data_list}. Error: {e}")

        return {
            'statusCode': 200,
            'body': json.dumps(results),
            'headers': {
                'Content-Type': 'application/json',
                'Access-Control-Allow-Origin': '*'
            }
        }

    except Exception as e:
        print(f"General error: {e}")
        return {
            'statusCode': 500,
            'body': json.dumps({'error': str(e)}),
            'headers': {
                'Content-Type': 'application/json',
                'Access-Control-Allow-Origin': '*'
            }
        }


# import boto3
# import time
```

```
#           classId,
#           currentCount,
#           maximumCapacity,
#           timestamp
#         FROM
#           "{athena_database}"."{athena_table}"
#         WHERE
#           timestamp IS NOT NULL
#           AND timestamp NOT LIKE '%(%'  -- Exclude entries like ("Monday",
"09:05")
#           AND timestamp NOT LIKE '%Monday%'  -- Extra safety
#           AND timestamp LIKE '%/%'  -- Must contain forward slash
#           AND timestamp LIKE '%, %'  -- Must contain comma and space
#           AND LENGTH(timestamp) > 10  -- Must be reasonable length
#       )
#         SELECT
#           T1.classId,
#           T1.currentCount,
#           T1.maximumCapacity
#         FROM
#           valid_timestamps AS T1
#         INNER JOIN (
#           SELECT
#             classId,
#             MAX(date_parse(timestamp, '%d/%m/%Y, %H:%i:%s.%f')) AS
latest_timestamp
#           FROM
#             valid_timestamps
#           GROUP BY
#             classId
#         ) AS T2 ON T1.classId = T2.classId
#             AND date_parse(T1.timestamp, '%d/%m/%Y, %H:%i:%s.%f') =
T2.latest_timestamp;
#     """

#     try:
#         response = athena_client.start_query_execution(
```

```
#             state = query_status['QueryExecution']['Status']['State']

#             if state in ['SUCCEEDED', 'FAILED', 'CANCELLED']:
#                 break
#             time.sleep(1)

#         if state == 'FAILED':
#             error_message =
query_status['QueryExecution']['Status']['StateChangeReason']
#             return {'statusCode': 500, 'body': json.dumps({'error':
'Athena query failed'})}

#         results_response =
athena_client.get_query_results(QueryExecutionId=query_execution_id)

#         results = []
#         if 'ResultSet' in results_response and 'Rows' in
results_response['ResultSet']:
#             for row in results_response['ResultSet']['Rows'][1:]:
#                 data_list = [d.get('VarCharValue') for d in row['Data']]

#                 if len(data_list) >= 3 and all(d is not None for d in
data_list[:3]):
#                     try:
#                         class_id = data_list[0]
#                         current_count = int(data_list[1])
#                         maximum_capacity = int(data_list[2])
#                         results.append({'classId': class_id,
'currentCount': current_count, 'maximumCapacity': maximum_capacity})
#                     except (ValueError, TypeError) as e:
#                         print(f"Error parsing row: {data_list}. Error:
{e}")

#         return {
#             'statusCode': 200,
#             'body': json.dumps(results),
#             'headers': {
```

fetchHistoricalData

```
import boto3
import time
import json
from urllib.parse import unquote

def lambda_handler(event, context):
    athena_client = boto3.client('athena')
    s3_bucket = 'utility-classroom-data-lake'
    athena_database = 'classroom_analytics'
    athena_table = 'classroom_classroom'

    try:
        query_params = event.get('queryStringParameters', {})
        class_id = query_params['classId']
        start_time_str = unquote(query_params['startTime'])
        end_time_str = unquote(query_params['endTime'])
    except (KeyError, TypeError):
        return {
            'statusCode': 400,
            'body': json.dumps({'error': 'Missing required query parameters:
classId, startTime, or endTime.'}),
            'headers': {
                'Content-Type': 'application/json',
                'Access-Control-Allow-Origin': '*'
            }
        }

    sql_query = f"""
    SELECT
        timestamp,
        currentCount,
        maximumCapacity
    FROM
        "{athena_database}"."{athena_table}"
```

```
            try(date_parse(timestamp, '%d/%m/%Y, %H:%i:%s'))
        ) <= COALESCE(
            try(date_parse('{end_time_str}', '%d/%m/%Y, %H:%i:%s.%f')),
            try(date_parse('{end_time_str}', '%d/%m/%Y, %H:%i:%s'))
        )
    ORDER BY
        COALESCE(
            try(date_parse(timestamp, '%d/%m/%Y, %H:%i:%s.%f')),
            try(date_parse(timestamp, '%d/%m/%Y, %H:%i:%s'))
        ) ASC;
    """

    try:
        response = athena_client.start_query_execution(
            QueryString=sql_query,
            QueryExecutionContext={'Database': athena_database},
            ResultConfiguration={'OutputLocation': f's3://{s3_bucket}/query-
results/'}
        )

        query_execution_id = response['QueryExecutionId']

        while True:
            query_status =
athena_client.get_query_execution(QueryExecutionId=query_execution_id)
            state = query_status['QueryExecution']['Status']['State']
            if state in ['SUCCEEDED', 'FAILED', 'CANCELLED']:
                break
            time.sleep(1)

        if state == 'FAILED':
            error_message =
query_status['QueryExecution']['Status']['StateChangeReason']
            print(f"Athena query failed: {error_message}")
            return {
                'statusCode': 500,
                'body': json.dumps({'error': f'Athena query failed:
```

```
        if 'ResultSet' in results_response and 'Rows' in
results_response['ResultSet']:
            for row in results_response['ResultSet']['Rows'][1:]:
                data_list = [d.get('VarCharValue') for d in row['Data']]
                if len(data_list) >= 3 and all(d is not None for d in
data_list):
                    try:
                        timestamp_str = data_list[0]
                        current_count = int(data_list[1])
                        maximum_capacity = int(data_list[2])
                        results.append({
                            'timestamp': timestamp_str,
                            'currentCount': current_count,
                            'maximumCapacity': maximum_capacity
                        })
                    except (ValueError, TypeError) as e:
                        print(f"Error parsing row: {data_list}. Error: {e}")

        return {
            'statusCode': 200,
            'body': json.dumps(results),
            'headers': {
                'Content-Type': 'application/json',
                'Access-Control-Allow-Origin': '*'
            }
        }

    except Exception as e:
        print(f"General error: {e}")
        return {
            'statusCode': 500,
            'body': json.dumps({'error': str(e)}),
            'headers': {
                'Content-Type': 'application/json',
                'Access-Control-Allow-Origin': '*'
            }
        }
```

SendFacultyAlerts

```
import { SNSClient, PublishCommand } from "@aws-sdk/client-sns";
import { unmarshall } from "@aws-sdk/util-dynamodb";

const snsClient = new SNSClient({ region: "ap-southeast-1" });
const topicArn = "arn:aws:sns:ap-southeast-
1:019662059667:FacultyAlertsTopic";

export const handler = async (event) => {
  try {
    for (const record of event.Records) {
      const eventName = record.eventName; // INSERT, MODIFY, REMOVE
      const newImage = record.dynamodb.NewImage ?
unmarshall(record.dynamodb.NewImage) : null;

      if (!newImage) continue;

      // Extract values
      const classId = newImage.classId || newImage.classroomId || "Unknown";
      const fullCapacity = parseInt(newImage.maximumCapacity, 10) || 0;
      const currentCapacity = parseInt(newImage.currentCount, 10) || 0;
      const timestamp = newImage.timestamp || new Date().toISOString();

      // Only trigger alert if overloaded
      if (currentCapacity > fullCapacity) {
        const message = `Classroom ID: ${classId}
Full capacity: ${fullCapacity}
Current capacity: ${currentCapacity}
Status: The class is overloaded
Timestamp: ${timestamp}`;

        console.info("Publishing OVERLOAD message to SNS:", message);
```

```
        );
      }
    }

    return { statusCode: 200, body: "Processing complete." };
  } catch (err) {
    console.error("Error publishing SNS message", err);
    return { statusCode: 500, body: "Failed to send notification" };
  }
};
```

FacultyAlertsTopic Access Policy

```
{

   "Version": "2008-10-17",

   "Id": "__default_policy_ID",

   "Statement": [

   {

      "Sid": "__default_statement_ID",

     "Effect": "Allow",

     "Principal": { "AWS": "*" },

    "Action": [ "SNS:Publish", "SNS:RemovePermission",
"SNS:SetTopicAttributes", "SNS:DeleteTopic", "SNS:ListSubscriptionsByTopic",
"SNS:GetTopicAttributes", "SNS:AddPermission", "SNS:Subscribe"

    ],

   "Resource": "arn:aws:sns:ap-southeast-1:019662059667:FacultyAlertsTopic",

   "Condition": {

   "StringEquals": {

   "AWS:SourceAccount": "019662059667"

     } }}]}
```